Security Assessment Report - Juice Shop

Analysis of security architecture and threats

Generated by appsec-advisor v0.6.0-beta.5 (analysis v5)

Project	Juice Shop
Repository	`https://github.com/juice-shop/juice-shop.git`
Homepage	`https://github.com/juice-shop/juice-shop`

Changelog

Append-only history of assessment runs. Most recent first.

Version	Date	Mode	Depth	Reasoning	Baseline → Current	Δ Threats	Code	Note
v1	2026-10-17 21:17 CEST	full	standard	sonnet-economy	*(initial)*	82 total	-	first full scan

Table of Contents

- Management Summary . . . 4
- Critical Findings by Root Cause . . . 17

1. System Overview . . . 18
   - Scope . . . 18
   - Identified Actors . . . 18
   - Trust Boundaries . . . 19
2. Architecture Diagrams . . . 21
   - 2.1 System Context . . . 21
   - 2.2 Container Architecture . . . 21
   - 2.3 Components . . . 22
3. Attack Walkthroughs . . . 27
   - 3.1 SQL injection in Search . . . 27
   - 3.2 SQL injection in login query . . . 28
   - 3.3 Missing Object Ownership Checks in Order . . . 29
   - 3.4 Broken authorization in Data Export . . . 30
   - 3.5 User registration accepts privileged fields in Juice Shop Express Server . . . 31
   - 3.6 Hardcoded Secrets in Source . . . 32
   - 3.7 Request data evaluated by notevil in vm sandbox . . . 33
4. Assets . . . 34
5. Attack Surface . . . 35
   - 5.1 Unauthenticated Entry Points (69) . . . 35
   - 5.2 Authenticated Entry Points (40) . . . 37
6. Security Architecture . . . 39
   - 6.1 Security Control Overview . . . 40
   - 6.2 Identity and Authentication Controls . . . 41
   - 6.3 Session and Token Controls . . . 48
   - 6.4 Authorization Controls . . . 51
   - 6.5 Query Construction and Data Access Controls . . . 53

- 6.6 Input Boundary Validation Controls . . . 55
- 6.7 Output Encoding and Rendering Controls . . . 59
- 6.8 Browser and Cross-Origin Controls . . . 60
- 6.9 Cryptography Secrets and Data Protection . . . 61
- 6.10 File Parser and Outbound Request Controls . . . 63
- 6.11 Operations Runtime and Supply Chain Controls . . . 65
- 6.12 Real-time and Not Applicable Controls . . . 70
- 6.13 Defense-in-Depth Summary . . . 70

7. Weakness Register . . . 71
8. Findings Register . . . 89
9. Abuse Cases . . . 129
10. Mitigation Register . . . 138
11. Out of Scope . . . 193

- Not Covered by This Method . . . 193
- Excluded from This Assessment . . . 193
- Appendix: Run Statistics . . . 193
- Appendix A - Attack Vector Taxonomy . . . 195

Management Summary

Verdict

● **Critical security concerns** - This web shop lets unauthenticated attackers sign in as any user, including administrators, and reach server-side code execution. It should run only as an isolated training target, never with real customer data or exposure beyond a controlled network.

Risk distribution: ● Critical: 3 · ● High: 14 · ● Medium: 24 · ● Low: n/a · **Total: 41**
Assessment evidence: 32 of 39 finding(s) confirmed in code
Weakness classes: 41 (35 implementation, 6 design) - see §7 Weakness Register

Security concerns behind this assessment:

- **Login bypass and database takeover** — SQL injection in login and product search lets anonymous internet users sign in as anyone, including administrators, and read database contents. *(● F-013, ● F-007 → W-001)*
- **Other customers' records read and changed** — Missing ownership checks (IDOR) let any signed-in customer read or modify another customer's baskets, orders and exported data. *(● F-001, ● F-009, ● F-006 → W-002)* — ✓ cited finding in a code-verified chain
- **Anyone can forge logins** — Signing secrets committed to source code let anyone with repository access mint valid sessions with any role. *(● F-005, ● F-016 → W-007, W-008)* — ✓ cited finding in a code-verified chain
- **Self-made administrator accounts** — Registration accepts privileged fields (mass assignment), so a new visitor can create an account with administrator rights. *(● F-038 → W-023)* — ✓ cited finding in a code-verified chain
- **Server takeover through code execution** — User-supplied text is evaluated as code in the B2B order and profile handlers, risking arbitrary code execution on the server. *(● F-048, ● F-033 → W-003)* — ✓ cited finding in a code-verified chain
- **Active login stolen from customers** — Stored cross-site scripting (XSS) combined with session tokens kept in browser storage lets attackers hijack a victim's account. *(● F-004, ● F-003 → W-005, W-006)*

Fixing these needs structural changes to data access, authorization and secret handling before any use beyond an isolated training setting.

About this assessment: An AI-assisted threat model derived from the implementation of Juice Shop. It reconstructs the implemented architecture (9 components and 3 external services, see §2) and identifies threats and control gaps in it.

Method and limits: Automated static analysis of code and configuration, not a pentest or a team threat-modeling session; Low and Informational findings not reported (threshold: medium) - see §11 Out of Scope.

Remediation Priorities

The root causes behind the findings, ranked by severity, each with the measures to do first. The lasting fix under a root cause removes it and every finding it causes. Every mitigation that fixes a Critical finding is listed. Details in the §7 Weakness Register and the §10 Mitigation Register.

Root cause → lasting fix	Do first	Resolves	Effort
● W-001 - Database access relies on concatenated queries → *provide one parameterised repository or query-builder path and prohibit application-value interpolation in database queries*	● M-030 - Parameterize the login query with bind replacements	● F-013	Low
	● M-029 - Use parameterized database queries (F-007)	● F-007	Medium
● W-002 - Authorization is implemented route by route → *enforce authorization through a shared server-side policy layer and make ownership and tenant scope mandatory inputs to data access*	● M-023 - Enforce object-level (ownership) authorization (F-001)	● F-001 ● F-009	Medium
	◕ M-028 - Check basket ownership before returning	● F-006	Low
	◕ M-045 - Enforce server-side authorization on every endpoint (F-036)	● F-036	Low
● W-007 - Secrets are committed to source instead of a managed store → *move every secret to a managed secret store or injected environment configuration, rotate the exposed values, and add secret-scanning to CI*	◕ M-027 - Load signing keys from a secret store	● F-005	Medium
● W-003 - Application data crosses into executable code → *replace evaluation of application values with a data parser or a fixed set of operations whose arguments remain data*	◕ M-044 - Remove eval of username in profile rendering	● F-033	Low
● W-023 - Privileged attributes lack an enforced write allowlist → *define operation-specific write schemas and assign*	◕ M-047 - Allowlist client-controlled fields	● F-038	Low

Root cause → lasting fix	Do first	Resolves	Effort
privileged attributes exclusively through server-side policy			
● W-006 - Frontend rendering lacks enforced output encoding → *use framework-safe rendering by default and isolate any required raw HTML behind one reviewed sanitisation and Trusted Types policy*	◕ M-024 - Remove sanitizer bypass and render feedback as text	● F-002	Low
	◕ M-025 - Remove sanitizer bypass and render IP as text	● F-003	Low
	◕ M-040 - Encode output instead of bypassing the framework sanitizer (F-028)	● F-028	Low

15 more P1/P2 mitigations and 69 P3/P4 backlog items in the §10 Mitigation Register. ● P1 · ◕ P2 · ◑ P3 · ○ P4

Open Questions for the Team

The code cannot answer these questions; the people who own the business and the deployment can. Each question states under it what its answer changes.

- *Which operations on other users' or tenants' data in Juice Shop Express Server or Authentication & Session Surface are intended for support staff or administrators?* (W-002: F-009, F-006, F-034 +3 more) Each access confirmed as intended narrows the finding to the remaining routes; if none are intended, every listed route is a live authorization gap.
- *Which sensitive business records can the production database account used by Juice Shop Express Server or Authentication & Session Surface access?* (W-001: F-007, F-013) Personal or regulated data in reach raises both the breach impact and any notification duty; a narrowly scoped account limits the blast radius.
- *If an attacker runs code in Juice Shop Express Server, which other services, secrets or credentials can that process reach?* (F-033, F-048 (unproven)) Every reachable service or credential extends this from one component to a wider compromise and raises the priority of isolating that process; an isolated process contains it.

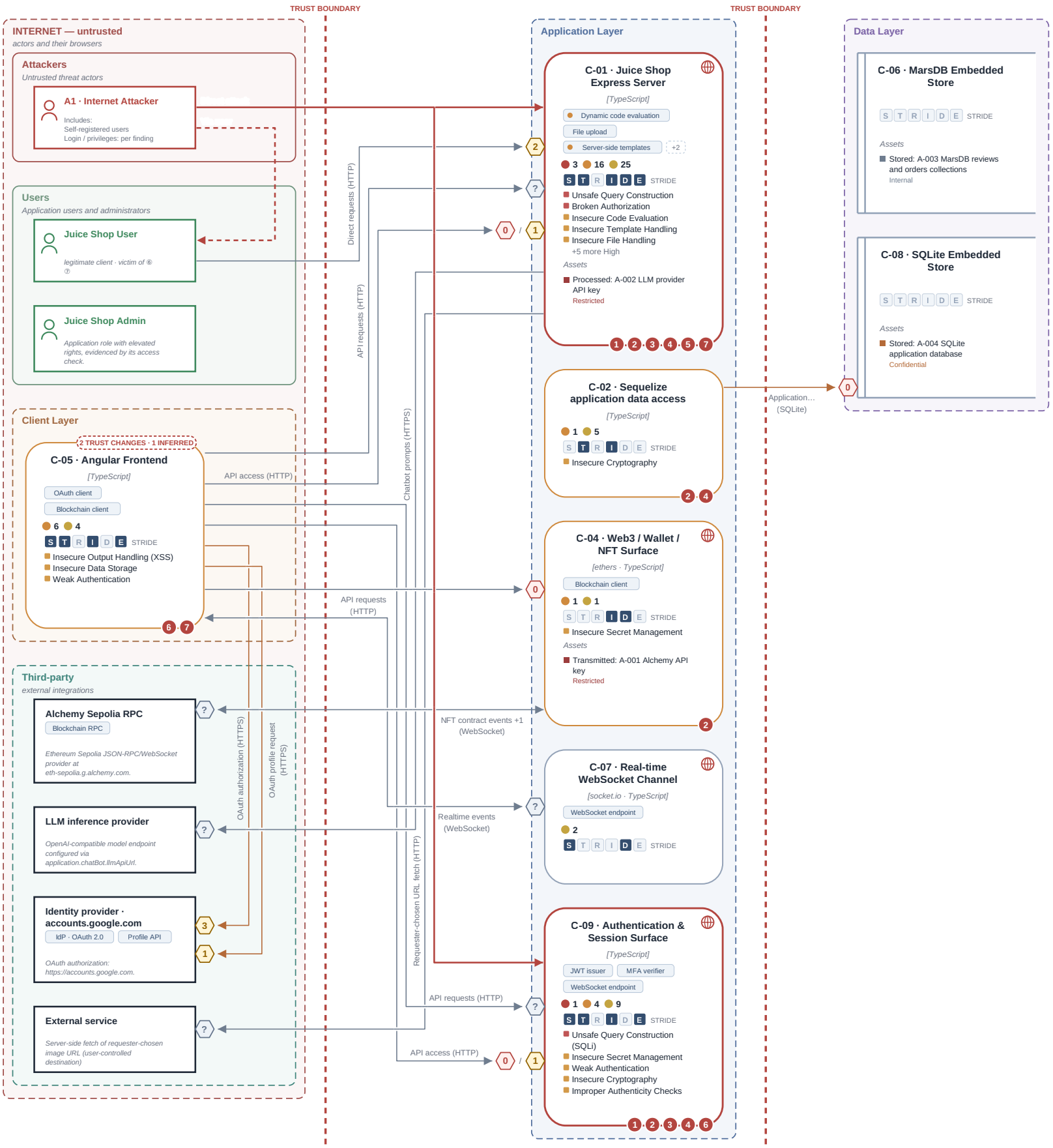
Security Posture & Top Threats

Figure 1a - Runtime Architecture and Threat Overview

At runtime, Juice Shop has 8 components in 3 layers and exchanges data with 4 external services. Figure 1a shows these data flows and where each of the 7 attack scenarios below begins. The complete boundary catalogue remains in §1 Trust Boundaries.

Figure 1a — Runtime Architecture and Threat Overview

Juice Shop · 8 components · 17 data flows · 12 trust boundaries (3 inferred) · 41 threats in the model



Notation

- external entity (actor, third party)
- process (component); double line = complex
- data store (evidenced asset locations)
- [ ] framework · language; data store: engine
- data flow · classification colour · two heads = bidirectional
  - Restricted · Confidential · Internal · Public
- trust boundary crossed between these columns
- trust change within a column (dashed: inferred)
- dashed outline = zone, not itself a boundary
- solid = direct attack · dashed = via user
- ⊕ internet-exposed entry point
- S T R I D E STRIDE

STRIDE-per-element: filled = threats found in that class

■ Unsafe Query Construction (SQLi)
All Critical · High fills to 5 · +N = omitted High categories

● Critical ● High ● Medium

counts on a node = threats per severity · ①–⑨ = attack scenario
badges also count scenarios that hit an asset stored there

Security-relevant capabilities / service roles

Dynamic code evaluation — function or service role

Dynamic code evaluation — dot: most severe linked finding

Dynamic code evaluation = runs runtime data as program code
Most security-critical functions always shown (at most 5), others up to 3 labels per element: most critical first, then most severe linked finding; +N = further labels, listed under Further capabilities.
Functions may go undetected: a missing label does not mean absence.

Attack scenarios — by actor

Internet Attacker

1	Inject Input into Server Interpreters	Critical
2	Bypass or Forge Authentication	High
3	Bypass Authorization to Gain Access	Critical
4	Access Exposed Data and Secrets	High
5	User input evaluated as server code	High

Internet Attacker → User (victim)

6	Inject Scripts into Victim Browsers	High
7	Abuse Cross-Site Requests	Medium

Further capabilities

Labels behind +N on their element:

C-01: WebSocket endpoint, Blockchain client

Authentication

- 0 **No Authentication** — The recipient performs no authentication check on this connection.
- 1 **Bearer token** — The recipient checks the token supplied with the request.
- 2 **Session cookie** — A session cookie identifies the caller.
- 3 **OAuth 2.0 · implicit** — Delegated access; only the evidenced protocol step is covered.
- ? **Unknown** — Authentication could not be determined from the code.

Numbered hexagon: authentication · circle: attack scenario
0: no authentication · ?: unknown
Red: absent / unsafe · yellow: standard / limited · green: stronger
Method properties, not proof of a secure implementation.
/: alternatives · →: successive checks (see access label)

Build pipeline, not drawn in this runtime view: GitHub Actions, GitLab CI. ® A2 · Supply-Chain / Build Attacker → Figure 1b.

Role access and finding assignments are listed in Identified Actors.

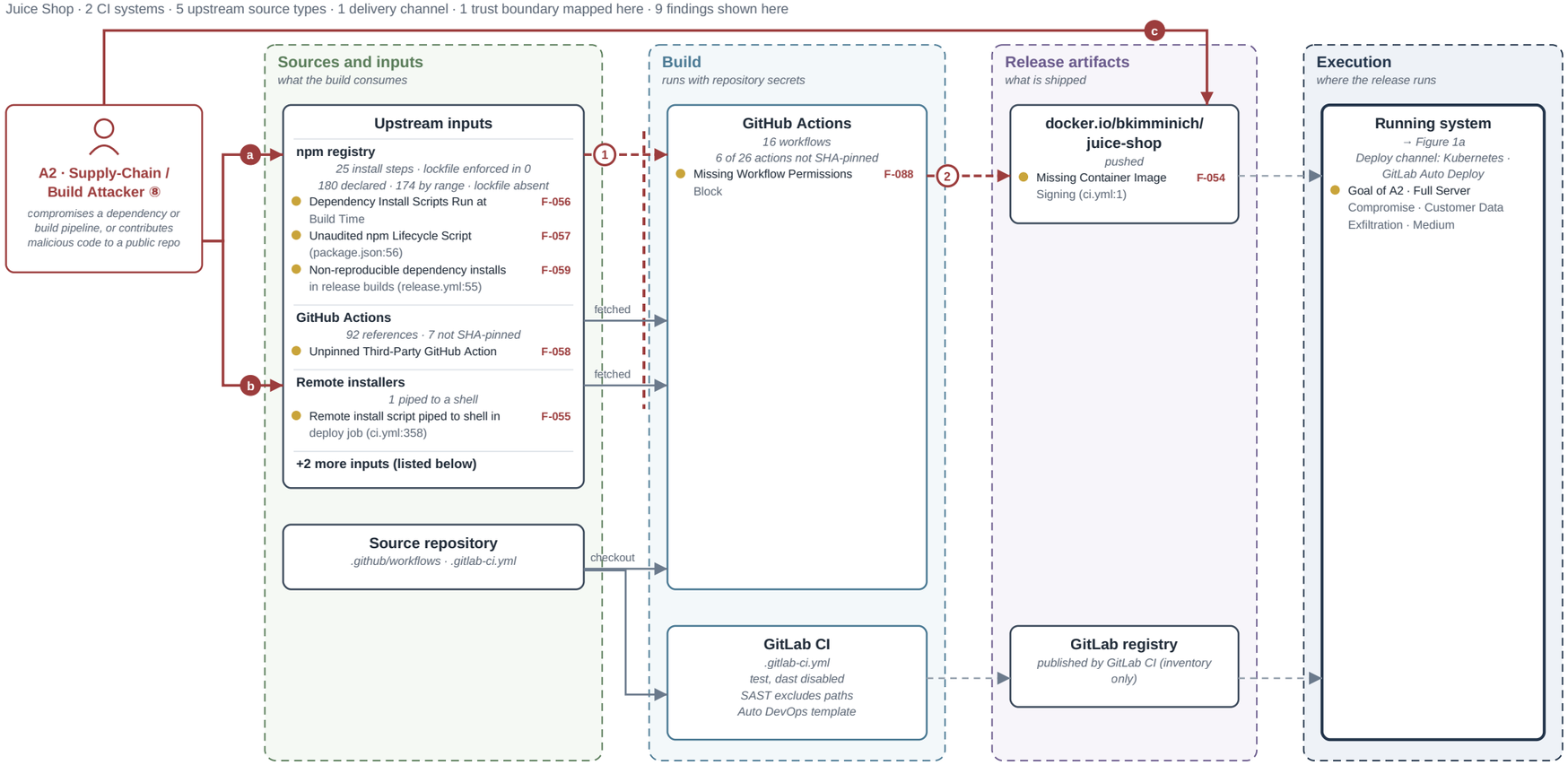
Actor grouping. Regular self-registered users are grouped with anonymous internet attackers because registration is open. Each finding retains its login and privilege requirements.

Figure 1b - Supply Chain and Build

Juice Shop builds in GitHub Actions, GitLab CI. Figure 1b shows the inputs, build systems and release artifacts the repository evidences, with 9 findings attached to them. The highlighted path follows ● F-056 — Dependency Install Scripts Run at Build Time as far as the repository evidences it. The control assessment is in §6.11.

Figure 1b — Supply Chain and Build

Juice Shop · 2 CI systems · 5 upstream source types · 1 delivery channel · 1 trust boundary mapped here · 9 findings shown here



Entry points of A2 · Supply-Chain / Build Attacker

a **Manipulated dependency** — Medium
npm registry: 25 install steps · lockfile enforced in 0
Findings: F-056, F-057, F-059

b **Manipulated CI input** — Medium
Remote installers: 1 piped to a shell
Findings: F-055, F-058, F-060

c **Replaced release artifact** — Medium
docker.io/bkimminich/juice-shop: pushed
Findings: F-054

Path of F-056, step by step:

1. npm registry → GitHub Actions (Dockerfile:4)
2. GitHub Actions → docker.io/bkimminich/juice-shop (.github/workflows/ci.yml:338)

The connection to the running system is not evidenced in the repository.

Findings without an evidenced CI owner: F-087

Not drawn (upstream): Base images, CI includes

Notation

- evidenced flow (checkout, fetch, push)
- relationship not evidenced in the repository
- attack route of the build-time attacker
- highlighted path of the most severe entry
- trust boundary crossed by these flows
- a — entry point (a, b, ...)
- 1 — step on the highlighted path
- High finding
- Medium finding or decision

Boxes and flows come from repository files: workflow uses:, Dockerfile FROM, install and push steps. Known CVEs in dependencies are runtime findings and appear in Figure 1a.

Figure 2 - Attack Routes and Impact

Each numbered route names one example finding from the corresponding Top Threats group with its access prerequisite, the register weakness it is linked to, and the group's potential business harm, which depends on deployment and affected assets. W-IDs in parentheses link the example to an existing weakness. The attack step, affected component and technical consequence are listed in the findings register. Red arrows indicate attacks; dashed red arrows involve a victim; grey arrows lead to potential harm.



Threat actors. The actors below drive the numbered attack paths in the figures above. The **Juice Shop User** is the *victim* of client-side attacks (XSS / CSRF), not an attacker. Figure 2 marks victim interaction on the applicable attack routes.

- **Juice Shop User** — legitimate end user; target of client-side attacks; target of ⑥ Output Encoding / Cross-Site Scripting, ⑦ CSRF / Permissive CORS.
- **Internet Attacker** — can self-register a regular account; drives ① Insecure Query Construction & Data Access, ② Hardcoded Secrets & Weak Cryptography, ③ Broken Authorization & Access Control, ④ Sensitive File & Secret Exposure, ⑤ Remote Code Execution (unsafe eval), ⑥ Output Encoding / Cross-Site Scripting.
- **Supply-Chain / Build Attacker** — compromises a dependency or build pipeline, or contributes malicious code to a public repo; drives ⑧ Build Pipeline & Dependency Integrity.

8 structural threats, grouped by weakness class - each row is one threat, not one finding. *Threat Description* states the general architectural weakness (STRIDE in brackets); *Findings* lists the concrete instances, each linked to §8 Findings Register with its component; *Risk & Impact* combines severity with business consequence.

#	Threat Description	Findings (→ Component)	Risk & Impact	Fix
①	**Insecure Query Construction & Data Access** *(T·I)* Request values are concatenated into database queries and templates, letting attackers change query logic to bypass authentication or read data.	● F-007 - SQL injection → C-01 Juice Shop Express Server ● F-013 - SQL injection in login query → C-09 Authentication & Session Surface ● F-025 - Input in executable NoSQL predicate → C-01 Juice Shop Express Server ● F-026 - Input in executable NoSQL predicate → C-01 Juice Shop Express Server ● F-031 - XML external entity expansion in upload parser → C-01 Juice Shop Express Server ● F-027 - Input compiled as template source → C-01 Juice Shop Express Server	● **Critical** Customer Data Exfiltration · Full Admin Takeover	● M-029 — Use parameterized database queries (F-007) ● M-030 — Parameterize the login query with bind replacements
②	**Hardcoded Secrets & Weak Cryptography** *(S·E)* Weak token verification and committed signing secrets let attackers forge credentials and impersonate other users.	● F-005 - Hardcoded Secrets in Source → C-09 Authentication & Session Surface ● F-016 - JWT verify call missing algorithms allowlist → C-01 Juice Shop Express Server ● F-021 - Password reset by security answer only → C-09 Authentication & Session Surface ● F-015 - Chat identity taken from unverified JWT → C-01 Juice Shop Express Server ● F-032 - Passwords hashed with unsalted fast `MD5` → C-02 Sequelize application data access ● F-040 - Hard-coded cookie signing secret → C-01 Juice Shop Express Server ● F-073 - Security answers HMAC with hardcoded key	● **High** Full Admin Takeover · Customer Session Hijack	◕ M-027 — Load signing keys from a secret store ◕ M-032 — Enforce JWT signature and algorithm verification (F-016)

#	Threat Description	Findings (→ Component)	Risk & Impact	Fix
		→ C-02 Sequelize application data access		
③	**Broken Authorization & Access Control** *(E·I)* Missing ownership and role checks let ordinary users access other users' records or obtain administrative privileges.	● F-001 - Missing Object Ownership Checks → C-01 Juice Shop Express Server ● F-006 - Systemic IDOR Missing Ownership Checks → C-01 Juice Shop Express Server ● F-009 - Broken authorization → C-01 Juice Shop Express Server ● F-036 - Missing authorization on product update route → C-01 Juice Shop Express Server ● F-038 - User registration accepts privileged fields → C-01 Juice Shop Express Server ● F-017 - Password change without current password → C-01 Juice Shop Express Server ● F-034 - Sensitive Routes Registered Without Authentication Middleware → C-01 Juice Shop Express Server ● F-037 - Sensitive Routes Registered Without Authentication Middleware → C-01 Juice Shop Express Server ● F-069 - Order history lookup by vowel-masked email → C-01 Juice Shop Express Server ● F-086 - Chatbot coupon discount unbounded server-side → C-01 Juice Shop Express Server ● F-090 - Any authenticated user can list all accounts → C-09 Authentication & Session Surface	● **Critical** Full Admin Takeover · Customer Data Exfiltration	● M-023 — Enforce object-level (ownership) authorization (F-001) ◕ M-028 — Check basket ownership before returning
④				

#	Threat Description	Findings (→ Component)	Risk & Impact	Fix
	Sensitive File & Secret Exposure *(I)* Unchecked file paths and archive entries let attackers read or overwrite files outside the intended directories.	● F-022 - Path Traversal → C-01 Juice Shop Express Server ● F-023 - Zip Slip path traversal during archive extraction → C-01 Juice Shop Express Server ● F-030 - Unauthenticated log directory listing → C-01 Juice Shop Express Server ● F-046 - Substring redirect allowlist check → C-09 Authentication & Session Surface ● F-062 - Null byte extension allowlist bypass in FTP file → C-01 Juice Shop Express Server ● F-063 - JSONP and arbitrary field selection on current user → C-01 Juice Shop Express Server ● F-064 - Unauthenticated memories listing joins user records → C-01 Juice Shop Express Server ● F-067 - Unauthenticated log directory listing and download → C-01 Juice Shop Express Server ● F-068 - Unauthenticated encryption key directory exposure → C-01 Juice Shop Express Server ● F-070 - Server-side request forgery → C-01 Juice Shop Express Server ● F-071 - Payment card numbers stored in plaintext → C-02 Sequelize application data access ● F-072 - TOTP secrets stored in plaintext → C-02 Sequelize application data access ● F-075 - Full user record with password hash in JWT	● **High** Customer Data Exfiltration	◕ M-036 — Constrain file paths to a safe base directory (F-022) ◕ M-037 — Constrain zip entries to the extraction directory

#	Threat Description	Findings (→ Component)	Risk & Impact	Fix
		→ C-09 Authentication & Session Surface		
⑤	**Remote Code Execution (unsafe eval)** *(E)* Untrusted input reaches code-evaluation routines on the server, allowing attackers to run commands in the application process.	● F-033 - Code injection → C-01 Juice Shop Express Server ● F-048 - Request data evaluated by notevil in vm sandbox → C-01 Juice Shop Express Server	● **High** Full Server Compromise	◕ M-044 — Remove eval of username in profile rendering ◑ M-057 — Remove dynamic evaluation of order lines
⑥	**Output Encoding / Cross-Site Scripting** *(T·I)* Unsafe HTML rendering lets injected script run in a victim's browser and use session tokens held in browser storage.	● F-002 - XSS via Trusted HTML Binding → C-05 Angular Frontend ● F-003 - Angular Sanitizer Bypass XSS → C-05 Angular Frontend ● F-004 - Session Token Stored in `localStorage` → C-05 Angular Frontend ● F-028 - DOM XSS → C-05 Angular Frontend ● F-061 - Unsanitised login IP header stored for later → C-09 Authentication & Session Surface ● F-113 - Session token stored in web-accessible client storage → C-05 Angular Frontend	● **High** Customer Session Hijack	◕ M-024 — Remove sanitizer bypass and render feedback as text ◕ M-025 — Remove sanitizer bypass and render IP as text
⑦	**CSRF / Permissive CORS** *(S·T)* a permissive CORS policy plus missing anti-CSRF tokens let any external page issue authenticated state-changing requests in the victim's session.	● F-042 - OAuth implicit flow without state or nonce → C-05 Angular Frontend ● F-049 - Cookie-authenticated profile update → C-01 Juice Shop Express Server	● **Medium** Customer Session Hijack	◑ M-051 — Use authorization code flow with PKCE and state ◑ M-058 — Add CSRF token and SameSite cookie flags
⑧	**Build Pipeline & Dependency Integrity** *(T·E)*	● F-054 - Missing Container Image Signing → C-03 CI/CD Pipeline	● **Medium** Full Server Compromise ·	◑ M-017 — Sign and verify

#	Threat Description	Findings (→ Component)	Risk & Impact	Fix
	a dependency, build input, pipeline step or published artifact can be replaced or altered, so attacker-controlled code reaches the shipped application. Path through the build: Figure 1b · controls: §6.11	● F-055 - Remote install script piped to shell in deploy job → C-03 CI/CD Pipeline ● F-056 - Dependency Install Scripts Run at Build Time → C-03 CI/CD Pipeline ● F-057 - Unaudited npm Lifecycle Script → C-03 CI/CD Pipeline ● F-058 - Unpinned Third-Party GitHub Action → C-03 CI/CD Pipeline ● F-059 - Non-reproducible dependency installs in release builds → C-03 CI/CD Pipeline ● F-060 - Unpinned floating container base images → C-03 CI/CD Pipeline ● F-087 - Container Runs as Root → C-03 CI/CD Pipeline ● F-088 - Missing Workflow Permissions Block → C-03 CI/CD Pipeline	Customer Data Exfiltration	release artifacts ◑ M-063 — Verify or remove curl-pipe Heroku CLI install

STRIDE: S spoofing · T tampering · R repudiation · I information disclosure · D denial of service · E elevation of privilege. Risk, findings, components, impact and Fix are derived deterministically; only the one-line weakness description is authored.

Verified attack chains. 7 fully viable (AC-B-003, AC-T-002, AC-T-004, AC-T-005, AC-T-007, AC-T-006, AC-B-005). These chains combine individual findings into end-to-end exploitation paths verified step-by-step against the code - see §9 Abuse Cases for the per-step breakdown and blocking mitigations.

AI / LLM Exposure

This system embeds an LLM/AI surface (an LLM/AI integration); the risks below are architectural - they follow from how untrusted input reaches the model's prompt, tools, and outputs. See **§6 Security Architecture** for the per-control detail.

- **LLM06 Excessive Agency** — The LLM is granted tool/function-calling authority without a secondary authorization boundary, so a manipulated model turn can invoke privileged actions on the user's behalf. *(C-01 — Juice Shop Express Server)*
  - ↳ ● F-015 — Chat identity taken from unverified JWT *(ASI03)*
  - ↳ ● F-086 — Chatbot coupon discount unbounded server-side *(ASI02)*

- **LLM01 Prompt Injection** — Untrusted user input reaches the LLM prompt/context without sufficient trust separation, letting an attacker override system instructions, redirect tool calls, or coerce unintended model behaviour. *(C-01 — Juice Shop Express Server)*
  - ↳ ● F-051 — Unvalidated client chat history passed to model *(ASI01)*
- **LLM05 Improper Output Handling** — LLM-generated output flows into a downstream sink (browser, shell, query, eval) without validation or encoding, turning a model completion into an injection vector. *(C-01 — Juice Shop Express Server)*
  - ↳ ● F-086 — Chatbot coupon discount unbounded server-side *(ASI02)*
- **LLM10 Unbounded Consumption** — The LLM endpoint imposes no authentication, rate, or quota boundary, so any client can drive unbounded model invocations — uncontrolled provider cost and denial of service for legitimate users. *(C-01 — Juice Shop Express Server)*
  - ↳ ● F-079 — Unbounded unauthenticated chat consumption

Operational Strengths

Operational controls rated Adequate or Partial - grouped into broad clusters (full per-control breakdown in §6). Clusters demoted to Weak by open Critical/High findings appear in §6 instead, not here.

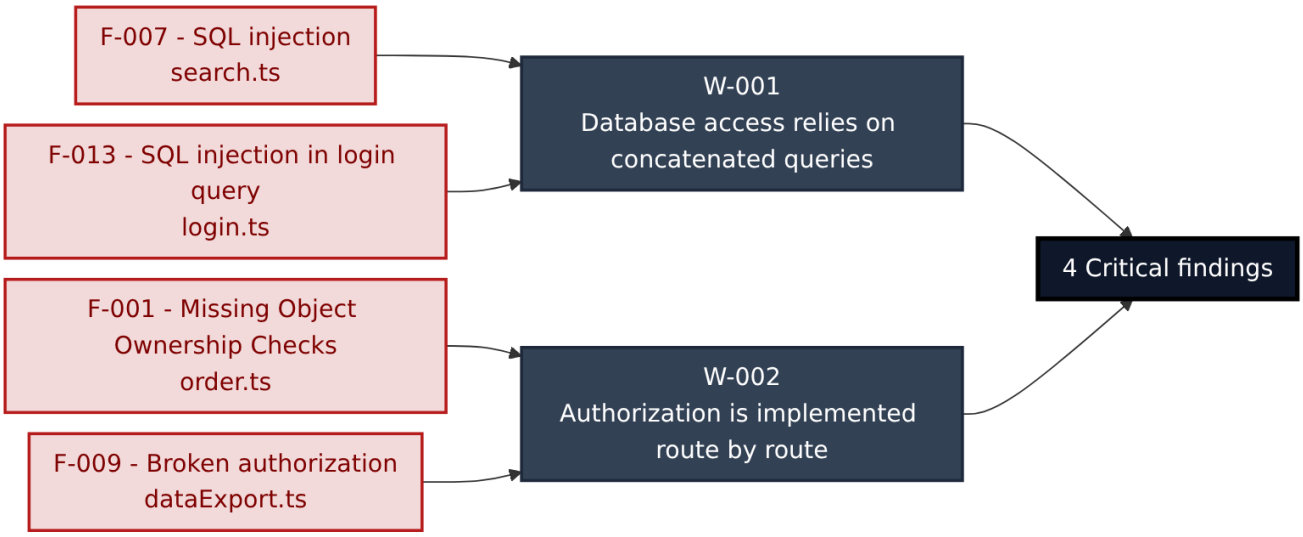
Strength	What's in Place	Effectiveness	Mitigates
Container & Supply-Chain Hardening	*Build-time and runtime hardening - minimal base image, non-root execution, dependency inventory.* Container image hardening	✓ Adequate	-
Hardened HTTP Stack	*Browser-facing HTTP hardening — security headers, cookie flags, cross-origin policy, and abuse-protection limits.* Socket.io CORS origin restriction Rate limiting	▲◌ Partial — 2 medium/low-severity finding(s) within the cluster's remit remain open — see §8 Findings Register for details.	● F-042 — OAuth implicit flow without state or nonce (login.component… ● F-049 — Cookie-authenticated profile update (routes/

Strength	What's in Place	Effectiveness	Mitigates
			updateUserPro…
Observability & Audit	*Runtime visibility - access logging, audit trails, and operational telemetry for post-incident review.* Error handling and logging for WebSocket provider Sensitive endpoint exposure (metrics, log directory, API docs)	▲◌ Partial - Coverage incomplete - see §7 control assessment.	-

Bottom line: These controls narrow specific attack surfaces but none eliminates a Critical finding on its own.

Critical Findings by Root Cause

4 of 4 Critical findings trace back to 2 root causes; fixing a root cause addresses every finding linked to it.



Structural fixes (full detail in §7 Weakness Register):

- **W-001** — Provide one parameterised repository or query-builder path and prohibit application-value interpolation in database queries *(closes F-007, F-013)*
- **W-002** — Enforce authorization through a shared server-side policy layer and make ownership and tenant scope mandatory inputs to data access *(closes F-001, F-009)*

---

1. System Overview

Probably the most modern and sophisticated insecure web application

Repository: `https://github.com/juice-shop/juice-shop.git` **Runtime:** Node.js 22 - 26

Scope

This threat model covers 9 components of juice-shop:

- Juice Shop Express Server
- Sequelize application data access
- CI/CD Pipeline
- Web3 / Wallet / NFT Surface
- Angular Frontend
- MarsDB Embedded Store
- Real-time WebSocket Channel
- SQLite Embedded Store
- Authentication & Session Surface

Not individually examined: 124 source files in 4 components had no route, lead or security signal (Sequelize application data access 11, CI/CD Pipeline 4, Angular Frontend 108, MarsDB Embedded Store 1).

Not assessed: 4 security-relevant code locations were read but could not be judged (CI/CD Pipeline 1, Web3 / Wallet / NFT Surface 1, Angular Frontend 1, Authentication & Session Surface 1); `security-assessment.yaml` records each with its reason under `meta.undecided_targets`.

Not assigned to any component: 134 program files belong to no component and were not analysed (`data/` 97, `lib/` 21, `frontend/` 5, `(repository root)` 4, `rsn/` 4, `models/` 1, `routes/` 1, `views/` 1).

Out of scope: third-party hosted dependencies, browser runtime, operating-system kernel, and the underlying network infrastructure.

Basis: An automated, AI-assisted threat model built from the repository's source code and configuration - static analysis only, no dynamic or penetration testing. It describes the system as built, not as designed.

Identified Actors

Figure 1a draws **Internet Attacker** as attacker and **Juice Shop User** and **Juice Shop Admin** as legitimate roles of Juice Shop. Figure 1b draws **Supply-Chain / Build Attacker** and the build it attacks. Each row gives that actor's access and the findings attributed to it.

Actor	Type	Access	Scenarios	Attributed findings
A1 · Internet Attacker	Attacker	can self-register a regular account	① ② ③ ④ ⑤ ⑥ ⑦	● 1 · ● 4 · ● 12
A2 · Supply-Chain / Build Attacker	Attacker	compromises a dependency or build pipeline, or contributes malicious code to a public repo · Figure 1b	⑧	● 9
Juice Shop User	Role (victim)	legitimate client	⑥ ⑦	-
Juice Shop Admin	Privileged role	Application role with elevated rights, evidenced by its access check.	-	-

56 findings are not represented in the actor counts (3 Critical, 21 High, 32 Medium). These counts cover assigned findings only; absence from them does not establish inaccessibility.

Trust Boundaries

Each row names a crossing and the condition it relies on. A modelled crossing does not prove that its control works.

ID	Crossing	Required condition	Findings / open question
tb-1	external → Real-time WebSocket Channel	Every Socket.io event payload is validated before it influences server state or challenge logic. Control: not identified	Gap evidenced: *Authentication* ● F-043
tb-2	external → Authentication & Session Surface	Every state-changing or user-specific auth route is registered behind `isAuthorized()` so unauthenticated callers are rejected. Control: `expressJwt` `isAuthorized` middleware	Gap evidenced: *Validation* ● F-013
tb-3	external → Juice Shop Express Server	Every state-changing or user-scoped API route is registered behind `security.isAuthorized()`. Control: `security.isAuthorized()` route middleware	43 related findings; relevance to this crossing remains unverified.
tb-4	external → CI/CD Pipeline	Code from pushes and pull requests runs in CI only with the privileges and secrets scoped to	9 related findings; relevance to this crossing remains unverified.

ID	Crossing	Required condition	Findings / open question
		that trigger. Control: GitHub Actions workflow triggers	
tb-5	external → Real-time WebSocket Channel	Every Socket.IO event payload is treated as untrusted and type-checked before it influences server state. Control: Socket.IO CORS origin setting	2 related findings; relevance to this crossing remains unverified.
tb-6	external → Web3 / Wallet / NFT Surface Crossing: inferred	Every `/rest/web3` route enforces a credential check before serving NFT state or accepting wallet keys. Control: not identified	1 related finding; relevance to this crossing remains unverified.
tb-7	external → Authentication & Session Surface Crossing: inferred	Every API request from the SPA is authenticated and authorized server-side, as the server cannot distinguish SPA calls from forged ones. Control: not identified	11 related findings; relevance to this crossing remains unverified.
tb-8	Angular Frontend → external Crossing: inferred	The OAuth access token returned by the identity provider is accepted only for a redirect URI registered for this client. Control: Google registered redirect URIs	No linked finding establishes a gap; control effectiveness is not established.
tb-9	Sequelize application data access → SQLite Embedded Store	Every query reaches SQLite through Sequelize parameter binding, never concatenated into raw SQL. Control: Sequelize parameter binding	No linked finding establishes a gap; control effectiveness is not established.
tb-10	Juice Shop Express Server → external	The destination of a server-side image fetch is restricted to hosts the server is meant to reach. Control: not identified	Gap evidenced: *Egress destination* ● F-070
tb-11	Juice Shop Express Server → external	Only content intended for the LLM provider, excluding confidential data, reaches the configured endpoint. Control: not identified	No linked finding establishes a gap; control effectiveness is not established.
tb-12	Web3 / Wallet / NFT Surface → external	Event data received from the Alchemy Sepolia RPC is treated as untrusted before it influences	No linked finding establishes a gap; control effectiveness is not established.

ID	Crossing	Required condition	Findings / open question
		challenge state. Control: not identified	
tb-13	Angular Frontend → external	The OAuth redirect URI is used only if it matches a configured authorized redirect. Control: `googleOauth` `authorizedRedirects` allow-list	No linked finding establishes a gap; control effectiveness is not established.

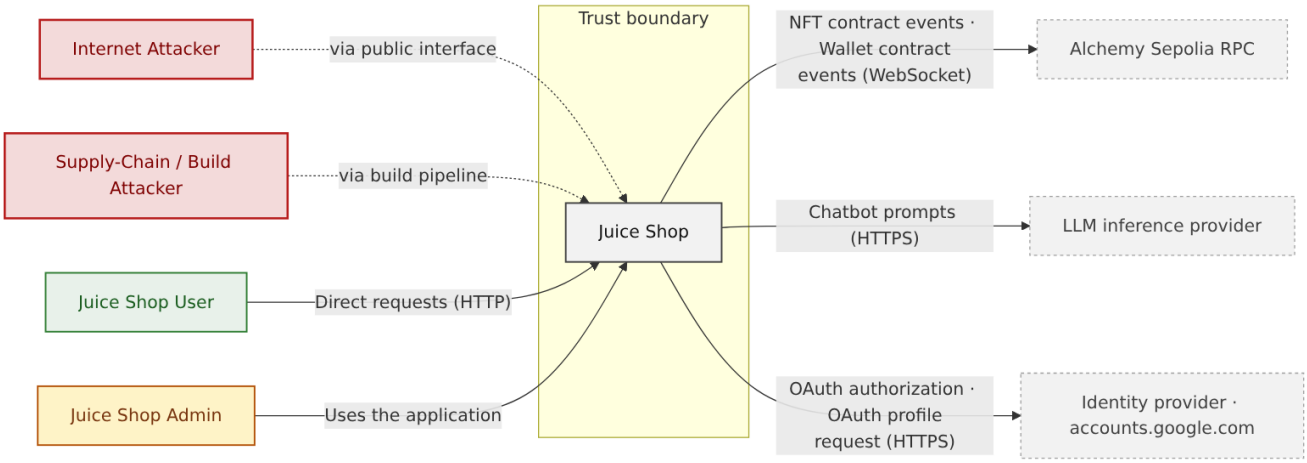
Crossing types, source evidence and provenance are retained in `security-assessment.yaml`.

---

2. Architecture Diagrams

2.1 System Context

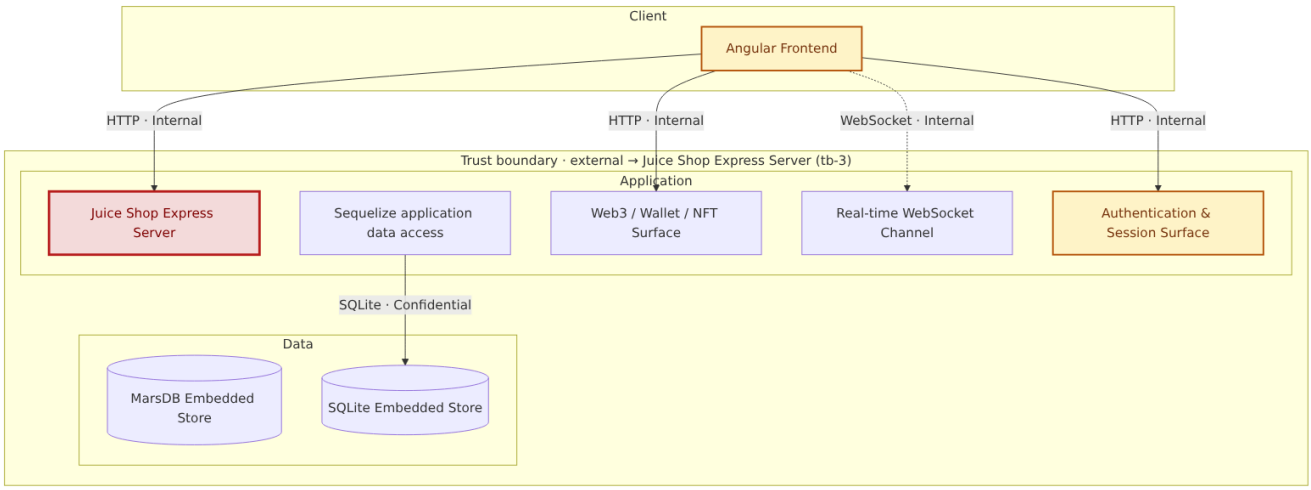
Who uses and attacks Juice Shop, and which external systems it exchanges data with. Solid arrows name the data a flow carries; dashed red arrows are attack routes. Actors carry the names Figure 1 uses (C4 Level 1).



Key takeaway: Juice Shop serves Juice Shop User and Juice Shop Admin and depends on Alchemy Sepolia RPC, LLM inference provider and Identity provider · accounts.google.com; Internet Attacker and Supply-Chain / Build Attacker attack it.

2.2 Container Architecture

How the system decomposes into deployable units. Each box is a separate runtime process or service container; arrows show synchronous request paths between them. Components with ≥3 Critical findings carry a red border, ≥2 High amber (C4 Level 2).



Trust boundaries not drawn above: Sequelize application data access → SQLite Embedded Store (tb-9), external → CI/CD Pipeline (tb-4), Angular Frontend → external (tb-8, tb-13), Juice Shop Express Server → external (tb-10, tb-11), +1 more - every boundary is listed in §1 Trust Boundaries.

Key takeaway: The system decomposes into 1 client, 5 application and 2 data unit(s); Juice Shop Express Server carries the most Critical findings (3) and bounds the worst-case blast radius. The build plane (CI/CD Pipeline) is shown in Figure 1b.

2.3 Components

How each component is reached, what it handles, how many threats hit it, and how effective the controls evidenced on it are. The component table below holds source paths and linked threats per `C-NN`; per-finding evidence is in §8 Findings Register.

Component	Inbound flows	Data handled	Threats	Controls: worst effectiveness per domain
C-01 · Juice Shop Express Server	2 unauthenticated 1 authentication unknown 2 authenticated (bearer, cookie)	Internal	44 (● 3 · ● 16 · ● 25)	● Missing: AuthN, AuthZ ● Weak: Input / query
C-02 · Sequelize application data access	none modeled	Confidential	6 (● 1 · ● 5)	● Weak: Secrets / crypto
C-03 · CI/CD Pipeline	none modeled	–	9 (● 9)	● Weak: Supply chain (7)
C-04 · Web3 / Wallet / NFT Surface	1 unauthenticated	Internal	2 (● 1 · ● 1)	–
	none modeled		10 (● 6 · ● 4)	–

Component	Inbound flows	Data handled	Threats	Controls: worst effectiveness per domain
C-05 · Angular Frontend		Confidential		
C-06 · MarsDB Embedded Store	none modeled	–	none	–
C-07 · Real-time WebSocket Channel	1 authentication unknown	Internal	2 (● 2)	● Missing: AuthN, AuthZ ● Weak: Input / query
C-08 · SQLite Embedded Store	1 unauthenticated	Confidential	none	● Weak: Secrets / crypto
C-09 · Authentication & Session Surface	1 unauthenticated 1 authentication unknown 1 authenticated (bearer)	Internal	14 (● 1 · ● 4 · ● 9)	● Missing: AuthN, AuthZ ● Weak: Input / query
System-wide: evidence not tied to one component	–	–	–	● Unsafe: AuthN (10), Input / query (3) ● Missing: Supply chain ● Weak: Session (4), Secrets / crypto (4), LLM ● Partial: AuthZ (3), Transport, Logging

● ***Unsafe****: relied upon but defeated or trivially bypassable - fix it* · ● ***Missing****: never built - add it* · ● ***Weak****: present, with exploitable gaps* · ● ***Partial****: covers some surfaces, meaningful gaps remain* · ● ***Adequate****: present and sound. A control counts for the component whose paths hold its implementation evidence; a domain without an evidenced control is not listed; (n) counts the controls behind a rating. Threat counts use the Figure 1 attribution; §6 holds the full assessment.*

Key takeaway: 3 components have at least one control rated Unsafe or Missing (C-01, C-07, C-09); 3 components have no component-specific control (C-04, C-05, C-06); 28 controls apply system-wide.

ID	Name	Type	Key Paths	Linked Threats
C-01	Juice Shop Express Server	application	`lib/startup/registerWebsocketEvents.ts`	● F-001 — Missing Object Ownership Checks ● F-007 — SQL injection

ID	Name	Type	Key Paths	Linked Threats
			`routes/address.ts` `routes/appConfiguration.ts` `routes/appVersion.ts` `routes/b2bOrder.ts`	● F-009 — Broken authorization ● F-006 — Systemic IDOR Missing Ownership Checks ● F-015 — Chat identity taken from unverified JWT ● F-016 — JWT verify call missing algorithms allowlist ● F-017 — Password change without current password ● F-022 — Path Traversal ● F-023 — Zip Slip path traversal during archive extraction ● F-025 — Input in executable NoSQL predicate ● F-026 — Input in executable NoSQL predicate ● F-027 — Input compiled as template source ● F-030 — Unauthenticated log directory listing ● F-031 — XML external entity expansion in upload parser ● F-033 — Code injection ● F-034 — Sensitive Routes Registered Without Authentication Middleware ● F-036 — Missing authorization on product update route ● F-037 — Sensitive Routes Registered Without Authentication Middleware ● F-038 — User registration accepts privileged fields ● F-039 — Unauthenticated review creation ● F-040 — Hard-coded cookie signing secret ● F-047 — Wallet top-up accepts unvalidated balance amount ● F-048 — Request data evaluated by notevil in vm sandbox ● F-049 — Cookie-authenticated profile update ● F-050 — Client-supplied coupon data accepted without server-side ● F-051 — Unvalidated client chat history passed to model ● F-062 — Null byte extension allowlist bypass in FTP file ● F-063 — JSONP and arbitrary field selection on current user

ID	Name	Type	Key Paths	Linked Threats
				● F-064 — Unauthenticated memories listing joins user records ● F-065 — Security question disclosure ● F-066 — Unauthenticated metrics endpoint ● F-067 — Unauthenticated log directory listing and download ● F-068 — Unauthenticated encryption key directory exposure ● F-069 — Order history lookup by vowel-masked email ● F-070 — Server-side request forgery ● F-077 — Blocking busy-wait sleep reachable from review ● F-078 — Unbounded zip extraction output size ● F-079 — Unbounded unauthenticated chat consumption ● F-080 — Disk upload without size limit ● F-081 — Unbounded disk upload processed before authentication ● F-084 — Basket ID check bypass ● F-085 — Deluxe upgrade without payment for unrecognized payment ● F-086 — Chatbot coupon discount unbounded server-side
C-02	Sequelize application data access	application	`models/address.ts` `models/basket.ts` `models/basketitem.ts` `models/captcha.ts` `models/card.ts`	● F-032 — Passwords hashed with unsalted fast MD5 ● F-052 — Product description stored without sanitization ● F-053 — User email stored without sanitization ● F-071 — Payment card numbers stored in plaintext ● F-072 — TOTP secrets stored in plaintext ● F-073 — Security answers HMAC with hardcoded key
C-03	CI/CD Pipeline	application	`.dockerignore` `.github/workflows/ci.yml` `.github/workflows/codeql-analysis.yml` `.github/workflows/frontend-bundle-analysis.yml`	● F-054 — Missing Container Image Signing ● F-055 — Remote install script piped to shell in deploy job ● F-056 — Dependency Install Scripts Run at Build Time ● F-057 — Unaudited npm Lifecycle Script ● F-058 — Unpinned Third-Party GitHub Action

ID	Name	Type	Key Paths	Linked Threats
			`.github/workflows/image_actions.yml`	● F-059 — Non-reproducible dependency installs in release builds ● F-060 — Unpinned floating container base images ● F-087 — Container Runs as Root ● F-088 — Missing Workflow Permissions Block
C-04	Web3 / Wallet / NFT Surface	application	`routes/checkKeys.ts` `routes/nftMint.ts` `routes/redirect.ts` `routes/web3Wallet.ts`	● F-082 — Unbounded wallet set growth without validation
C-05	Angular Frontend	client	`frontend/src/app/Models/backup.model.ts` `frontend/src/app/Models/challenge.model.ts` `frontend/src/app/Models/deliveryMethod.model.ts` `frontend/src/app/Models/hint.model.ts` `frontend/src/app/Models/product.model.ts`	● F-002 — XSS via Trusted HTML Binding ● F-003 — Angular Sanitizer Bypass XSS ● F-004 — Session Token Stored in localStorage ● F-018 — Predictable password derived from OAuth email address ● F-028 — DOM XSS ● F-113 — Session token stored in web-accessible client storage ● F-041 — Client-supplied review author identity ● F-042 — OAuth implicit flow without state or nonce ● F-074 — Passwords sent in GET query string unencoded ● F-089 — Client-side role guards trust unverified JWT claims
C-06	MarsDB Embedded Store	data	`data/mongodb.ts`	-
C-07	Real-time WebSocket Channel	application	`lib/challengeUtils.ts` `lib/startup/registerWebsocketEvents.ts`	● F-043 — Unauthenticated WebSocket Channel ● F-083 — Unhandled TypeError from non-string socket
C-08	SQLite Embedded Store	data	`models/index.ts`	-
C-09	Authentication & Session Surface	application	`lib/insecurity.ts` `lib/startup/`	● F-013 — SQL injection in login query ● F-005 — Hardcoded Secrets in Source ● F-021 — Password reset by security

ID	Name	Type	Key Paths	Linked Threats
			`registerWebsocketEvents.ts` `routes/2fa.ts` `routes/authenticatedUsers.ts` `routes/login.ts`	answer only ● F-044 — Weak 2FA code guessing bounds ● F-045 — No brute-force protection on login ● F-046 — Substring redirect allowlist check ● F-061 — Unsanitised login IP header stored for later ● F-075 — Full user record with password hash in JWT ● F-076 — Session token cookie without security attributes ● F-090 — Any authenticated user can list all accounts ● F-091 — Sessions cannot be revoked

> **Legend:** `-->` synchronous request/response (REST, HTTPS, gRPC) · `-.->` asynchronous / event-driven (WebSocket, queue, pub-sub) · **red border** ≥ 3 Critical threats on the component · **amber border** ≥ 2 High threats

---

3. Attack Walkthroughs

This section walks through how the highest-risk findings are exploited - one short walkthrough per Critical, each with attack steps, a focused sequence diagram, and the primary mitigation. Which root cause each Critical traces back to is in Critical Findings by Root Cause; full per-finding context (severity rationale, assets, detection signals) is in the §8 Findings Register.

3.1 SQL injection in Search

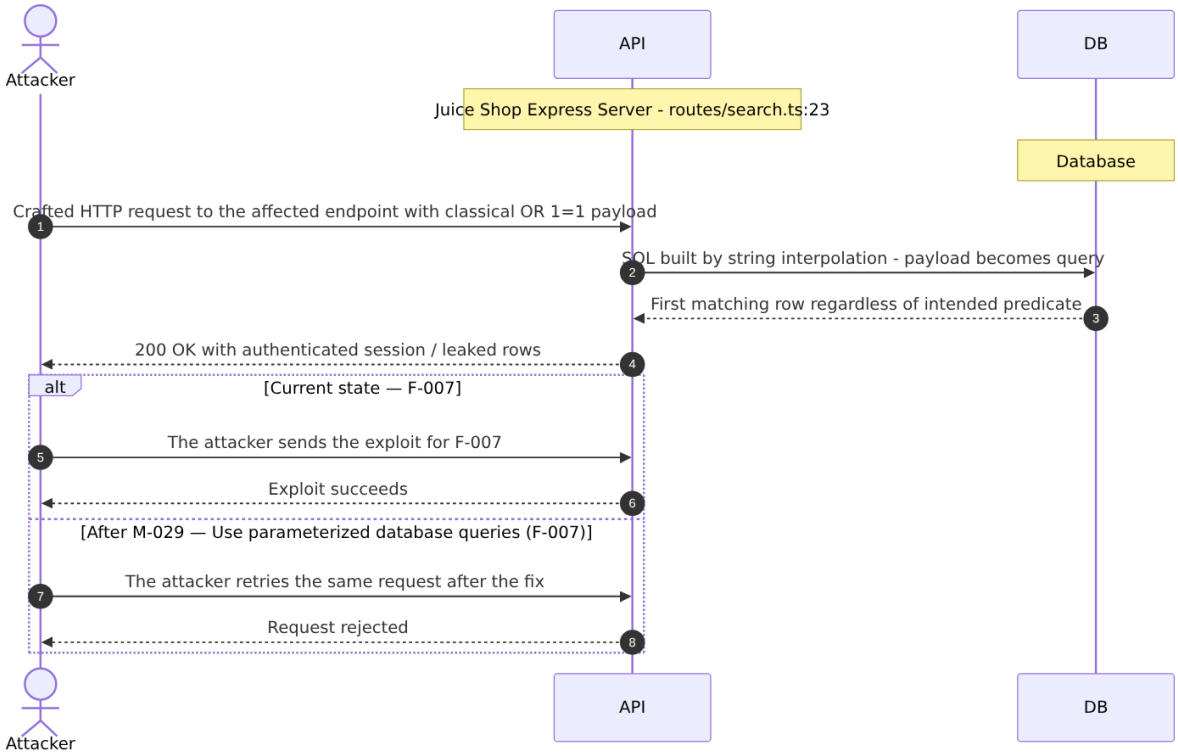
Source: ● F-007 — `routes/search.ts:23`

Severity **Critical** (CWE-89). STRIDE: Tampering. See §8 F-007 for the full register row.

Attack Steps

1. Find the request parameter that reaches the raw query at `routes/search.ts:23`.
2. Interpolating request-controlled text into a SQL statement lets an attacker alter the query - exfiltrating or modifying arbitrary rows, bypassing authentication, or escalating to full database control.

Sequence Diagram



Key takeaway: Until ● M-029 (Use parameterized database queries (● F-007)) lands, ● F-007 — SQL injection is exploitable at `routes/search.ts:23` (Critical-severity, CWE-89).

Defense in Depth

- Primary mitigation: ● M-029 (Use parameterized database queries (● F-007))

3.2 SQL injection in login query

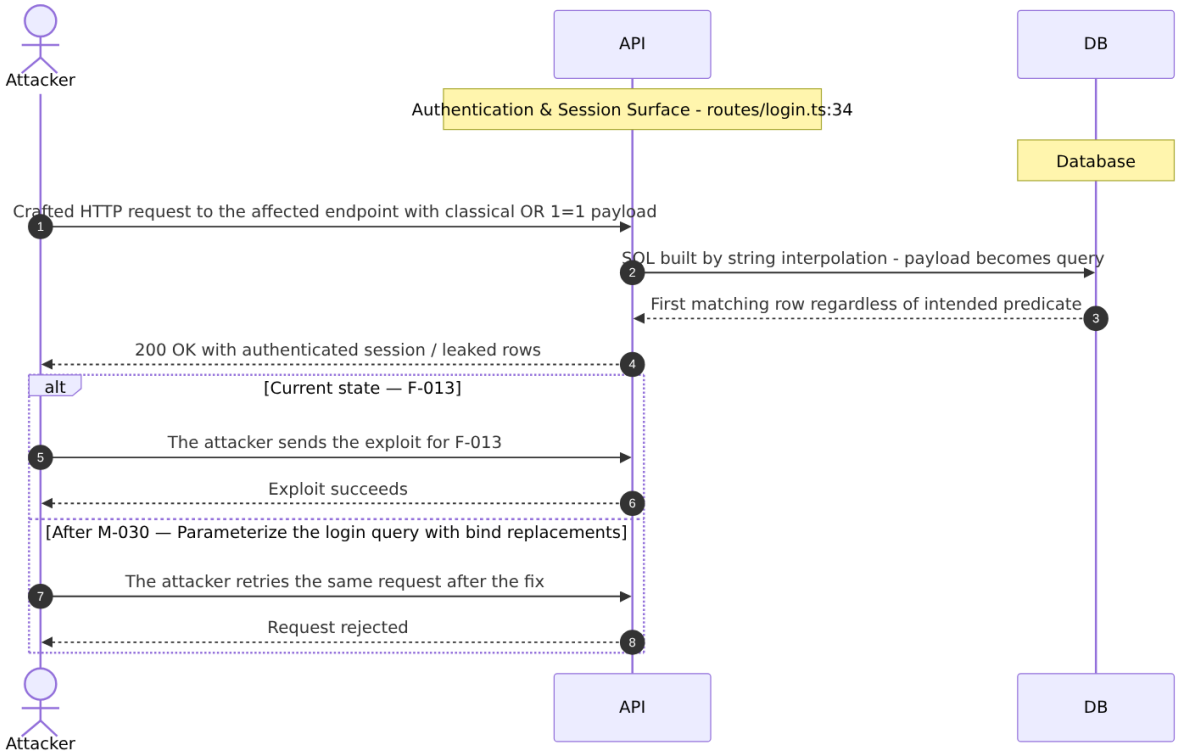
Source: ● F-013 — `routes/login.ts:34`

Severity **Critical** (CWE-89). STRIDE: Tampering. See §8 F-013 for the full register row.

Attack Steps

1. POST `/rest/user/login` with email set to admin@host'-- and any password.
2. The injected quote ends the email literal and comments out the password and `deletedAt` checks.
3. Receive the JWT issued for the admin row and call privileged APIs.

Sequence Diagram



Key takeaway: Until ● M-030 (Parameterize the login query with bind replacements) lands, ● F-013 — SQL injection in login query is exploitable at `routes/login.ts:34` (Critical-severity, CWE-89).

Defense in Depth

- Primary mitigation: ● M-030 (Parameterize the login query with bind replacements)

3.3 Missing Object Ownership Checks in Order

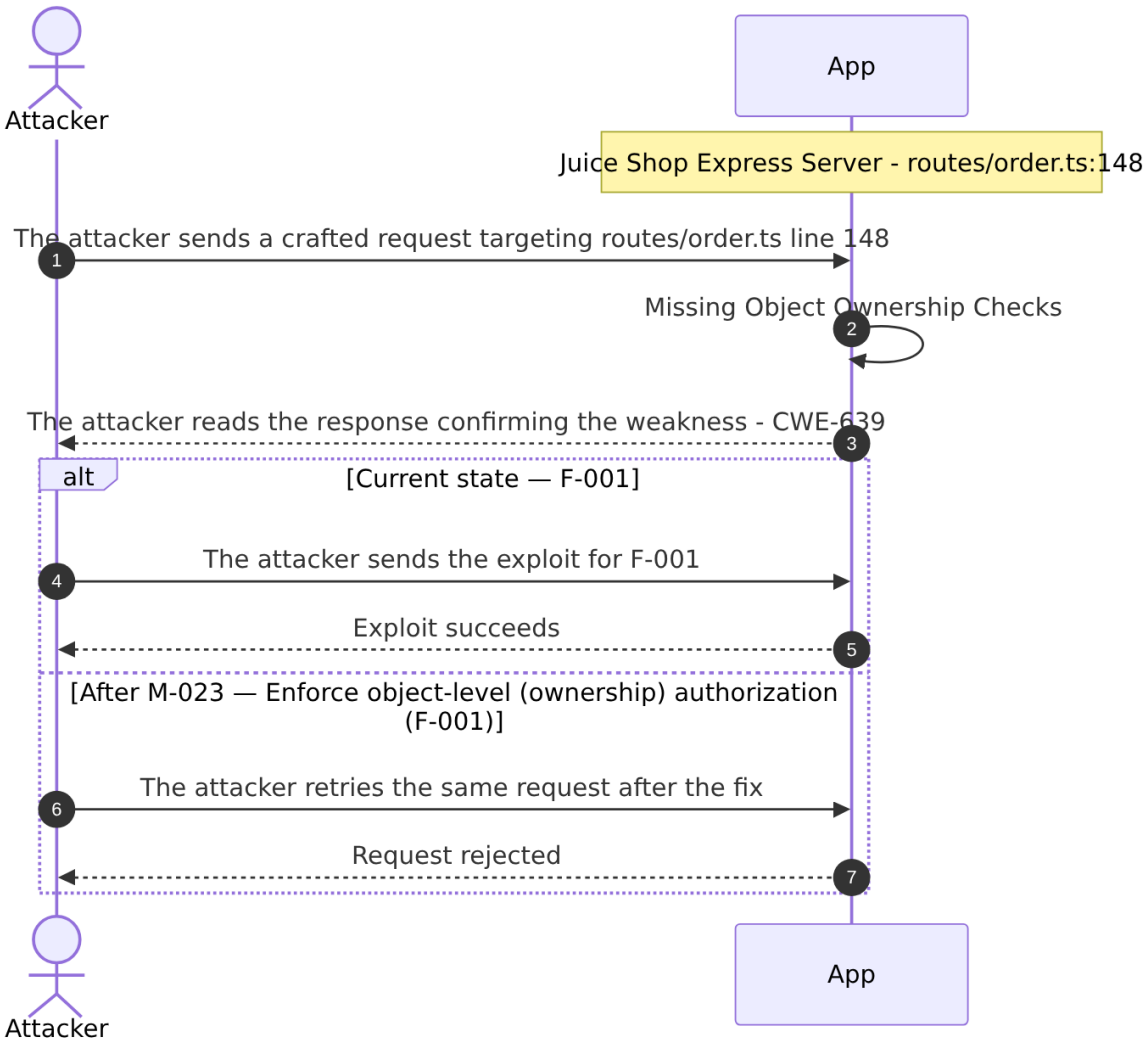
Source: ● F-001 — `routes/order.ts:148`

Severity **Critical** (CWE-639). STRIDE: Tampering. See §8 F-001 for the full register row.

Attack Steps

1. An attacker crafts a request that reaches the code at `routes/order.ts:148`.
2. A signed-in attacker puts another user's ID into the owner field of the request (`req.body.UserId` and the like); the query trusts it instead of the session identity, so the attacker reads or changes every other user's records - horizontal privilege escalation across all accounts.

Sequence Diagram



Key takeaway: Until ● M-023 (Enforce object-level (ownership) authorization (● F-001)) lands, ● F-001 — Missing Object Ownership Checks is exploitable at `routes/order.ts:148` (Critical-severity, CWE-639).

Defense in Depth

- Primary mitigation: ● M-023 (Enforce object-level (ownership) authorization (● F-001))
- Defence in depth: ◑ M-001 (Manual review: verify Missing Object Ownership Checks)

3.4 Broken authorization in Data Export

Source: ● F-009 — `routes/dataExport.ts:26`

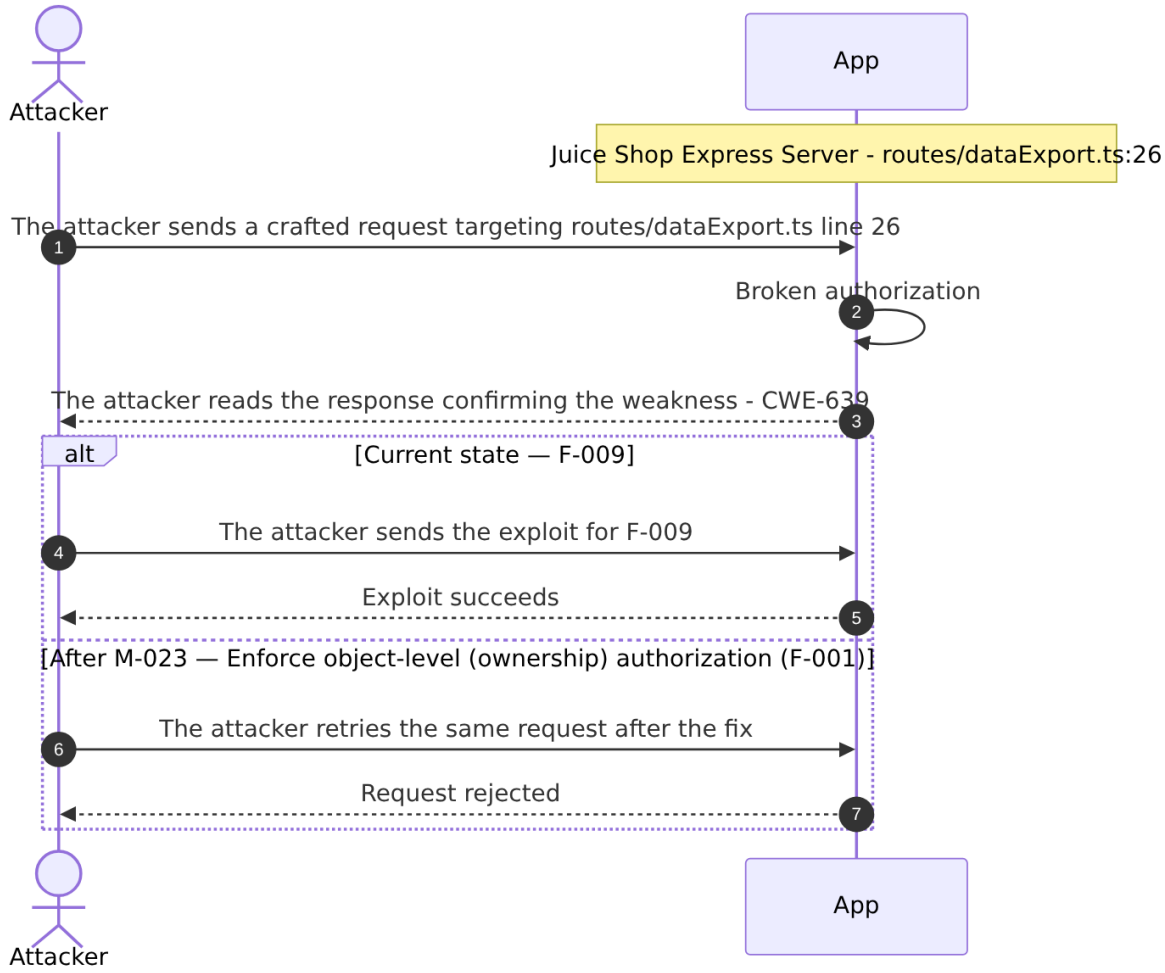
Severity **Critical** (CWE-639). STRIDE: Tampering. See §8 F-009 for the full register row.

Attack Steps

1. An attacker crafts a request that reaches the code at `routes/dataExport.ts:26`.
2. A signed-in attacker puts another user's ID into the owner field of the request (`req.body.UserId` and the like); the query trusts it instead of the session identity, so the

attacker reads or changes every other user's records - horizontal privilege escalation across all accounts.

Sequence Diagram



Key takeaway: Until ● M-023 (Enforce object-level (ownership) authorization (● F-001)) lands, ● F-009 — Broken authorization is exploitable at `routes/dataExport.ts:26` (Critical-severity, CWE-639).

Defense in Depth

- Primary mitigation: ● M-023 (Enforce object-level (ownership) authorization (● F-001))

3.5 User registration accepts privileged fields in Juice Shop Express Server

Source: ● F-038 — `server.ts:503`

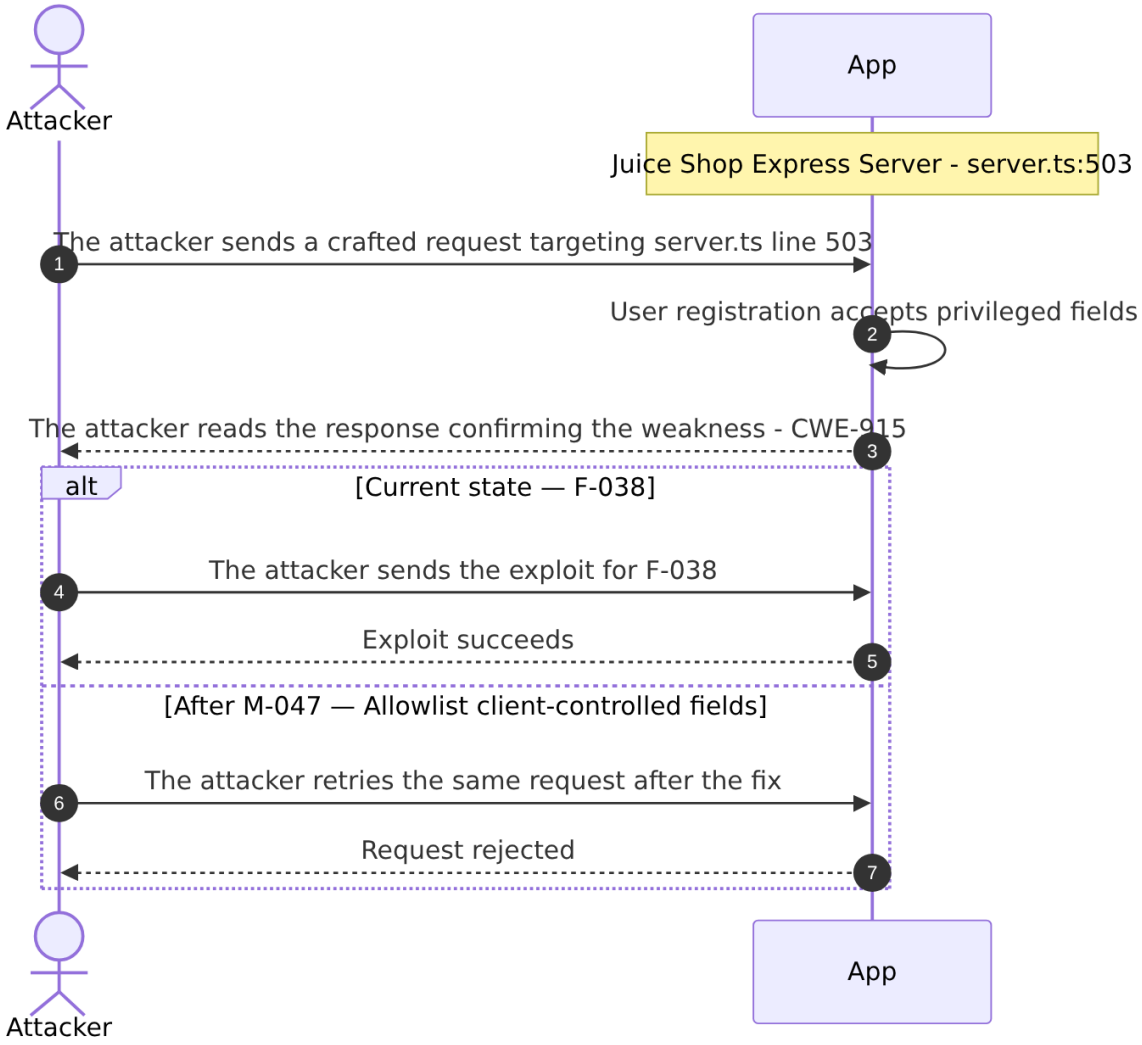
Severity **High** (CWE-915). STRIDE: Elevation of Privilege. See §8 F-038 for the full register row.

Attack Steps

1. An attacker crafts a request that reaches the code at `server.ts:503`.

2. POST `/api/Users` is passed to the generated finale User resource after only trimming; the body is not filtered, so the attacker supplies a role such as admin at registration.

Sequence Diagram



Key takeaway: Until ◕ M-047 (Allowlist client-controlled fields) lands, ● F-038 — User registration accepts privileged fields is exploitable at `server.ts:503` (High-severity, CWE-915).

Defense in Depth

- Primary mitigation: ◕ M-047 (Allowlist client-controlled fields)

3.6 Hardcoded Secrets in Source

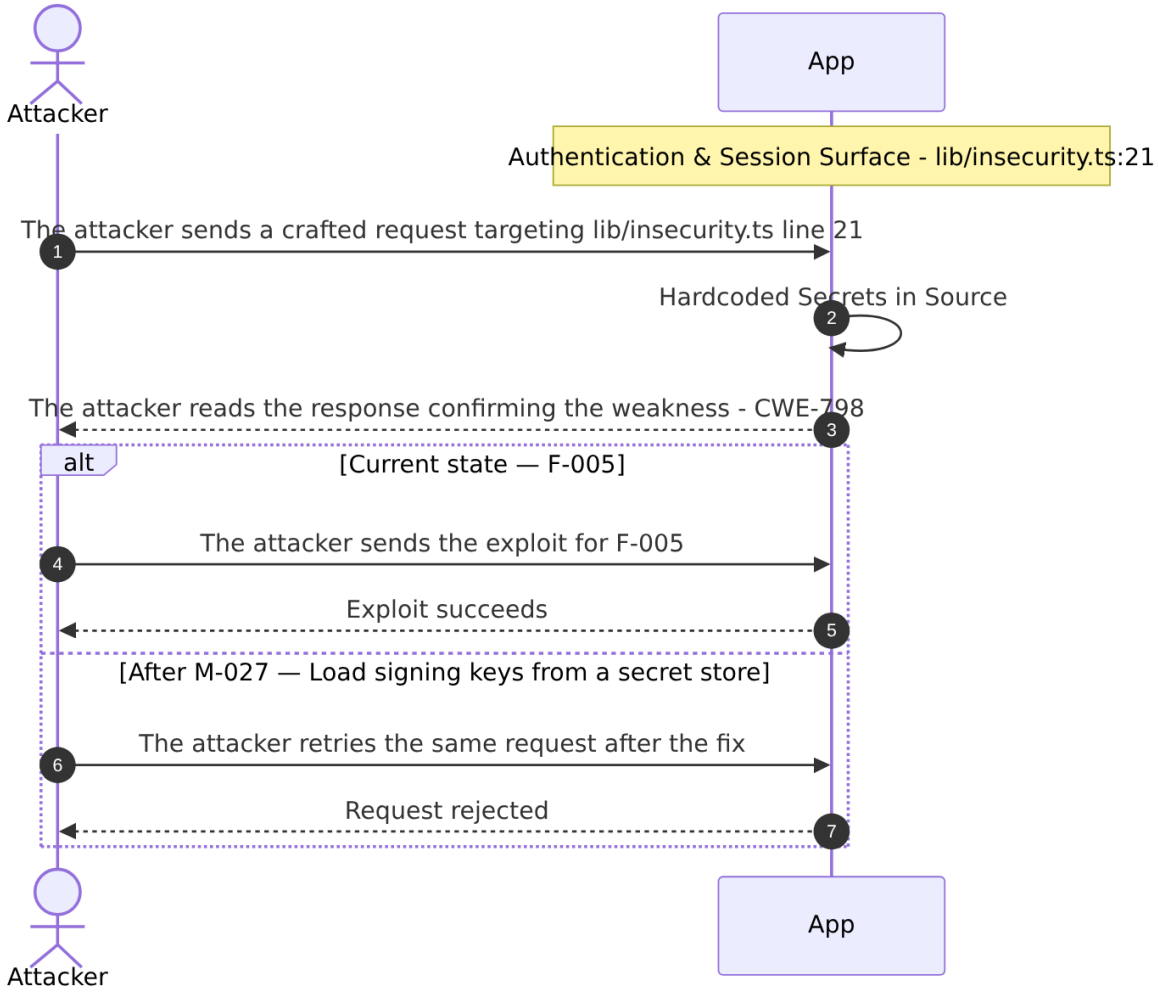
Source: ● F-005 — `lib/insecurity.ts:21`

Severity **High** (CWE-798). STRIDE: Information Disclosure. See §8 F-005 for the full register row.

Attack Steps

1. An attacker crafts a request that reaches the code at `lib/insecurity.ts:21`.
2. The `RS256` signing key is a literal in source and the security-answer HMAC key is a literal at line 42.
3. Anyone with the source can sign tokens for any user or role, or compute answer hashes offline.

Sequence Diagram



Key takeaway: Until ◕ M-027 (Load signing keys from a secret store) lands, ● F-005 — Hardcoded Secrets in Source is exploitable at `lib/insecurity.ts:21` (High-severity, CWE-798).

Defense in Depth

- Primary mitigation: ◕ M-027 (Load signing keys from a secret store)

3.7 Request data evaluated by notevil in vm sandbox

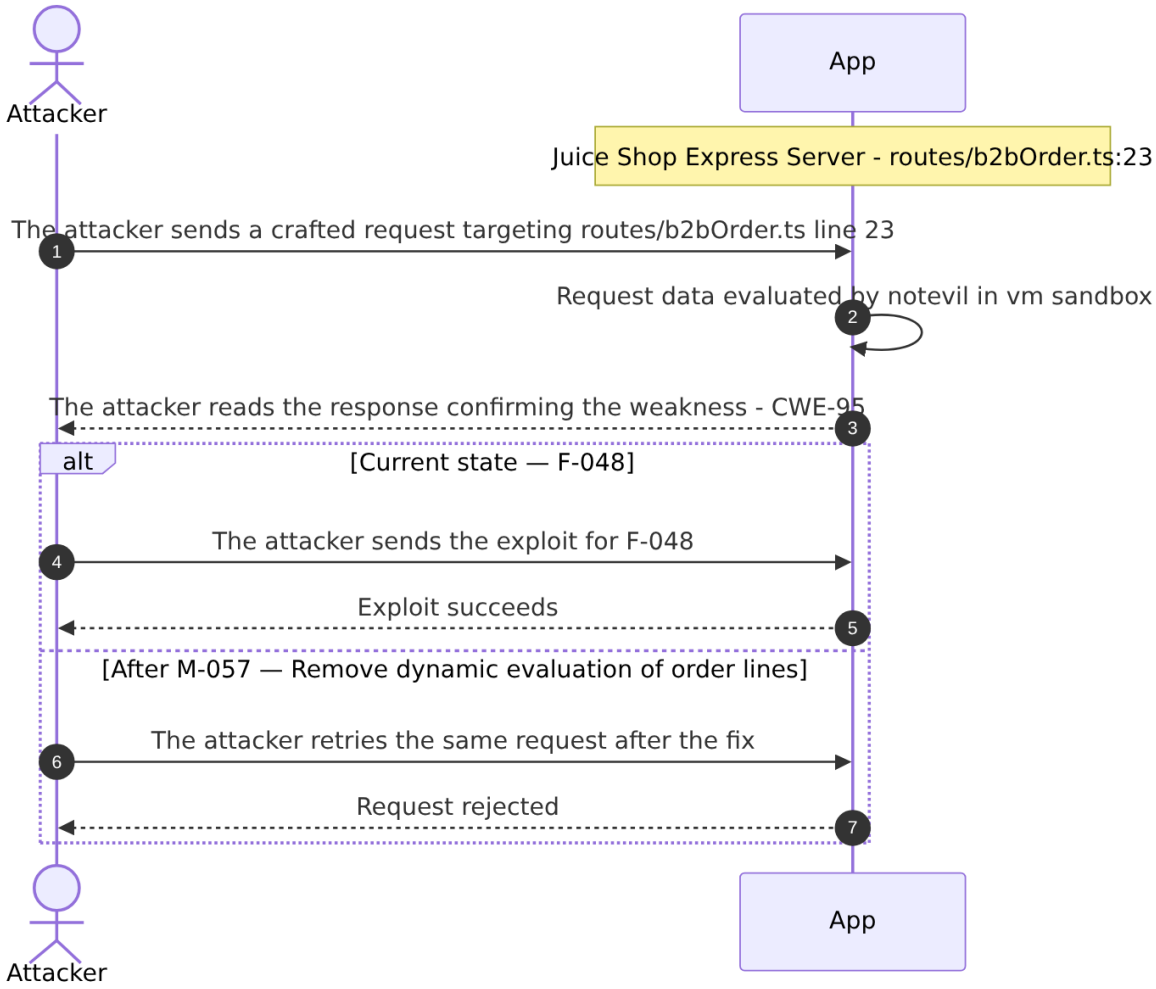
Source: ● F-048 — `routes/b2bOrder.ts:23`

Severity **Medium** (CWE-95). STRIDE: Tampering. See §8 F-048 for the full register row.

Attack Steps

1. Authenticate and POST `/b2b/v2/orders` with a crafted `orderLinesData` expression.
2. Escape the notevil/vm sandbox to execute code in the server process.

Sequence Diagram



Key takeaway: Until ◑ M-057 (Remove dynamic evaluation of order lines) lands, ● F-048 — Request data evaluated by notevil in vm sandbox is exploitable at `routes/b2bOrder.ts:23` (Medium-severity, CWE-95).

Defense in Depth

- Primary mitigation: ◑ M-057 (Remove dynamic evaluation of order lines)

---

4. Assets

Information assets and the classification level that drives the Confidentiality / Integrity / Availability targets used in §8 Findings Register risk scoring.

Asset	Classification	Description
Alchemy API key	Restricted	API key read from environment and embedded in the Alchemy RPC URL.
LLM provider API key	Restricted	API key read from environment and used for chatbot model calls.
SQLite application database	Confidential	Sequelize-managed SQLite file data/juiceshop.sqlite holding application model data.
MarsDB reviews and orders collections	Internal	In-process collections for product reviews (posts) and orders.

---

5. Attack Surface

Network-reachable entry points classified by authentication requirement. Each row links to the threat(s) referenced in its **Notes** column. The **Risk** column reflects the highest-severity linked finding. Entry points with no linked finding are still listed when they sit on a sensitive surface (authentication, registration, management) or look like a missing-auth/authz suspect - marked ⚑ **Review** in Notes.

5.1 Unauthenticated Entry Points (69)

Method	Route	Risk	Notes
POST	`/rest/user/data-export`	● Critical	● F-009 — Broken authorization ● F-028 — DOM XSS
POST	`/rest/user/login`	● Critical	● F-013 — SQL injection in login query ● F-045 — No brute-force protection on login ● F-075 — Full user record with password hash in JWT Public by design
GET	`/rest/products/search`	● Critical	● F-007 — SQL injection
POST	`/api/Users`	● High	● F-038 — User registration accepts privileged fields
POST	`/file-upload`	● High	● F-023 — Zip Slip path traversal during archive extraction ● F-031 — XML external entity expansion in upload parser ● F-034 — Sensitive Routes Registered Without Authentication Middleware
POST	`/rest/2fa/verify`	● High	● F-016 — JWT verify call missing algorithms allowlist ● F-044 — Weak 2FA code guessing bounds Public by design

Method	Route	Risk	Notes
POST	`/rest/chat`	● High	● F-015 — Chat identity taken from unverified JWT ● F-051 — Unvalidated client chat history passed to model ● F-079 — Unbounded unauthenticated chat consumption
POST	`/rest/user/reset-password`	● High	● F-021 — Password reset by security answer only ● F-065 — Security question disclosure Public by design
GET	`/rest/track-order/:id`	● High	● F-026 — Input in executable NoSQL predicate
GET	`/rest/user/change-password`	● High	● F-017 — Password change without current password ● F-074 — Passwords sent in GET query string unencoded
ALL	`/support/logs/:file`	● High	● F-030 — Unauthenticated log directory listing ● F-067 — Unauthenticated log directory listing and download
GET	`/metrics`	● Medium	● F-066 — Unauthenticated metrics endpoint Management surface
POST	`/rest/deluxe-membership`	● Medium	● F-085 — Deluxe upgrade without payment for unrecognized payment
POST	`/rest/memories`	● Medium	● F-064 — Unauthenticated memories listing joins user records
PUT	`/rest/wallet/balance`	● Medium	● F-047 — Wallet top-up accepts unvalidated balance amount
ALL	`/encryptionkeys/:file`	● Medium	● F-068 — Unauthenticated encryption key directory exposure
GET	`/rest/deluxe-membership`	● Medium	● F-085 — Deluxe upgrade without payment for unrecognized payment
GET	`/rest/memories`	● Medium	● F-064 — Unauthenticated memories listing joins user records
GET	`/rest/saveLoginIp`	● Medium	● F-061 — Unsanitised login IP header stored for later
GET	`/rest/user/security-question`	● Medium	● F-065 — Security question disclosure
GET	`/rest/user/whoami`	● Medium	● F-063 — JSONP and arbitrary field selection on current user

Method	Route	Risk	Notes
GET	`/rest/wallet/balance`	● Medium	● F-047 — Wallet top-up accepts unvalidated balance amount
POST	`/api/Addresss`	-	⚑ *Review: no auth guard detected*
PUT	`/api/Addresss/:id`	-	⚑ *Review: no auth guard detected*
DELETE	`/api/Addresss/:id`	-	⚑ *Review: no auth guard detected*
POST	`/api/Cards`	-	⚑ *Review: no auth guard detected*
DELETE	`/api/Cards/:id`	-	⚑ *Review: no auth guard detected*
POST	`/api/Feedbacks`	-	⚑ *Review: no auth guard detected*
GET	`/rest/admin/application-configuration`	-	Management surface ⚑ *Review: no auth guard detected*
GET	`/rest/admin/application-version`	-	Management surface ⚑ *Review: no auth guard detected*
PUT	`/rest/continue-code-findIt/apply/:continueCode`	-	⚑ *Review: no auth guard detected*
PUT	`/rest/continue-code-fixIt/apply/:continueCode`	-	⚑ *Review: no auth guard detected*
PUT	`/rest/continue-code/apply/:continueCode`	-	⚑ *Review: no auth guard detected*
PUT	`/rest/products/:id/reviews`	-	⚑ *Review: no auth guard detected*
POST	`/rest/web3/submitKey`	-	⚑ *Review: no auth guard detected*
POST	`/rest/web3/walletExploitAddress`	-	⚑ *Review: no auth guard detected*
POST	`/rest/web3/walletNFTVerify`	-	⚑ *Review: no auth guard detected*
POST	`/snippets/fixes`	-	⚑ *Review: no auth guard detected*
POST	`/snippets/verdict`	-	⚑ *Review: no auth guard detected*

30 further entry point(s) in this category carry no linked finding and no elevated review signal, and are not listed individually (69 total). The complete route inventory is available in `route-inventory.json` and, when exported, `pentest-tasks.yaml`.

5.2 Authenticated Entry Points (40)

Method	Route	Risk	Notes
DELETE	`/api/Products/:id`	● High	● F-036 — Missing authorization on product update route Authorization not confirmed (review)

Method	Route	Risk	Notes
PUT	`/api/Products/:id`	● High	● F-036 — Missing authorization on product update route Authorization not confirmed (review)
GET	`/rest/basket/:id`	● High	● F-006 — Systemic IDOR Missing Ownership Checks
POST	`/rest/basket/:id/checkout`	● High	● F-006 — Systemic IDOR Missing Ownership Checks Authorization not confirmed (review)
PUT	`/rest/basket/:id/coupon/:coupon`	● High	● F-006 — Systemic IDOR Missing Ownership Checks Authorization not confirmed (review)
POST	`/api/Products`	● High	● F-036 — Missing authorization on product update route Authorization not confirmed (review)
GET	`/api/Users`	● High	● F-038 — User registration accepts privileged fields
GET	`/profile`	● High	● F-033 — Code injection ● F-049 — Cookie-authenticated profile update ● F-070 — Server-side request forgery
POST	`/profile`	● High	● F-033 — Code injection ● F-049 — Cookie-authenticated profile update ● F-070 — Server-side request forgery Authorization not confirmed (review)
PUT	`/api/BasketItems/:id`	● Medium	● F-084 — Basket ID check bypass Authorization not confirmed (review)
PUT	`/rest/order-history/:id/delivery-status`	● Medium	● F-069 — Order history lookup by vowel-masked email Authorization not confirmed (review)
POST	`/api/BasketItems`	● Medium	● F-084 — Basket ID check bypass Authorization not confirmed (review)
POST	`/profile/image/file`	● Medium	● F-070 — Server-side request forgery Authorization not confirmed (review)
POST	`/profile/image/url`	● Medium	● F-070 — Server-side request forgery Authorization not confirmed (review)
GET	`/rest/order-history`	● Medium	● F-069 — Order history lookup by vowel-masked email
GET	`/rest/order-history/orders`	● Medium	● F-069 — Order history lookup by vowel-masked email

Method	Route	Risk	Notes
GET	`/rest/user/authentication-details`	● Medium	● F-090 — Any authenticated user can list all accounts
PUT	`/api/Cards/:id`	-	Authorization not confirmed (review) ⚑ *Review: no authz guard detected*
PUT	`/api/Feedbacks/:id`	-	Authorization not confirmed (review) ⚑ *Review: no authz guard detected*
DELETE	`/api/Quantitys/:id`	-	Authorization not confirmed (review) ⚑ *Review: no authz guard detected*
PUT	`/api/Recycles/:id`	-	Authorization not confirmed (review) ⚑ *Review: no authz guard detected*
DELETE	`/api/Recycles/:id`	-	Authorization not confirmed (review) ⚑ *Review: no authz guard detected*
POST	`/rest/2fa/disable`	-	Public by design ⚑ *Review: auth/token endpoint*
POST	`/rest/2fa/setup`	-	Public by design ⚑ *Review: auth/token endpoint*
GET	`/rest/2fa/status`	-	Public by design ⚑ *Review: auth/token endpoint*

15 further entry point(s) in this category carry no linked finding and no elevated review signal, and are not listed individually (40 total). The complete route inventory is available in `.route-inventory.json` and, when exported, `pentest-tasks.yaml`.

---

6. Security Architecture

This chapter is organized by security-control category. Findings are listed only where the affected control is described.

Cataloged controls: 43 total - 3 adequate, 11 partial, 19 weak, 3 unsafe, 7 missing.

How to read the verdicts. Every control category (and every sub-control below it) carries exactly one status. The two red verdicts do **not** mean the same thing - this is the distinction that decides what you have to do about a finding:

Status	Meaning	What it asks of you
● Adequate	Control is present and sound	Nothing - keep it
● Partial	Present, but with meaningful gaps	Close the gap
● Weak	Present, but has exploitable gaps	Strengthen it
● Unsafe	**Present and relied upon, but defeated / trivially bypassable**	**Fix the existing control**

Status	Meaning	What it asks of you
● Missing	**Control was never built**	**Add the control**
-	Not applicable to this codebase	-

So "● Unsafe" on a control category does *not* mean the control is absent - it means the control exists but does not hold (e.g. an `MD5` password hash, a raw-SQL query path, a hardcoded signing key). "● Missing" is reserved for controls that were never built (e.g. no `Content-Security-Policy` header).

6.1 Security Control Overview

Control category	Verdict	Main reason
6.2 Identity and Authentication Controls	● Unsafe	11 findings; catalogued controls are present but defeated (e.g. Chat endpoint caller identity, Wallet ownership verification).
6.3 Session and Token Controls	● Weak	4 findings; catalogued controls are weak (e.g. Cookie signing secret, Client-side session token storage).
6.4 Authorization Controls	● Missing	12 findings; required controls not in place (e.g. Route-level authorization middleware, LLM agent tool authorization and coupon policy).
6.5 Query Construction and Data Access Controls	● Unsafe	4 findings; catalogued controls are present but defeated (e.g. Chatbot tool query construction, Sequelize connection and transaction configuration).
6.6 Input Boundary Validation Controls	● Missing	6 findings; required controls not in place (e.g. Rate limiting, Request input validation for wallet address).
6.7 Output Encoding and Rendering Controls	● Weak	6 findings; no compensating controls catalogued.
6.8 Browser and Cross-Origin Controls	● Weak	2 findings; catalogued controls are weak (e.g. Socket.io CORS origin restriction).
6.9 Cryptography Secrets and Data Protection	● Weak	5 findings; catalogued controls are weak (e.g. Database access credentials and storage protection, RPC API key handling and transport).
6.10 File Parser and Outbound Request Controls	● Missing	13 findings; required controls not in place (e.g. Profile image URL fetch (SSRF protection), Upload size and type limits).
6.11 Operations Runtime and Supply Chain Controls	● Missing	10 findings; required controls not in place (e.g. Sensitive endpoint exposure (metrics,

Control category	Verdict	Main reason
		log directory, API docs), GitHub Actions pinning to commit SHAs).
6.12 Real-time and Not Applicable Controls	● Weak	0 findings; catalogued controls are weak (e.g. Socket.io client connection).

6.2 Identity and Authentication Controls

Systemic weaknesses: W-008, W-009, W-011, W-012 **Verdict:** ● Unsafe - login, reset, OAuth, and WebSocket entry points exist but are defeated by missing throttling, answer-only reset, and absent caller verification.

Controls covered:

- 6.2.1 OAuth implicit flow login and token handling
- 6.2.2 Hardcoded test credentials and OAuth client ID in client code
- 6.2.3 Two-factor authentication
- 6.2.4 Password reset via security question

Implemented controls: Chat endpoint caller identity, Wallet ownership verification, OAuth implicit flow login and token handling, Hardcoded test credentials and OAuth client ID in client code, Two-factor authentication (TOTP).

Assessment: Password login has no attempt limit (● F-045), reset needs only a security answer (● F-021), and the OAuth adapter derives a predictable password (● F-018). TOTP is present but has loose guess bounds (● F-044). Each successful flow ends with the server issuing a session token. Signing, validation, storage, and lifecycle of that token are described in §6.3 Session and Token Controls.

Authentication mechanisms (at a glance). Every authentication mechanism detected on the application, its effective status, where it is assessed, and its linked findings. Controls are catalogued by domain, so JWT/session handling is assessed under §6.3 Session and Token Controls and password hashing under §6.9 Cryptography Secrets and Data Protection.

Mechanism	Status	Assessed in	Findings
User registration	● Unsafe	§6.2	W-002 — Authorization is implemented route by route W-023 — Privileged attributes lack an enforced write allowlist ● F-083 — Unhandled TypeError from non-string socket — `registerWebsocketEvents.ts:46`
Password login	● Missing	§6.2	W-012 — No brute-force protection on login (`login.ts:32`)
Password reset / change	● Weak	§6.2	● F-017 — Password change without current password — `routes/changePassword.ts:39`

Mechanism	Status	Assessed in	Findings
			● F-021 — Password reset by security answer only ● F-065 — Security question disclosure — `routes/securityQuestion.ts:15`
Password storage (hashing)	● Weak	§6.9	W-041 — Security-sensitive data uses weak cryptographic primitives W-030 — Full user record with password hash in JWT (`login.ts:24`)
JWT / bearer-token session	● Partial	§6.3	W-008 — Authentication relies on hand-rolled token verification W-030 — Full user record with password hash in JWT (`login.ts:24`) W-020 — Client-side role guards trust unverified JWT claims (`app.guard.ts:54`)
Session-token storage	● Weak	§6.3	W-005 — Browser scripts retain reusable session credentials ● F-113 — Session token stored in web-accessible client storage — `basket.service.ts:64`
Multi-factor authentication (TOTP / 2FA)	● Partial	§6.2	W-011 — Weak 2FA code guessing bounds (`2fa.ts:31`) W-036 — TOTP secrets stored in plaintext (`user.ts:112`)
OAuth / OIDC federated login	● Unsafe	§6.2	● F-018 — Predictable password derived from OAuth email address — `oauth.component.ts:30` W-021 — OAuth sign-in requests are not bound to the user's browser session

Findings in this category without a control:

- ● F-039 — Unauthenticated review creation
- ● F-041 — Client-supplied review author identity
- ● F-043 — Unauthenticated WebSocket Channel
- ● F-066 — Unauthenticated metrics endpoint

Chat endpoint caller identity.

Status: ● Weak - the route derives the caller name from a token that is decoded but never signature-checked.

`POST /rest/chat` decodes the bearer token to read the user name, then inserts that name into the chatbot system prompt and uses it for order lookups.

Security assessment

The caller identity on the chat route is client-asserted:

- The token is decoded without signature verification and the route has no `isAuthorized` guard (● F-015).
- Chat history arrives in the request body and goes to the model verbatim (● F-051).

Relevant findings

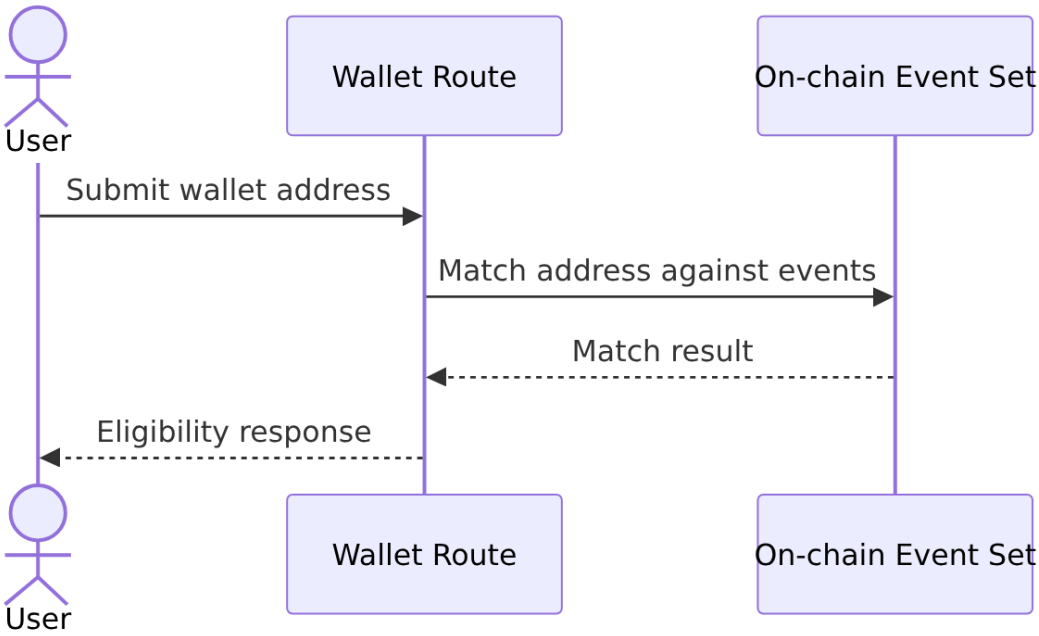
- None linked to this control; see the other findings of this category.

Wallet ownership verification.

Status: ● Weak - addresses come from the request body and nothing binds the caller to them.

The wallet and NFT handlers read `req.body.walletAddress` and match it against on-chain event sets to decide eligibility.

The diagram shows the intended wallet check path:



Security assessment

Possession of an address is never proven, so any caller can claim any address:

- No signature or session ties the request to the address owner.
- The route accepts raw balance and address values without validation (● F-047).
- The address set grows without bound (● F-082).

Relevant findings

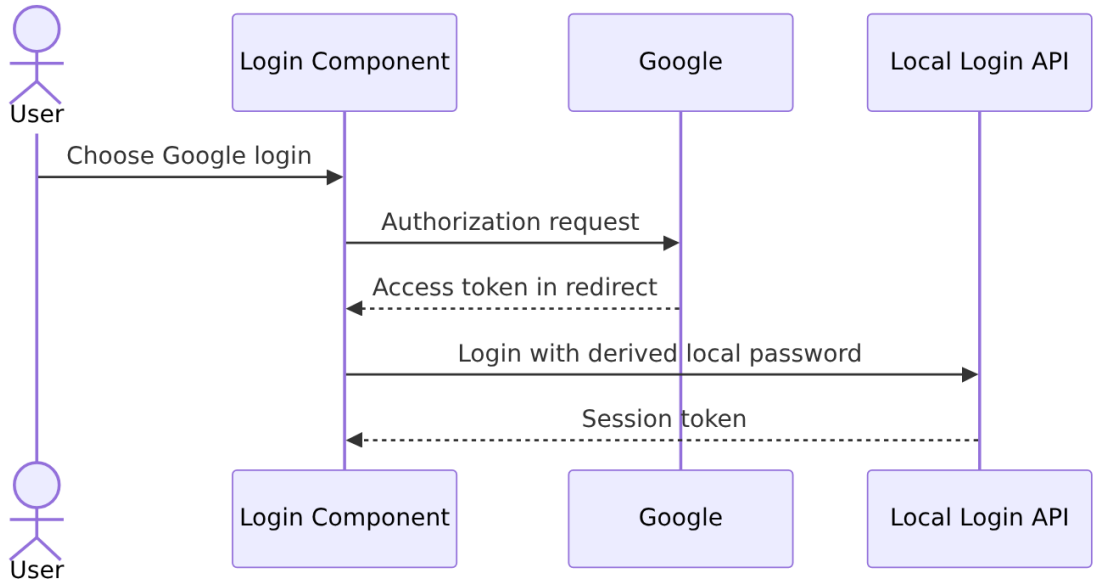
- None linked to this control; see the other findings of this category.

6.2.1 OAuth implicit flow login and token handling

Status: ● Weak - the implicit flow carries no state, nonce, or PKCE, and the result becomes a predictable local password.

`login.component.ts` starts Google login with `response_type=token`, and `UserService.oauthLogin()` turns the returned profile into a local session. A server-supplied allow-list of redirect URIs is checked before OAuth is enabled.

The diagram shows how the frontend OAuth adapter ends in the local login flow:



Security assessment

The flow only partly resembles a server-side OAuth control:

- No `state` or `nonce` is generated or verified (● F-042).
- The local password is derived deterministically from the email address (● F-018).
- The access token travels in a query string, where logs and `Referer` headers can capture it.
- The redirect allow-list check is a partial mitigation.

Relevant findings

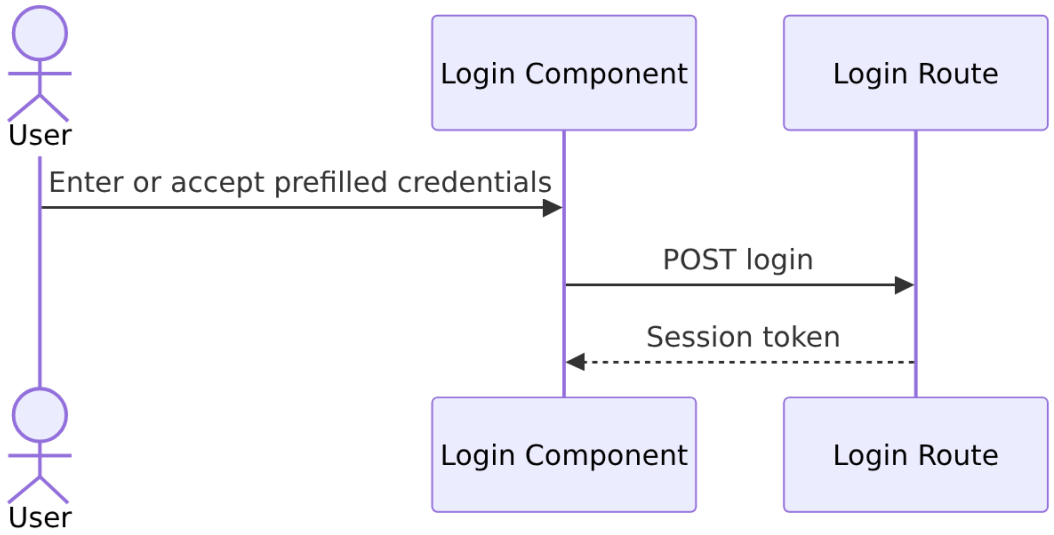
- None linked to this control; see the other findings of this category.

6.2.2 Hardcoded test credentials and OAuth client ID in client code

Status: ● Unsafe - test credentials and an OAuth client ID are hardcoded in the shipped login component.

`login.component.ts` pre-populates test credentials and embeds a Google OAuth client ID in the client bundle.

The diagram shows how the login form submits credentials:



Security assessment

Exposure here is limited and may be deliberate in a training application:

- Anyone reading the bundle sees working test credentials.
- The OAuth client ID is public by design and carries little risk by itself.
- No finding is routed to this control.

Relevant findings

- None linked to this control; see the other findings of this category.

WebSocket connection authentication.

Status: ● Missing - the connection handler accepts every client without a token, session, or middleware check.

This control is not evidenced in the repository.

Security assessment

The `socket.io` server has no `io.use` authentication step:

- Any client receives stored notifications and may emit handler events (● F-043).
- A non-string payload raises an unhandled `TypeError` (● F-083).

Relevant findings

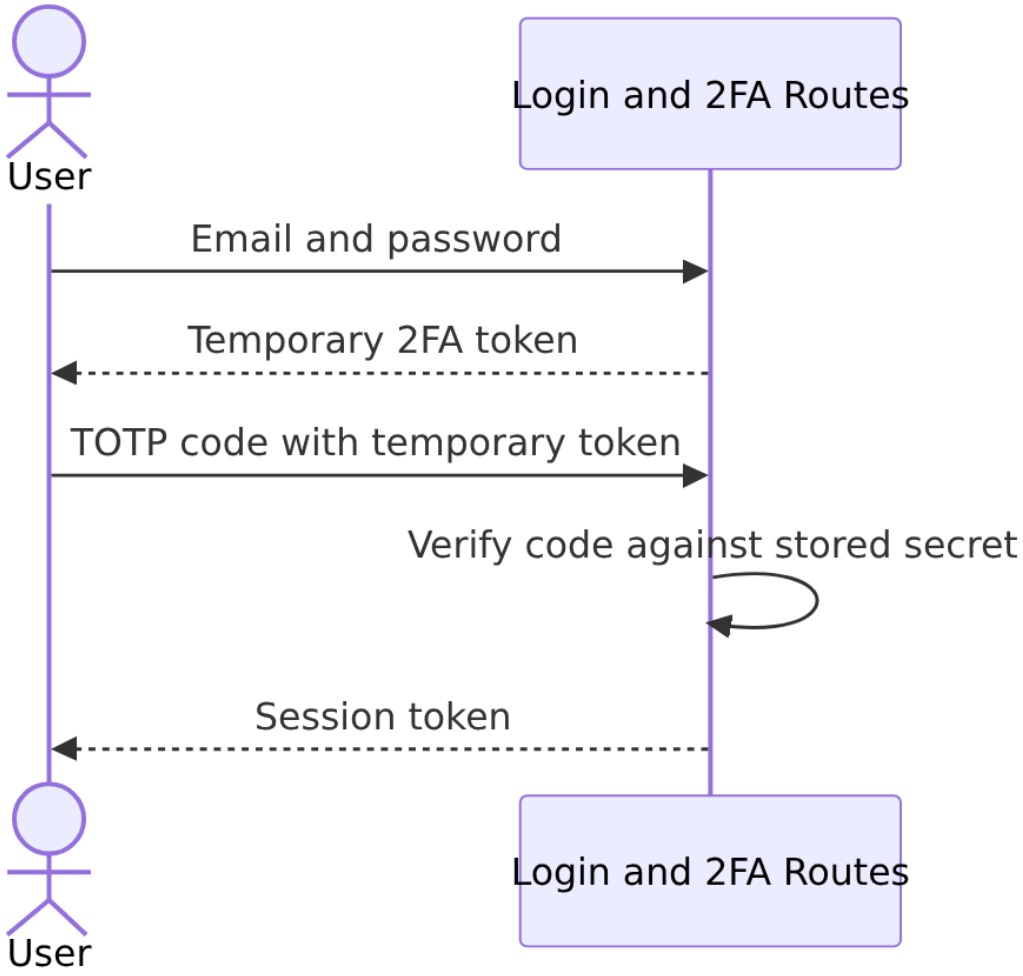
- None linked to this control; see the other findings of this category.

6.2.3 Two-factor authentication

Status: ● Partial - TOTP is enforced after the password step, but guess bounds are loose and secrets are stored in plaintext.

`routes/2fa.ts` issues a typed temporary token after the password step and requires a TOTP code before the session token is released. Setup and disable require the password.

The diagram shows the positive password login path, including the branch into TOTP verification:



Security assessment

The possession factor holds in structure but is weakened in three places:

- Verification tolerates a wide time window with no per-user lockout (● F-044).
- The TOTP secret is stored as plaintext (● F-072).
- The only limit is an IP-based rate limit.

Relevant findings

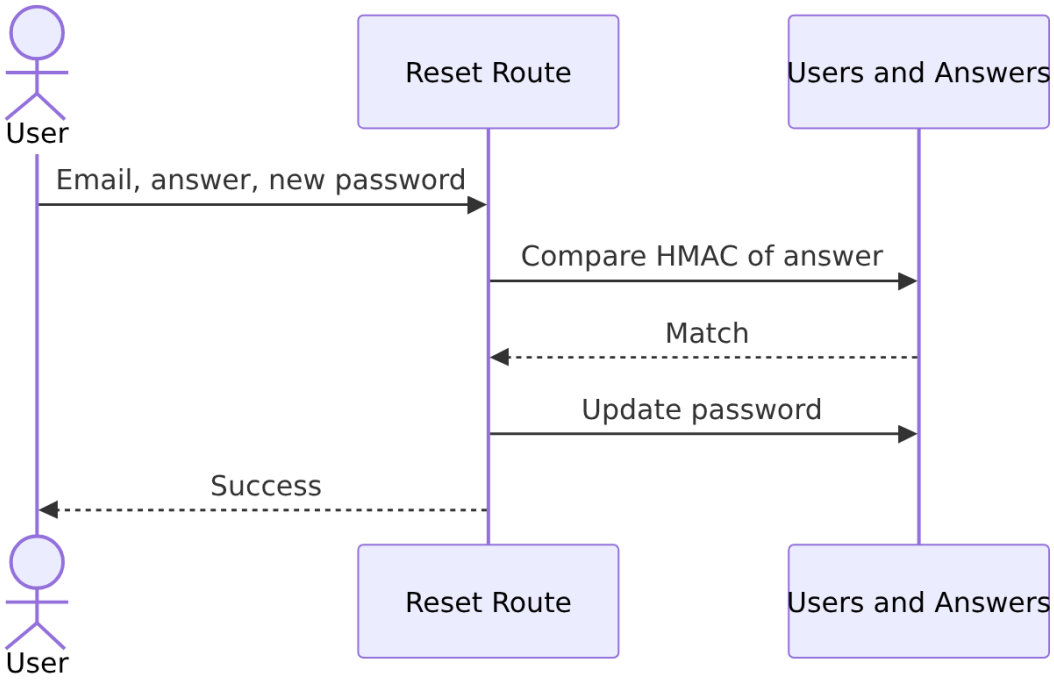
- None linked to this control; see the other findings of this category.

6.2.4 Password reset via security question

Status: ● Weak - an email and a security-question answer are the only proof needed to set a new password.

Password reset compares an HMAC of the submitted answer with the stored one, then updates the password. A separate route reveals the security question for an email.

The diagram shows the intended reset path:



Security assessment

Account takeover needs only a guessable or researchable answer:

- Reset has no out-of-band step and no notification (● F-021).
- The question disclosure route distinguishes existing accounts (● F-065).
- Password change can skip the current-password check (● F-017).

Relevant findings

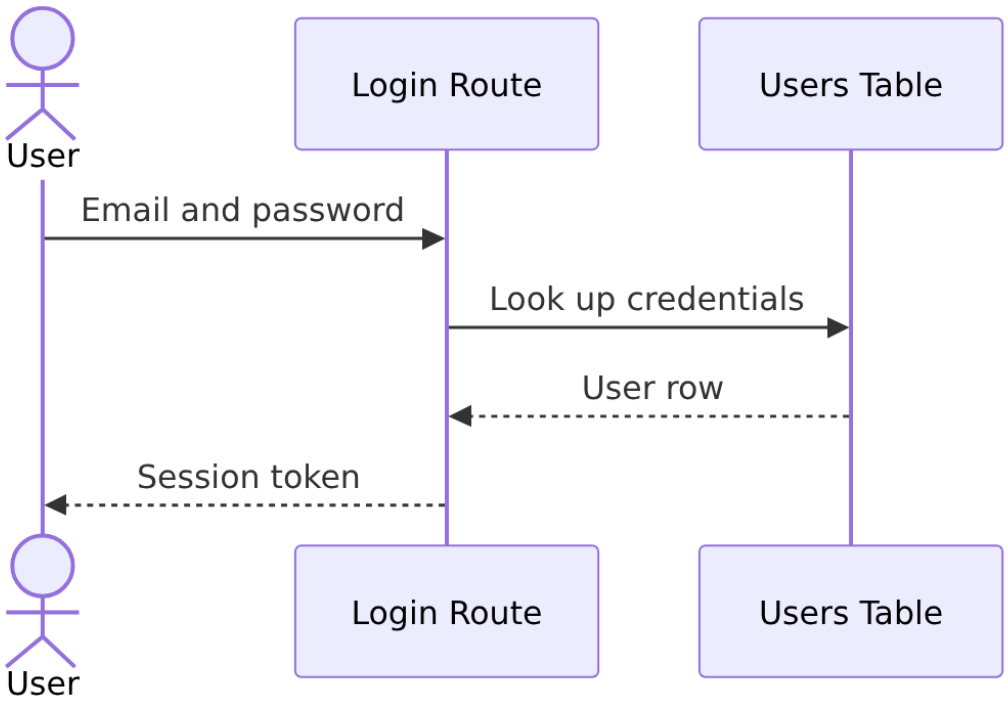
- None linked to this control; see the other findings of this category.

Brute-force protection on login.

Status: ● Missing - the login handler applies no lockout or throttling.

This control is not evidenced in the repository.

The diagram shows the plain login path where an attempt limit would need to sit:



Security assessment

Unlimited guessing is possible against the login endpoint:

- No attempt limit exists on the handler or its registration (● F-045).
- Fast unsalted `MD5` hashing makes offline guessing cheap (● F-032 — Passwords hashed with unsalted fast MD5 (Passwords hashed with unsalted fast `MD5`)).
- A limiter elsewhere in `server.ts` was not inspected.

Relevant findings

- None linked to this control; see the other findings of this category.

6.3 Session and Token Controls

Verdict: ● Weak - `RS256` tokens are used, but the signing key and cookie secret are in source, tokens sit in script-readable storage, and revocation does not exist.

Controls covered:

- 6.3.1 Cookie signing secret
- 6.3.2 Client-side session token storage
- 6.3.3 JWT signing and verification
- 6.3.4 In-memory token map and session cookie

Implemented controls: Cookie signing secret, Client-side session token storage, JWT signing and verification, In-memory token map and session cookie.

Assessment: This application uses a single locally-signed token format (JWT) for every authenticated session, regardless of the §6.2 flow that established it. The sub-sections below trace the lifecycle: cookie signing, browser storage, signing and validation, and revocation with cookie attributes. Storage, revocation, and signing-key handling are the weakest stages.

Findings in this category without a control:

- ● F-004 — Session Token Stored in localStorage
- ● F-076 — Session token cookie without security attributes
- ● F-091 — Sessions cannot be revoked
- ● F-113 — Session token stored in web-accessible client storage

6.3.1 Cookie signing secret

Status: ● Weak - the cookie-signing secret is a literal in source.

`cookieParser` in `server.ts` is initialised with a fixed literal secret that signs cookies.

Security assessment

Anyone with the source can forge signed cookies:

- The secret is committed to the repository (● F-040).

Relevant findings

- None linked to this control; see the other findings of this category.

6.3.2 Client-side session token storage

Status: ● Weak - the token is readable by any script in the page origin.

`login()` in `login.component.ts` writes the JWT to `localStorage` and to a script-set cookie that expires after 8 hours.

Security assessment

Both token copies are exposed to injected script:

- The token lives in script-accessible storage (● F-004 — Session Token Stored in localStorage (Session Token Stored in `localStorage`), ● F-113 — Session token stored in web-accessible client storage).
- The cookie is set from JavaScript, so it cannot be HttpOnly.
- Only the login error path removes the token.

Relevant findings

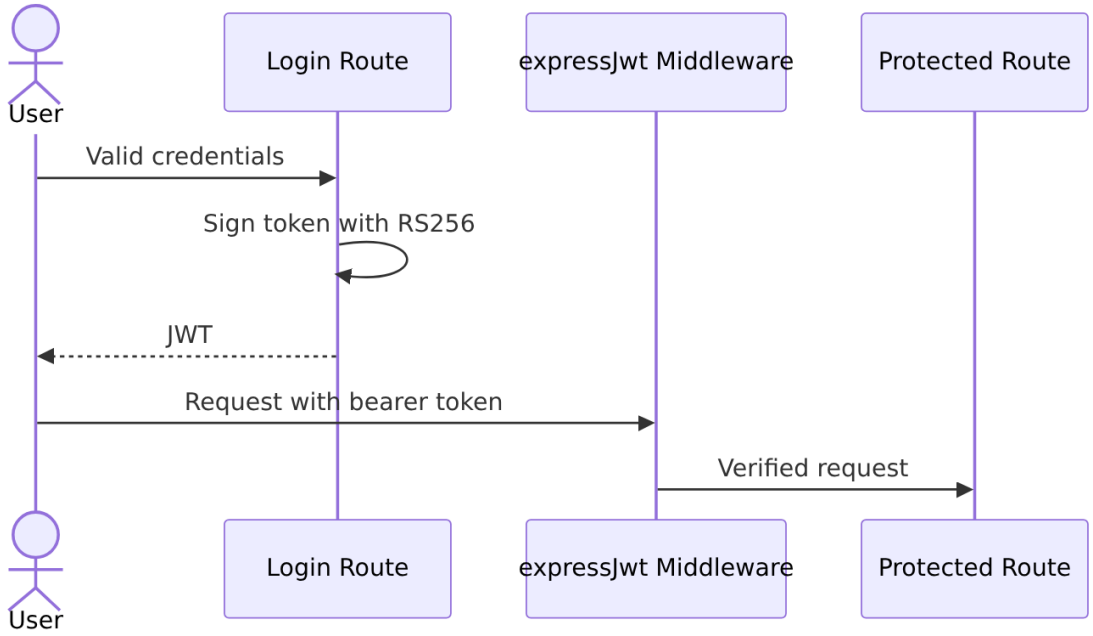
- None linked to this control; see the other findings of this category.

6.3.3 JWT signing and verification

Status: ● Partial - `RS256` signing and verification exist, but the key is embedded and the algorithm is not pinned.

Tokens are signed with `RS256`, expire after 6 hours, and `expressJwt` verifies them against the public key. The full user record goes into the payload.

The diagram shows token issuance and validation on a protected request:



Security assessment

Signing is sound in format but the surrounding handling is not:

- The private key is a string literal in source (● F-005).
- The `verify()` call passes no algorithm allowlist (● F-016).
- The payload carries the user record including the password hash (● F-075).
- The chat route and client guards trust unverified claims (● F-015, ● F-089).

Relevant findings

- None linked to this control; see the other findings of this category.

6.3.4 In-memory token map and session cookie

Status: ● Weak - sessions are never removed and the cookie carries no security attributes.

`lib/insecurity.ts` keeps issued tokens in a process-local map and sets the token cookie through `res.cookie()` without an options object.

Security assessment

Revocation and cookie hardening are both absent:

- No code deletes entries from the token map, so sessions cannot be revoked (● F-091).
- The cookie has no `httpOnly`, `secure`, or `sameSite` attributes (● F-076).

Relevant findings

- None linked to this control; see the other findings of this category.

6.4 Authorization Controls

Systemic weaknesses: W-002, W-020, W-021, W-022, W-023 **Verdict:** ● Missing - server-side authorization is not applied consistently, and object ownership checks are absent across the API.

Controls covered:

- 6.4.1 Route-level authorization middleware
- 6.4.2 LLM agent tool authorization and coupon policy
- 6.4.3 Per-event authorization on shared notification state
- 6.4.4 Role checks

Implemented controls: Route-level authorization middleware, LLM agent tool authorization and coupon policy, Role checks.

Assessment: Guards cover baskets, complaints, B2B, and 2FA routes, but coverage is uneven. Product update, registration, and several data routes lack guards (● F-036, ● F-034, ● F-037), ownership checks are missing (● F-001, ● F-006), and Angular guards decide access from unverified claims (● F-089). The missing piece is a server-side authorization pattern.

Findings in this category without a control:

- ● F-001 — Missing Object Ownership Checks
- ● F-006 — Systemic IDOR Missing Ownership Checks
- ● F-009 — Broken authorization
- ● F-034 — Sensitive Routes Registered Without Authentication Middleware
- ● F-036 — Missing authorization on product update route
- ● F-037 — Sensitive Routes Registered Without Authentication Middleware
- ● F-038 — User registration accepts privileged fields
- ● F-050 — Client-supplied coupon data accepted without server-side
- ● F-069 — Order history lookup by vowel-masked email
- ● F-086 — Chatbot coupon discount unbounded server-side
- ● F-089 — Client-side role guards trust unverified JWT claims
- ● F-090 — Any authenticated user can list all accounts

6.4.1 Route-level authorization middleware

Status: ● Partial - guards exist on several route groups but coverage is uneven.

`isAuthorized`, `denyAll`, and `isAccounting` middleware in `server.ts` guard baskets, complaints, B2B, and 2FA routes.

Security assessment

Route coverage has visible holes:

- Product update has its guard commented out (● F-036).
- Sensitive routes are registered without authentication middleware (● F-034, ● F-037).
- Object ownership is not checked (● F-001, ● F-006, ● F-009).
- Any signed-in user can list all accounts (● F-090).

Relevant findings

- None linked to this control; see the other findings of this category.

6.4.2 LLM agent tool authorization and coupon policy

Status: ● Weak - discount limits live only in the system prompt, not in server code.

The chatbot calls tools including `generateCoupon` and `getOrderById`. The order tool checks ownership through the authenticated user, but compares masked emails.

Security assessment

The model is trusted to enforce policy that the server does not:

- The coupon tool accepts any number and issues a coupon without bounds, order check, or authentication (● F-086).
- Client-supplied coupon data is accepted without server-side validation (● F-050).
- Messages from the request body reach the model unfiltered (● F-051).
- External LLM endpoint enforcement was not inspected.

Relevant findings

- None linked to this control; see the other findings of this category.

6.4.3 Per-event authorization on shared notification state

Status: ● Missing - any client can change shared notification state, and no ownership check exists.

This control is not evidenced in the repository.

Security assessment

The global notifications array is replayed to every new connection and mutated through the `notification received` event:

- No ownership or authorization check applies, so any client can delete shared entries.
- The unauthenticated channel is the root cause (● F-043).

Relevant findings

- None linked to this control; see the other findings of this category.

6.4.4 Role checks

Status: ● Partial - role checks verify the signature first but cover only a few roles.

`isAccounting` and `isDeluxe` verify the token signature before reading the role. The deluxe token is an HMAC keyed with the private key.

Security assessment

Server-side role checks are sound where present, but narrow:

- Coverage is limited to the accounting and deluxe roles.
- Frontend guards read the role from an unverified token (● F-089).

Relevant findings

- None linked to this control; see the other findings of this category.

6.5 Query Construction and Data Access Controls

Systemic weaknesses: W-001 **Verdict:** ● Unsafe - raw SQL and string-built NoSQL predicates are in use on login, search, and review paths.

Controls covered:

- 6.5.1 Chatbot tool query construction
- 6.5.2 Sequelize connection and transaction configuration
- 6.5.3 Login SQL query construction

Implemented controls: Chatbot tool query construction, Sequelize connection and transaction configuration.

Assessment: Sequelize backs most data access, but login and search build raw SQL (● F-013, ● F-007), and two routes build executable NoSQL predicates from input (● F-025, ● F-026). The chatbot tools are the better-constrained path.

Findings in this category without a control:

- ● F-007 — SQL injection
- ● F-013 — SQL injection in login query

- ● F-025 — Input in executable NoSQL predicate
- ● F-026 — Input in executable NoSQL predicate

6.5.1 Chatbot tool query construction

Status: ● Unsafe - the id is coerced to a number before use, which limits injection, but a string-built predicate remains.

The chatbot reviews tool coerces the id with `Number()` before building a MongoDB `$where` string. Product search uses a parameterized Sequelize operator, and tool arguments are typed with `zod`.

Security assessment

The chatbot path is mostly contained:

- Numeric coercion limits injection to NaN or digits.
- Parameterized search and schema-typed arguments are adequate.
- The same string-built predicate pattern is exploitable in the review and tracking routes (● F-025, ● F-026).

Relevant findings

- None linked to this control; see the other findings of this category.

6.5.2 Sequelize connection and transaction configuration

Status: ● Partial - transactions and retry are configured, and the rating covers connection setup only.

`createSequelize` in `models/index.ts` configures SQLite with IMMEDIATE transactions and bounded retry on `SQLITE_BUSY`.

Security assessment

The rating is narrow by design:

- Write integrity and availability under contention are supported.
- Query construction in models and callers was not reviewed here.
- No finding is routed to this control.

Relevant findings

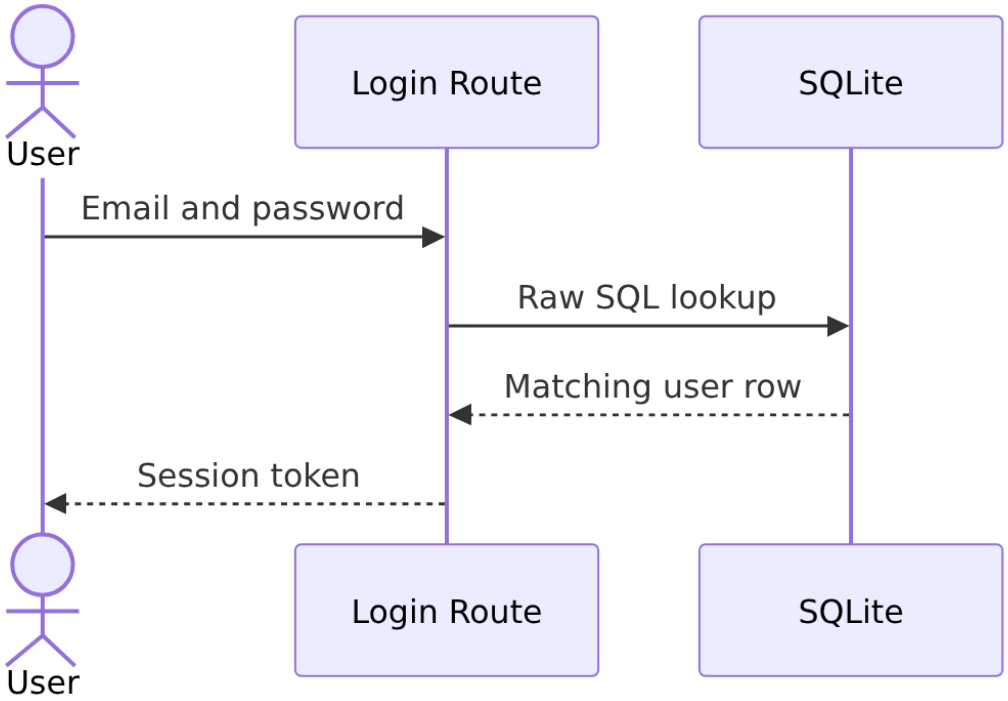
- None linked to this control; see the other findings of this category.

6.5.3 Login SQL query construction

Status: ● Missing - the login query is built by concatenating the email into raw SQL.

This control is not evidenced in the repository.

The diagram shows the login lookup path:



Security assessment

No parameter binding separates query structure from input on this path:

- `req.body.email` is interpolated into the login query, which allows authentication bypass (● F-013).
- Product search follows the same raw construction (● F-007).

Relevant findings

- None linked to this control; see the other findings of this category.

6.6 Input Boundary Validation Controls

Verdict: ● Missing - rate limits cover two endpoints only, and wallet, upload, chat, and amount inputs have no boundary checks.

Controls covered:

- 6.6.1 Validation Approach
- 6.6.2 Rate limiting
- 6.6.3 Request input validation for wallet address
- 6.6.4 Client-side login form validation
- 6.6.5 Guest basket sessionStorage parsing
- 6.6.6 WebSocket message payload validation

Implemented controls: Rate limiting, Client-side login form validation, Guest basket `sessionStorage` parsing, WebSocket message payload validation.

Assessment: Validation is sparse. Rate limits exist for reset and 2FA only, wallet and top-up values are unchecked (● F-047, ● F-082), and chat and upload paths have no size or usage bounds (● F-079, ● F-080, ● F-081). Only guest basket parsing is solid.

Findings in this category without a control:

- ● F-047 — Wallet top-up accepts unvalidated balance amount
- ● F-077 — Blocking busy-wait sleep reachable from review
- ● F-079 — Unbounded unauthenticated chat consumption
- ● F-080 — Disk upload without size limit
- ● F-081 — Unbounded disk upload processed before authentication
- ● F-082 — Unbounded wallet set growth without validation

6.6.1 Validation Approach

Status: ● Weak - bounds exist on a few inputs, while chat, uploads, wallet, and review paths are unbounded.

Validation is handled per route. The guest basket parser and the in-memory upload cap are the clearest examples of enforced input bounds.

Security assessment

Boundary checks are uneven across the API:

- Wallet top-up accepts a raw balance value (● F-047).
- Chat consumption has no input or output bounds (● F-079).
- Disk uploads have no size limit (● F-080, ● F-081).
- A blocking sleep is reachable from review input (● F-077).

Relevant findings

- None linked to this control; see the other findings of this category.

6.6.2 Rate limiting

Status: ● Weak - limits exist only on reset-password and 2FA, and the reset key can be rotated.

`express-rate-limit` guards the reset-password and 2FA endpoints. The reset key generator reads the `X-Forwarded-For` header.

Security assessment

Throttling misses the endpoints that need it most:

- Login has no visible limit (● F-045).
- The 2FA limit is per IP, with no per-user lockout (● F-044).
- Chat has no limit (● F-079).
- A client can rotate `X-Forwarded-For` to evade the reset limit.

Relevant findings

- None linked to this control; see the other findings of this category.

6.6.3 Request input validation for wallet address

Status: ● Missing - wallet addresses are stored without type, format, or length checks.

This control is not evidenced in the repository.

Security assessment

`req.body.walletAddress` is added to an in-memory `Set` directly:

- No format or length validation applies.
- Repeated calls grow the set without bound (● F-082).

Relevant findings

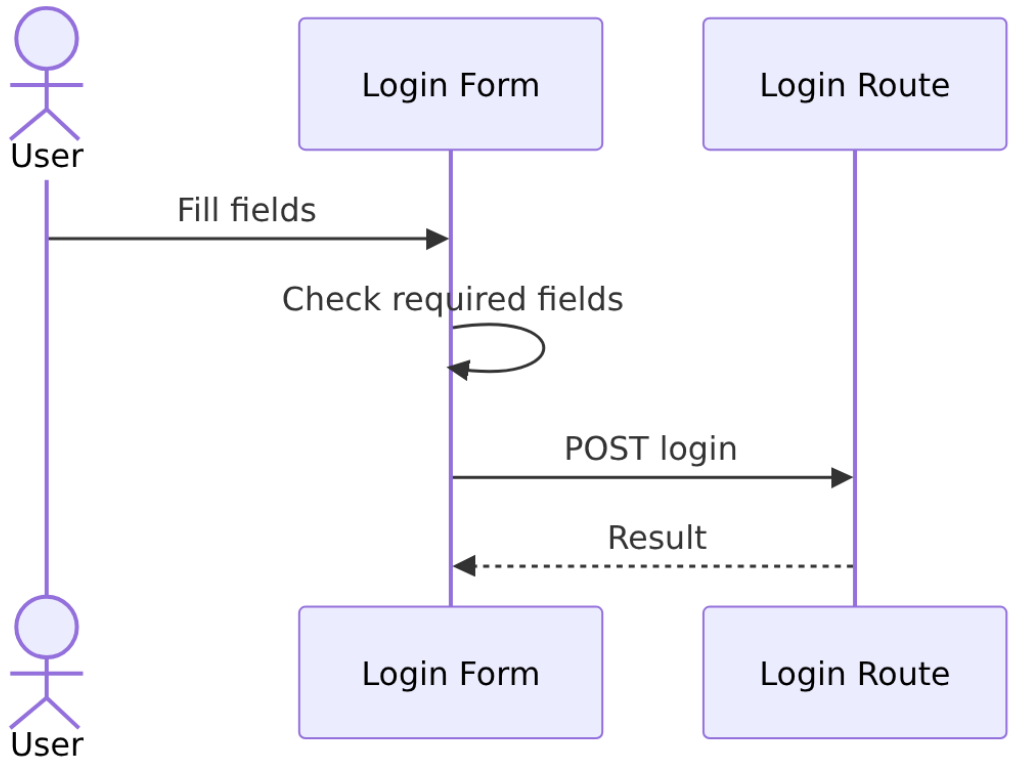
- None linked to this control; see the other findings of this category.

6.6.4 Client-side login form validation

Status: ● Weak - the form checks only that fields are present, and attackers can bypass it anyway.

`login.component.ts` marks the email and password fields as required with a minimum length of one.

The diagram shows the client-side form path before the request is sent:



Security assessment

Client validation is a usability aid only:

- No email format is enforced.
- Direct requests skip the form entirely.
- Server-side validation was not inspected here.

Relevant findings

- None linked to this control; see the other findings of this category.

6.6.5 Guest basket sessionStorage parsing

Status: ● Adequate - stored basket data is parsed defensively and malformed items are dropped.

`getGuestBasketItems` in `basket.service.ts` parses `sessionStorage` in a try/catch and coerces each item's product id and quantity to a positive integer.

Security assessment

This is the one well-formed input check in the section:

- Malformed items are dropped rather than passed on.
- Server-side checks still decide what is accepted.
- No finding is routed to this control.

Relevant findings

- None linked to this control; see the other findings of this category.

6.6.6 WebSocket message payload validation

Status: ● Weak - handlers take untyped data with little validation, and any client can call them.

Socket handlers receive data typed `any`. The SVG handler uses a regex and `isRedirectAllowed`, and the close-notifications handler checks `Array.isArray`.

Security assessment

Validation depth varies by handler:

- Some handlers use only `utils.contains` or `findIndex`.
- A non-string payload raises an unhandled error (● F-083).
- Any unauthenticated client can invoke handlers, which can solve challenges or delete shared notifications (● F-043).

Relevant findings

- None linked to this control; see the other findings of this category.

6.7 Output Encoding and Rendering Controls

Systemic weaknesses: W-006, W-025, W-026, W-027 **Verdict:** ● Weak - raw HTML is bound in several components and unsanitised values are persisted for later rendering.

Controls covered:

- 6.7.1 Output Encoding and Client-Side Rendering

Implemented controls: Angular default template escaping and a `sanitizeSecure` helper on some persistence paths.

Assessment: Default escaping holds for ordinary bindings, but trusted-HTML bypasses (● F-002, ● F-003) and `document.write` (● F-028) defeat it. Product descriptions, emails, and login IP headers are stored unsanitised (● F-052, ● F-053, ● F-061), so output encoding is the only barrier.

Findings in this category without a control:

- ● F-061 — Unsanitised login IP header stored for later

6.7.1 Output Encoding and Client-Side Rendering

Status: ● Weak - default escaping exists but is bypassed in several components, and stored values are not sanitised.

Users see feedback, product claims, and exported data rendered in their browser. Standard template escaping handles ordinary values, and a sanitising helper runs on some write paths.

Security assessment

The frontend marks feedback and claim text as trusted HTML and binds it to `innerHTML`. `data-export.component.ts:71` passes server data straight to `document.write`. On the write side, `models/product.ts:45`, `models/user.ts:60`, and `routes/saveLoginIp.ts:32` persist raw values because sanitisation runs only in the else branch of a challenge flag.

Relevant findings

- ● F-002 — XSS via Trusted HTML Binding
- ● F-003 — Angular Sanitizer Bypass XSS
- ● F-028 — DOM XSS
- ● F-052 — Product description stored without sanitization
- ● F-053 — User email stored without sanitization

6.8 Browser and Cross-Origin Controls

Verdict: ● Weak - the only catalogued browser control is a single hardcoded Socket.io origin, and cross-site request protections are missing elsewhere.

Controls covered:

- 6.8.1 Socket.io CORS origin restriction

Implemented controls: Socket.io CORS origin restriction.

Assessment: The Socket.io CORS origin is restricted but hardcoded. The OAuth flow lacks a state parameter (● F-042), and cookie-authenticated profile updates have no CSRF protection (● F-049).

Findings in this category without a control:

- ● F-042 — OAuth implicit flow without state or nonce
- ● F-049 — Cookie-authenticated profile update

6.8.1 Socket.io CORS origin restriction

Status: ● Partial - CORS is restricted to one origin, but it is hardcoded and not an authentication control.

The `socket.io` server allows one hardcoded browser origin, `http://localhost:4200`.

Security assessment

The origin restriction has limited reach:

- It limits browser cross-origin reads only, and non-browser clients ignore it.

- The localhost value does not fit a deployed environment.
- Cross-site gaps sit in neighbouring flows (● F-042, ● F-049).

Relevant findings

- None linked to this control; see the other findings of this category.

6.9 Cryptography Secrets and Data Protection

Systemic weaknesses: W-041 **Verdict:** ● Weak - secrets are committed to source and password and TOTP data are stored with weak protection.

Controls covered:

- 6.9.1 Database access credentials and storage protection
- 6.9.2 RPC API key handling and transport
- 6.9.3 Sensitive data in URL query strings
- 6.9.4 Database storage protection
- 6.9.5 Password Hashing and Credential Storage
- 6.9.6 Hard-coded HMAC key for security answers

Implemented controls: Database access credentials and storage protection, RPC API key handling and transport, Sensitive data in URL query strings, Database storage protection (SQLite file), Password hashing.

Assessment: Signing keys and the HMAC key are literals in source (● F-005, ● F-073), the cookie secret is hardcoded (● F-040), and an encryption key directory is served (● F-068). TOTP secrets are plaintext (● F-072) and passwords use unsalted `MD5` (● F-032 (Passwords hashed with unsalted fast `MD5`)).

Findings in this category without a control:

- ● F-005 — Hardcoded Secrets in Source
- ● F-040 — Hard-coded cookie signing secret
- ● F-068 — Unauthenticated encryption key directory exposure
- ● F-072 — TOTP secrets stored in plaintext
- ● F-073 — Security answers HMAC with hardcoded key

6.9.1 Database access credentials and storage protection

Status: ● Weak - the SQLite file has placeholder credentials that the engine ignores.

`createSequelize` in `models/index.ts` points Sequelize at a plain SQLite file, `data/juiceshop.sqlite`, with placeholder credentials.

Security assessment

Protection depends on the host, not the database:

- The engine enforces no login, so access control is file permissions and process isolation.

- No at-rest encryption appears in this file. Volume-level encryption was not inspected.

Relevant findings

- None linked to this control; see the other findings of this category.

6.9.2 RPC API key handling and transport

Status: ● Partial - the key comes from an environment variable over `wss`, but it sits in the URL path and fails open when unset.

The Alchemy RPC key is read from an environment variable and used on a `wss` connection.

Security assessment

Handling is better than hardcoding but still loose:

- The key is embedded in the URL path.
- It falls back to an empty string when unset, so misconfiguration fails open.
- No finding is routed to this control.

Relevant findings

- None linked to this control; see the other findings of this category.

6.9.3 Sensitive data in URL query strings

Status: ● Weak - passwords and an OAuth token travel in URL query strings.

`changePassword` and `oauthLogin` in `user.service.ts` build request URLs that include the current password, new password, and access token.

Security assessment

Query strings leak into logs, history, and `Referer` headers:

- Passwords are concatenated into a GET URL without encoding (● F-074).
- Special characters can alter the query string.

Relevant findings

- None linked to this control; see the other findings of this category.

6.9.4 Database storage protection

Status: ● Weak - the database is an unencrypted file protected only by host permissions.

`models/index.ts` configures the sqlite dialect with file storage, and the embedded engine has no login of its own.

Security assessment

No database-level protection is visible:

- No encryption at rest or database access control appears in the inspected source.
- Sensitive columns such as card numbers are stored unencrypted (● F-071).
- Host filesystem permissions are not shown by the source.

Relevant findings

- None linked to this control; see the other findings of this category.

6.9.5 Password Hashing and Credential Storage

Status: ● Weak - passwords use unsalted `MD5`, a single fast hash with no work factor.

Password hashing happens in a shared helper in `lib/insecurity.ts`, used at login and in 2FA setup and disable.

Security assessment

A database dump yields recoverable passwords:

- Unsalted `MD5` is open to rainbow-table and brute-force recovery (● F-032 — Passwords hashed with unsalted fast MD5 (Passwords hashed with unsalted fast `MD5`)).
- The JWT payload also embeds the hash (● F-075).

Relevant findings

- None linked to this control; see the other findings of this category.

6.9.6 Hard-coded HMAC key for security answers

Status: ● Weak - the HMAC key for security answers is a literal anyone with the source can read.

`models/securityAnswer.ts` stores security answers through `security.hmac`, an HMAC-SHA256 with a fixed key.

Security assessment

The key is not secret, so answers can be guessed offline:

- The key is a source literal with no per-record salt or work factor (● F-073).
- Reset depends on this answer alone (● F-021).

Relevant findings

- None linked to this control; see the other findings of this category.

6.10 File Parser and Outbound Request Controls

Systemic weaknesses: W-003, W-040 **Verdict:** ● Missing - the profile image fetch has no SSRF controls, and parser and upload handling are exploitable.

Controls covered:

- 6.10.1 Profile image URL fetch
- 6.10.2 Upload size and type limits

Implemented controls: Upload size and type limits.

Assessment: Upload limits are partial. Archive and XML handling allow path traversal and entity expansion (● F-023, ● F-031), the profile image fetch is open to request forgery (● F-070), and code is evaluated from request data (● F-033, ● F-048). Exposure of logs and key files adds further leaks.

Findings in this category without a control:

- ● F-022 — Path Traversal
- ● F-023 — Zip Slip path traversal during archive extraction
- ● F-027 — Input compiled as template source
- ● F-030 — Unauthenticated log directory listing
- ● F-031 — XML external entity expansion in upload parser
- ● F-033 — Code injection
- ● F-046 — Substring redirect allowlist check
- ● F-048 — Request data evaluated by notevil in vm sandbox
- ● F-063 — JSONP and arbitrary field selection on current user
- ● F-064 — Unauthenticated memories listing joins user records
- ● F-070 — Server-side request forgery
- ● F-075 — Full user record with password hash in JWT
- ● F-078 — Unbounded zip extraction output size

6.10.1 Profile image URL fetch

Status: ● Missing - the server fetches any requester-supplied URL with no host or scheme validation.

This control is not evidenced in the repository.

Security assessment

Outbound fetches are unrestricted:

- Server-side request forgery is possible through the profile image URL route (● F-070).
- On fetch failure the raw URL is stored as the profile image.
- The only URL inspection flags a challenge.

Relevant findings

- None linked to this control; see the other findings of this category.

6.10.2 Upload size and type limits

Status: ● Partial - size and MIME limits exist for memory uploads, and disk uploads lack a size cap.

In-memory uploads are capped at 200 KB. Disk uploads check the client-declared MIME type against an allowlist and sanitise the file name.

Security assessment

The limits are shallow:

- The MIME check trusts the client header.
- Disk uploads have no size limit (● F-080, ● F-081).
- Decompressed archive output is not bounded (● F-078).
- Archive extraction accepts traversal paths (● F-023).

Relevant findings

- None linked to this control; see the other findings of this category.

6.11 Operations Runtime and Supply Chain Controls

Build path. Figure 1b shows the inputs, build systems and release artifacts that Juice Shop evidences.

Highlighted path of ● F-056 — Dependency Install Scripts Run at Build Time: npm registry → GitHub Actions → docker.io/bkimminich/juice-shop. The path stops where the evidence ends.

Attack entries:

- **Manipulated dependency**
  - ● F-056 — Dependency Install Scripts Run at Build Time
  - ● F-057 — Unaudited npm Lifecycle Script
  - ● F-059 — Non-reproducible dependency installs in release builds
- **Manipulated CI input**
  - ● F-055 — Remote install script piped to shell in deploy job
  - ● F-058 — Unpinned Third-Party GitHub Action
  - ● F-060 — Unpinned floating container base images
- **Replaced release artifact**
  - ● F-054 — Missing Container Image Signing

Not attributable to a build step:

- ● F-087 — Container Runs as Root

Systemic weaknesses: W-010, W-024 **Verdict:** 🔴 Missing - pipeline hardening has gaps in lockfile enforcement, install integrity, and base image pinning, and management endpoints are exposed.

Controls covered:

- 6.11.1 Sensitive endpoint exposure
- 6.11.2 GitHub Actions pinning to commit SHAs
- 6.11.3 Dependency installation integrity in build and release
- 6.11.4 Container image hardening
- 6.11.5 CI secret handling and token scoping
- 6.11.6 Build script isolation in lint job
- 6.11.7 Error handling and logging for WebSocket provider
- 6.11.8 Build pipeline and dependency controls

Implemented controls: Sensitive endpoint exposure (metrics, log directory, API docs), GitHub Actions pinning to commit SHAs, Dependency installation integrity in build and release, Container image hardening, CI secret handling and token scoping.

Assessment: Container hardening and lint-job isolation hold, but management endpoints are unauthenticated (🟠 F-030, 🟠 F-066, 🟠 F-067), builds resolve dependencies at install time (🟠 F-059, 🟠 F-060), and a remote script is piped to a shell (🟠 F-055). Workflow permissions are unset (🟠 F-088).

Findings in this category without a control:

- 🟠 F-054 — Missing Container Image Signing
- 🟠 F-055 — Remote install script piped to shell in deploy job
- 🟠 F-059 — Non-reproducible dependency installs in release builds
- 🟠 F-067 — Unauthenticated log directory listing and download
- 🟠 F-087 — Container Runs as Root
- 🟠 F-088 — Missing Workflow Permissions Block

6.11.1 Sensitive endpoint exposure

Status: 🟠 Weak - logs, metrics, and API docs are served without an authentication check.

`server.ts` registers a log directory listing, `/metrics`, and `/api-docs`. Access logging uses `morgan` with daily rotation and two-day retention.

Security assessment

Operational data is publicly reachable:

- The log directory is listed and downloadable (🟠 F-030, 🟠 F-067).
- The metrics endpoint has no authentication (🟠 F-066).
- The encryption key directory is also exposed (🟠 F-068).

Relevant findings

- None linked to this control; see the other findings of this category.

6.11.2 GitHub Actions pinning to commit SHAs

Status: 🟠 Partial - most actions are pinned to commit SHAs, with at least one mutable tag.

Release and Docker login actions in the workflows are pinned to full commit SHAs.

Security assessment

Pinning is incomplete:

- `coverallsapp/github-action@v2` uses a mutable tag and receives `GITHUB_TOKEN` (🟠 F-058).
- An archived action is pinned by SHA but flagged in a TODO for replacement.

Relevant findings

- None linked to this control; see the other findings of this category.

6.11.3 Dependency installation integrity in build and release

Status: 🟠 Weak - builds resolve dependencies at install time instead of from the lockfile.

Release and image builds run `npm install`, and global tools are installed from semver ranges.

Security assessment

Release output may differ from what was reviewed:

- `npm install` is used instead of `npm ci` (🟠 F-059).
- The builder image uses a floating `node:24` tag (🟠 F-060).
- Lockfile enforcement was not inspected.

Relevant findings

- None linked to this control; see the other findings of this category.

6.11.4 Container image hardening

Status: 🟢 Adequate - a multi-stage distroless image runs as a non-root user.

The `Dockerfile` builds in stages and ships a distroless runtime image as UID 65532, with dev dependencies omitted and an SBOM generated at build.

Security assessment

This is the strongest operational control, with caveats:

- The base image is not digest-pinned (🟠 F-060).

- A separate finding reports root execution (● F-087), which should be reconciled against the non-root runtime stage.

Relevant findings

- None linked to this control; see the other findings of this category.

6.11.5 CI secret handling and token scoping

Status: ● Partial - secrets are step-scoped, but workflow token permissions are not restricted.

Secrets pass through GitHub secrets and attach to single steps, for example `ALCHEMY_API_KEY` on the API test step. The release workflow runs only on `v*` tags.

Security assessment

Token scope is wider than needed:

- No top-level `permissions:` block is set, so `GITHUB_TOKEN` uses repository defaults (● F-088).
- Pull request triggers can run install lifecycle scripts on PR code in test jobs.
- Fork-PR secret exposure was not verified.

Relevant findings

- None linked to this control; see the other findings of this category.

6.11.6 Build script isolation in lint job

Status: ● Adequate - the lint job installs with scripts disabled.

The lint job in `ci.yml` installs with `--ignore-scripts`, and fork test jobs run a single ubuntu and node-24 combination.

Security assessment

Isolation holds for lint only:

- Other jobs run full installs.
- Install-time scripts remain a concern elsewhere (● F-056, ● F-057).

Relevant findings

- None linked to this control; see the other findings of this category.

6.11.7 Error handling and logging for WebSocket provider

Status: ● Partial - errors are logged and recovery works, but error text reaches the client.

The WebSocket provider logs errors and resets its listener flag so the listener can be recreated.

Security assessment

Error handling is partial:

- The 500 handler returns the message from `utils.getErrorMessage`, which may leak details.
- Listener creation and the verify endpoint have no rate limit in the inspected files.
- No finding is routed to this control.

Relevant findings

- None linked to this control; see the other findings of this category.

6.11.8 Build pipeline and dependency controls

Status: 🔴 Missing - gaps in Lockfile hygiene, CI install integrity, Action pinning, Base image pinning, Install scripts

Build and release run through GitHub Actions workflows and Dockerfiles. Automated SCA scanning and static analysis are in place, which the lifecycle bullets below grade.

- **Automated SCA scanning** — 🟢 Adequate. Automated SCA scanning present: `.gitlab-ci.yml:2`.
- **Automated dependency updates** — Not evidenced in the repository; it may run outside it (GitHub App, default setup, organization policy, agent).
- **Lockfile hygiene** — 🔴 Missing. Lockfile hygiene not detected in the repository (.gitignore: 10, .npmrc:1).
- **CI install integrity** — 🔴 Missing. CI installs dependencies by resolving versions at install time: `.github/workflows/ci.yml:36`, `.github/workflows/ci.yml:38`, `.github/workflows/ci.yml:51`.
- **Action pinning** — 🟠 Partial. Some CI action or image references use a mutable tag or branch: `.github/workflows/ci.yml:188`, `.github/workflows/codeql-analysis.yml:23`, `.github/workflows/codeql-analysis.yml:34`. → 🟠 F-058 — Unpinned Third-Party GitHub Action
- **Base image pinning** — 🔴 Missing. Base images use no tag or a mutable tag: `Dockerfile:1`, `Dockerfile:22`, `test/smoke/Dockerfile:1`. → 🟠 F-060 — Unpinned floating container base images
- **Install scripts** — 🟠 Weak. A build step executes code fetched from an external URL: `.github/workflows/ci.yml:358`. → 🟠 F-056 — Dependency Install Scripts Run at Build Time, 🟠 F-057 — Unaudited npm Lifecycle Script
- **Static analysis (SAST)** — 🟢 Adequate. Static analysis runs in CI: `.github/workflows/codeql-analysis.yml:36`, `.gitlab-ci.yml:2`.

Security assessment

Gaps: Lockfile hygiene (Missing), CI install integrity (Missing), Action pinning (Partial), Base image pinning (Missing), Install scripts (Weak). Not evidenced in the repository: Automated dependency updates.

Relevant findings

- ● F-058 — Unpinned Third-Party GitHub Action
- ● F-060 — Unpinned floating container base images
- ● F-056 — Dependency Install Scripts Run at Build Time
- ● F-057 — Unaudited npm Lifecycle Script

6.12 Real-time and Not Applicable Controls

Verdict: ● Weak - the real-time channel accepts unauthenticated clients and untyped events.

Controls covered:

- 6.12.1 Socket.io client connection

Implemented controls: Socket.io client connection.

Assessment: `Socket.IO` is in use, so this domain applies. The channel has no connection authentication (● F-043) and handlers can fail on unexpected payloads (● F-083); CORS is the only restriction.

6.12.1 Socket.io client connection

Status: ● Weak - the client connects with no credential, and server-side authorization is absent.

`socket-io.service.ts` opens the connection in its constructor using the page origin or a configured host.

Security assessment

The client sends no identity, so the server cannot tell callers apart:

- No credential or auth option is passed (● F-043).
- No transport enforcement is visible for a configured host.
- Server-side connection authorization is covered in §6.2.

Relevant findings

- None.

6.13 Defense-in-Depth Summary

What holds

- Distroless non-root runtime image.

- `RS256` token signing with expiry.
- Pinned actions for most workflows.
- Lint job without install scripts.

Repair first

- Replace raw SQL and string-built predicates on login, search, and review routes.
- Add server-side authorization and ownership checks on routes and the chat tools.
- Move signing keys and secrets out of source.
- Authenticate the WebSocket channel and add login throttling.

---

7. Weakness Register

Systemic control gaps behind the findings, ordered by severity (W-001 = most severe). Each weakness names the missing, home-grown, or misused control, the findings that evidence it, the components it spans, and its remediation. A weakness may also rest on observed unsafe practice or an absent architectural control with no confirmed exploit - only confirmed findings carry a CVSS score.

- ● **Critical** · W-001 - Database access relies on concatenated queries · confirmed · 2 findings · 2 components
- ● **Critical** · W-002 - Authorization is implemented route by route · confirmed · 9 findings · 2 components
- ● **High** · W-003 - Application data crosses into executable code · confirmed · 2 findings · 1 component
- ● **High** · W-004 - Document-database queries take their structure from application values · observed-practice · 2 findings · 1 component
- ● **High** · W-005 - Browser scripts retain reusable session credentials · observed-practice · 1 finding · 1 component
- ● **High** · W-006 - Frontend rendering lacks enforced output encoding · design-risk · 4 findings · 2 components
- ● **High** · W-007 - Secrets are committed to source instead of a managed store · design-risk · 3 findings · 4 components
- ● **Medium** · W-008 - Authentication relies on hand-rolled token verification · confirmed · 3 findings · 3 components
- ● **Medium** · W-009 - Endpoints are reachable without enforced authentication · confirmed · 2 findings · 2 components
- ● **Medium** · W-010 - Build pipeline trusts mutable third-party references · confirmed · 3 findings · 1 component
- ● **Medium** · W-011 - Weak 2FA code guessing bounds (`2fa.ts:31`) · observed-practice · 1 finding · 1 component
- ● **Medium** · W-012 - No brute-force protection on login (`login.ts:32`) · observed-practice · 1 finding · 1 component

- ● **Medium** · W-013 - Blocking busy-wait sleep reachable from review query (`showProductReviews.ts:23`) · observed-practice · 1 finding · 1 component
- ● **Medium** · W-014 - Unbounded unauthenticated chat consumption (`chat.ts:203`) · observed-practice · 1 finding · 1 component
- ● **Medium** · W-015 - Disk upload without size limit (`server.ts:698`) · observed-practice · 1 finding · 1 component
- ● **Medium** · W-016 - Unbounded disk upload processed before authentication (`server.ts:698`) · observed-practice · 1 finding · 1 component
- ● **Medium** · W-017 - Application data builds operating-system commands · observed-practice · 0 findings · 1 component
- ● **Medium** · W-018 - Application data crosses into template source · observed-practice · 1 finding · 1 component
- ● **Medium** · W-019 - Stored username interpolated into system prompt (`chat.ts:83`) · observed-practice · 0 findings · 1 component
- ● **Medium** · W-020 - Client-side role guards trust unverified JWT claims (`app.guard.ts:54`) · observed-practice · 1 finding · 1 component
- ● **Medium** · W-021 - OAuth sign-in requests are not bound to the user's browser session · observed-practice · 1 finding · 1 component
- ● **Medium** · W-022 - Cookie-authenticated changes lack request authenticity checks · observed-practice · 2 findings · 2 components
- ● **Medium** · W-023 - Privileged attributes lack an enforced write allowlist · observed-practice · 1 finding · 1 component
- ● **Medium** · W-024 - Unaudited npm Lifecycle Script · observed-practice · 1 finding · 1 component
- ● **Medium** · W-025 - Product description stored without sanitization (`product.ts:45`) · observed-practice · 1 finding · 1 component
- ● **Medium** · W-026 - User email stored without sanitization (`user.ts:60`) · observed-practice · 1 finding · 1 component
- ● **Medium** · W-027 - Sensitive cookies remain accessible to browser scripts · observed-practice · 1 finding · 1 component
- ● **Medium** · W-028 - JSONP and arbitrary field selection on current user endpoint (`currentUser.ts:58`) · observed-practice · 1 finding · 1 component
- ● **Medium** · W-029 - Unauthenticated memories listing joins user records (`memory.ts:24`) · observed-practice · 1 finding · 1 component
- ● **Medium** · W-030 - Full user record with password hash in JWT (`login.ts:24`) · observed-practice · 1 finding · 1 component
- ● **Medium** · W-031 - Full application configuration served unauthenticated (`appConfiguration.ts:15`) · observed-practice · 0 findings · 1 component
- ● **Medium** · W-032 - Confidential offer embedded in system prompt (`chat.ts:105`) · observed-practice · 0 findings · 1 component
- ● **Medium** · W-033 - Upstream LLM error text streamed to client (`chat.ts:255`) · observed-practice · 0 findings · 1 component
- ● **Medium** · W-034 - Payment card numbers stored in plaintext (`card.ts:38`) · observed-practice · 1 finding · 1 component

- ● **Medium** · W-035 - Unencrypted SQLite database file storage (`index.ts:41`) · observed-practice · 0 findings · 1 component
- ● **Medium** · W-036 - TOTP secrets stored in plaintext (`user.ts:112`) · observed-practice · 1 finding · 1 component
- ● **Medium** · W-037 - Unauthenticated encryption key directory exposure (`keyServer.ts:14`) · observed-practice · 1 finding · 1 component
- ● **Medium** · W-038 - Passwords sent in GET query string unencoded (`user.service.ts:54`) · observed-practice · 1 finding · 1 component
- ● **Medium** · W-039 - OAuth access token placed in query string (`user.service.ts:68`) · observed-practice · 0 findings · 1 component
- ● **Medium** · W-040 - Substring redirect allowlist check (`insecurity.ts:136`) · observed-practice · 1 finding · 1 component
- ● **Medium** · W-041 - Security-sensitive data uses weak cryptographic primitives · observed-practice · 1 finding · 1 component

W-001 — Database access relies on concatenated queries

● **Critical** · design weakness · confirmed · 2 findings

Database queries are assembled from application values instead of passing those values through an enforced parameterised data-access path. This leaves every call site responsible for preserving query structure.

Architectural anti-pattern - Raw SQL string interpolation. Queries are built by concatenating request values across several routes, so one fix cannot close the class. Parameterized queries must become the default data-access path.

Confirmed findings:

- ● F-007 — SQL injection
- ● F-013 — SQL injection in login query

Practice sites:

- `routes/login.ts:34`
- `routes/search.ts:23`

Architecture evidence: Parameterized Queries, ORM / Repository Layer, Source evidence (`routes/vulnCodeSnippet.ts:46`)

Affected components: C-01, C-09

Remediation:

- **Structural** — provide one parameterised repository or query-builder path and prohibit application-value interpolation in database queries
- **Tactical** — ● M-029 — Use parameterized database queries (F-007), ● M-030 — Parameterize the login query with bind replacements

W-002 — Authorization is implemented route by route

● **Critical** · design weakness · confirmed · 9 findings

Authorization depends on per-handler checks instead of a policy boundary that consistently enforces role, ownership, and tenant scope. New routes can bypass protection by omitting a local check.

Architectural anti-pattern - Missing server-side authorization layer. Authorization is implemented route by route with no central ownership or role enforcement, so each new route can silently omit it.

Confirmed findings:

- ● F-006 — Systemic IDOR Missing Ownership Checks
- ● F-009 — Broken authorization
- ● F-034 — Sensitive Routes Registered Without Authentication Middleware
- ● F-036 — Missing authorization on product update route
- ● F-086 — Chatbot coupon discount unbounded server-side
- ● F-090 — Any authenticated user can list all accounts

Practice sites:

- ● F-001 — Missing Object Ownership Checks
- ● F-037 — Sensitive Routes Registered Without Authentication Middleware
- ● F-069 — Order history lookup by vowel-masked email
- `lib/startup/registerWebsocketEvents.ts:34`

Architecture evidence: Centralised AuthZ Policy, Role / Scope Enforcement, Ownership Check, Source evidence (`server.ts:371`), Source evidence (`server.ts:389`), Source evidence (`server.ts:390`) (+66 more)

Affected components: C-01, C-09

Remediation:

- **Structural** — enforce authorization through a shared server-side policy layer and make ownership and tenant scope mandatory inputs to data access
- **Tactical** — ◕ M-028 — Check basket ownership before returning, ● M-023 — Enforce object-level (ownership) authorization (F-001), ◕ M-045 — Enforce server-side authorization on every endpoint (F-036), ◑ M-091 — Enforce coupon limits in server code, ◑ M-093 — Enforce server-side authorization, ◑ M-001 — Manual review: verify Missing Object Ownership Checks, ◑ M-006 — Manual review: verify Sensitive Routes Registered Without Authentication Middleware, ◕ M-046 — Enforce server-side authorization on every endpoint (F-037), ◑ M-074 — Enforce object-level (ownership) authorization (F-069)

W-003 — Application data crosses into executable code

● **High** · implementation weakness · confirmed · 2 findings

An interpreter evaluates application values as code. The affected path mixes data processing with executable program construction instead of constraining callers to predefined operations.

Confirmed findings:

- ● F-033 — Code injection

Practice sites:

- ● F-048 — Request data evaluated by notevil in vm sandbox

Affected components: C-01

Remediation:

- **Structural** — replace evaluation of application values with a data parser or a fixed set of operations whose arguments remain data
- **Tactical** — ◕ M-044 — Remove eval of username in profile rendering, ◑ M-057 — Remove dynamic evaluation of order lines

W-004 — Document-database queries take their structure from application values

● **High** · implementation weakness · observed-practice · 2 findings

Document-store queries take operators or JavaScript predicates from application values instead of fixed query structures with typed values. Callers can change what a query selects, for example through injected operators or evaluated expressions.

Practice sites:

- ● F-025 — Input in executable NoSQL predicate
- ● F-026 — Input in executable NoSQL predicate

Affected components: C-01

Remediation:

- **Structural** — build document queries from fixed operator structures, coerce request values to the expected scalar types, and disable server-side JavaScript evaluation in the database
- **Tactical** — ◑ M-004 — Manual review: verify Input in executable NoSQL predicate, ◕ M-038 — Use typed query predicates instead of $where

W-005 — Browser scripts retain reusable session credentials

● **High** · implementation weakness · observed-practice · 1 finding

Browser code retains reusable authentication credentials in script-readable storage. Script execution in the same origin can obtain the credentials and reuse the authenticated session.

Practice sites:

- ● F-004 — Session Token Stored in localStorage

Affected components: C-05

Remediation:

- **Structural** — move reusable credentials behind a server-managed session boundary and keep browser-visible state free of reusable authentication secrets
- **Tactical** — ◕ M-026 — Move session token to HttpOnly cookie

W-006 — Frontend rendering lacks enforced output encoding

● **High** · design weakness · design-risk · 4 findings

Browser rendering paths write HTML or DOM content directly without one enforced contextual-encoding or sanitisation boundary. Safety depends on each individual component using the right API.

Architectural anti-pattern - SPA without BFF. The Angular frontend keeps the session token in browser storage and no Backend-for-Frontend holds it server-side, so any script injection yields account takeover. Move the token to an HttpOnly Secure `SameSite=Strict` cookie.

Practice sites:

- ● F-002 — XSS via Trusted HTML Binding
- ● F-003 — Angular Sanitizer Bypass XSS
- ● F-028 — DOM XSS
- ● F-061 — Unsanitised login IP header stored for later
- `frontend/src/app/about/about.component.ts:119`
- `frontend/src/app/administration/administration.component.ts:73`
- `frontend/src/app/administration/administration.component.ts:91`
- `frontend/src/app/data-export/data-export.component.ts:57`
- `frontend/src/app/last-login-ip/last-login-ip.component.ts:39`

Architecture evidence: Template Autoescape, Output Encoding, Content Security Policy, Source evidence (`frontend/src/app/about/about.component.ts:119`), Source evidence (`frontend/src/app/last-login-ip/last-login-ip.component.ts:39`), Source evidence (`frontend/src/app/administration/administration.component.ts:73`) (+5 more)

Affected components: C-05, C-09

Remediation:

- **Structural** — use framework-safe rendering by default and isolate any required raw HTML behind one reviewed sanitisation and Trusted Types policy
- **Tactical** — ◕ M-024 — Remove sanitizer bypass and render feedback as text, ◑ M-002 — Manual review: verify Angular Sanitizer Bypass XSS, ◕ M-025 — Remove sanitizer bypass and render IP as text, ◑ M-005 — Manual review: verify DOM XSS via document.write of exported user data, ◕ M-040 — Encode output instead of bypassing the framework sanitizer (F-028), ◑ M-012 — Manual review: verify Unsanitised login IP header stored for later rendering, ◑ M-066 — Encode output instead of bypassing the framework sanitizer (F-061)

W-007 — Secrets are committed to source instead of a managed store

● **High** · design weakness · design-risk · 3 findings

Cryptographic keys, credentials, and other high-entropy secrets are embedded as literals in source or config rather than resolved at runtime from a managed secret store, so anyone with repository read access obtains reusable signing material and credentials.

Architectural anti-pattern - Secrets hardcoded in source. Signing keys and other secrets are committed to the repository instead of a managed store, so anyone with source access can forge sessions.

Practice sites:

- ● F-005 — Hardcoded Secrets in Source
- ● F-040 — Hard-coded cookie signing secret
- ● F-073 — Security answers HMAC with hardcoded key
- `frontend/src/app/login/login.component.ts:62`

Architecture evidence: Managed Secret Store, Runtime Secret Injection, Source evidence (`routes/checkKeys.ts:10`), Source evidence (`lib/insecurity.ts:21`)

Affected components: C-01, C-02, C-05, C-09

Remediation:

- **Structural** — move every secret to a managed secret store or injected environment configuration, rotate the exposed values, and add secret-scanning to CI
- **Tactical** — ◕ M-027 — Load signing keys from a secret store, ◑ M-007 — Manual review: verify Hard-coded cookie signing secret, ◑ M-049 — Load the cookie secret from environment config, ◑ M-078 — Move cryptographic keys to a managed secret store

W-008 — Authentication relies on hand-rolled token verification

● **Medium** · implementation weakness · confirmed · 3 findings

Token validation is implemented in application code rather than behind one vetted and consistently configured authentication path. Algorithm, issuer, audience, and key-handling checks can diverge between callers.

Confirmed findings:

- ● F-054 — Missing Container Image Signing

Practice sites:

- ● F-015 — Chat identity taken from unverified JWT
- ● F-016 — JWT verify call missing algorithms allowlist

Affected components: C-01, C-03, C-04

Remediation:

- **Structural** — centralise token validation on a vetted library with pinned algorithms, issuer and audience checks, and managed signing keys
- **Tactical** — ◑ M-017 — Sign and verify release artifacts, ◕ M-031 — Verify JWT signature, ◑ M-003 — Manual review: verify JWT verify call missing algorithms allowlist, ◕ M-032 — Enforce JWT signature and algorithm verification (F-016)

W-009 — Endpoints are reachable without enforced authentication

● **Medium** · design weakness · confirmed · 2 findings

Sensitive API routes and real-time channels are exposed without an enforced authentication check at the endpoint boundary. Access control depends on each handler (or the caller) remembering to require a session, so an unauthenticated client can reach privileged operations directly.

Confirmed findings:

- ● F-043 — Unauthenticated WebSocket Channel

Practice sites:

- ● F-066 — Unauthenticated metrics endpoint

Architecture evidence: Route Authentication Middleware, Server-Side Session Enforcement, Source evidence (`server.ts:309`), Source evidence (`server.ts:312`), Source evidence (`server.ts:402`), Source evidence (`server.ts:404`) (+4 more)

Affected components: C-01, C-07

Remediation:

- **Structural** — enforce authentication centrally at the routing and channel boundary so every exposed endpoint requires a verified session unless explicitly marked public
- **Tactical** — ◑ M-052 — Require authentication on every exposed endpoint (F-043), ◑ M-071 — Restrict metrics endpoint access

W-010 — Build pipeline trusts mutable third-party references

● **Medium** · design weakness · confirmed · 3 findings

CI/CD workflows resolve third-party actions and other build dependencies to mutable tags or branches instead of immutable commit digests, so a retagged or compromised upstream runs inside the pipeline with its token and secret scope.

Confirmed findings:

- ● F-058 — Unpinned Third-Party GitHub Action

Practice sites:

- ● F-059 — Non-reproducible dependency installs in release builds
- ● F-060 — Unpinned floating container base images

Architecture evidence: Commit-SHA Action Pinning, Dependency Provenance Verification, Source evidence (`.github/workflows/image_actions.yml:33`), Source evidence (`.github/workflows/image_actions.yml:42`), Source evidence (`.github/workflows/ci.yml:188`)

Affected components: C-03

Remediation:

- **Structural** — pin every third-party action and build dependency to an immutable commit SHA (or a vetted internal mirror) and enforce SHA-pinning in CI
- **Tactical** — ◑ M-020 — Third-party GitHub Actions pinned to commit SHA, ◑ M-011 — Manual review: verify Non-reproducible dependency installs in release builds, ◑ M-064 — Commit lockfiles and install from them, ◑ M-065 — Pin base images by digest

W-011 — Weak 2FA code guessing bounds (2fa.ts:31)

● **Medium** · implementation weakness · observed-practice · 1 finding

Practice sites:

- ● F-044 — Weak 2FA code guessing bounds

Affected components: C-09

Remediation:

- **Tactical** — ◑ M-053 — Add per-user lockout and shrink the TOTP window

W-012 — No brute-force protection on login (login.ts:32)

● **Medium** · implementation weakness · observed-practice · 1 finding

Practice sites:

- ● F-045 — No brute-force protection on login

Affected components: C-09

Remediation:

- **Tactical** — ◑ M-009 — Manual review: verify No brute-force protection on login, ◑ M-054 — Add per-account and per-IP throttling to login

W-013 — Blocking busy-wait sleep reachable from review query (showProductReviews.ts:23)

● **Medium** · implementation weakness · observed-practice · 1 finding

Practice sites:

- ● F-077 — Blocking busy-wait sleep reachable from review

Affected components: C-01

Remediation:

- **Tactical** — ◑ M-014 — Manual review: verify Blocking busy-wait sleep reachable from review query, ◑ M-082 — Rate-limit expensive requests and bound input size

W-014 — Unbounded unauthenticated chat consumption (chat.ts:203)

● **Medium** · implementation weakness · observed-practice · 1 finding

Practice sites:

- ● F-079 — Unbounded unauthenticated chat consumption

Affected components: C-01

Remediation:

- **Tactical** — ◑ M-084 — Add rate limit and token caps to chat

W-015 — Disk upload without size limit (server.ts:698)

● **Medium** · implementation weakness · observed-practice · 1 finding

Practice sites:

- ● F-080 — Disk upload without size limit

Affected components: C-01

Remediation:

- **Tactical** — ◑ M-085 — Set fileSize limit on disk uploads

W-016 — Unbounded disk upload processed before authentication (server.ts:698)

● **Medium** · implementation weakness · observed-practice · 1 finding

Practice sites:

- ● F-081 — Unbounded disk upload processed before authentication

Affected components: C-01

Remediation:

- **Tactical** — ◑ M-015 — Manual review: verify Unbounded disk upload processed before authentication, ◑ M-086 — Authenticate first and cap disk upload size

W-017 — Application data builds operating-system commands

● **Medium** · implementation weakness · observed-practice

A command line is assembled from application values and handed to a shell. Shell syntax in those values runs as further commands with the privileges of the service.

Practice sites:

- `.github/workflows/update-news-www.yml:19`

Affected components: C-03

Remediation:

- **Structural** — call a fixed executable with an argument list and no shell, and accept only allowlisted argument values from callers

W-018 — Application data crosses into template source

● **Medium** · implementation weakness · observed-practice · 1 finding

Template construction treats application values as template source. Rendering can therefore interpret data as template instructions.

Practice sites:

- ● F-027 — Input compiled as template source

Affected components: C-01

Remediation:

- **Structural** — compile fixed templates and pass application values only through their data-binding interface
- **Tactical** — ◕ M-039 — Remove server-side evaluation of untrusted input (F-027)

W-019 — Stored username interpolated into system prompt (chat.ts: 83)

● **Medium** · implementation weakness · observed-practice

Practice sites:

- `routes/chat.ts:83`

Affected components: C-01

W-020 — Client-side role guards trust unverified JWT claims (app.guard.ts:54)

● **Medium** · implementation weakness · observed-practice · 1 finding

Practice sites:

- ● F-089 — Client-side role guards trust unverified JWT claims

Affected components: C-05

Remediation:

- **Tactical** — ◑ M-092 — Enforce authorization on the server (F-089)

W-021 — OAuth sign-in requests are not bound to the user's browser session

● **Medium** · implementation weakness · observed-practice · 1 finding

The client starts OAuth or OpenID Connect sign-in without a per-request state value that its callback verifies. A forged authorization response can then complete sign-in in the victim's browser with the attacker's grant.

Practice sites:

- ● F-042 — OAuth implicit flow without state or nonce

Affected components: C-05

Remediation:

- **Structural** — start every sign-in through a maintained OAuth/OIDC client that generates and verifies state (and PKCE) for each authorization request
- **Tactical** — ◑ M-051 — Use authorization code flow with PKCE and state

W-022 — Cookie-authenticated changes lack request authenticity checks

● **Medium** · implementation weakness · observed-practice · 2 findings

A state-changing operation accepts ambient browser credentials without an enforced check that the authenticated user initiated the request. The affected mutation path lacks a request-authenticity boundary.

Practice sites:

- ● F-042 — OAuth implicit flow without state or nonce
- ● F-049 — Cookie-authenticated profile update

Affected components: C-01, C-05

Remediation:

- **Structural** — enforce framework CSRF protection or validated origin and anti-CSRF tokens on every mutation that accepts ambient browser credentials
- **Tactical** — ◑ M-051 — Use authorization code flow with PKCE and state, ◑ M-058 — Add CSRF token and SameSite cookie flags

W-023 — Privileged attributes lack an enforced write allowlist

● **Medium** · implementation weakness · observed-practice · 1 finding

Model binding accepts caller-selected attributes across a privileged write boundary. The affected binding path does not separate fields callers may change from fields reserved for server-side policy.

Practice sites:

- ● F-038 — User registration accepts privileged fields

Affected components: C-01

Remediation:

- **Structural** — define operation-specific write schemas and assign privileged attributes exclusively through server-side policy
- **Tactical** — ◕ M-047 — Allowlist client-controlled fields

W-024 — Unaudited npm Lifecycle Script

● **Medium** · implementation weakness · observed-practice · 1 finding

Practice sites:

- ● F-057 — Unaudited npm Lifecycle Script

Affected components: C-03

Remediation:

- **Tactical** — ◑ M-010 — Manual review: verify Unaudited npm Lifecycle Script, ◑ M-019 — No postinstall scripts

W-025 — Product description stored without sanitization (product.ts: 45)

● **Medium** · implementation weakness · observed-practice · 1 finding

Practice sites:

- ● F-052 — Product description stored without sanitization

Affected components: C-02

Remediation:

- **Tactical** — ◑ M-061 — Always sanitize product description

W-026 — User email stored without sanitization (user.ts:60)

● **Medium** · implementation weakness · observed-practice · 1 finding

Practice sites:

- ● F-053 — User email stored without sanitization

Affected components: C-02

Remediation:

- **Tactical** — ◑ M-062 — Sanitize email and username unconditionally

W-027 — Sensitive cookies remain accessible to browser scripts

● **Medium** · implementation weakness · observed-practice · 1 finding

Sensitive cookies are issued without HttpOnly protection. Browser scripts in the same origin can read values that should remain confined to the browser's HTTP cookie handling.

Practice sites:

- ● F-076 — Session token cookie without security attributes

Affected components: C-09

Remediation:

- **Structural** — issue sensitive cookies server-side with HttpOnly protection and keep script-readable application preferences separate from session credentials
- **Tactical** — ◑ M-081 — Set secure session-cookie attributes

W-028 — JSONP and arbitrary field selection on current user endpoint (currentUser.ts:58)

● **Medium** · implementation weakness · observed-practice · 1 finding

Practice sites:

- ● F-063 — JSONP and arbitrary field selection on current user

Affected components: C-01

Remediation:

- **Tactical** — ◑ M-068 — Remove JSONP and allowlist returned user fields

W-029 — Unauthenticated memories listing joins user records (memory.ts:24)

● **Medium** · implementation weakness · observed-practice · 1 finding

Practice sites:

- ● F-064 — Unauthenticated memories listing joins user records

Affected components: C-01

Remediation:

- **Tactical** — ◑ M-013 — Manual review: verify Unauthenticated memories listing joins user records, ◑ M-069 — Return only whitelisted user fields from memories

W-030 — Full user record with password hash in JWT (login.ts:24)

● **Medium** · implementation weakness · observed-practice · 1 finding

Practice sites:

- ● F-075 — Full user record with password hash in JWT

Affected components: C-09

Remediation:

- **Tactical** — ◑ M-080 — Stop exposing internal information to clients (F-075)

W-031 — Full application configuration served unauthenticated (appConfiguration.ts:15)

● **Medium** · implementation weakness · observed-practice

Practice sites:

- `routes/appConfiguration.ts:15`

Affected components: C-01

W-032 — Confidential offer embedded in system prompt (chat.ts:105)

● **Medium** · implementation weakness · observed-practice

Practice sites:

- `routes/chat.ts:105`

Affected components: C-01

W-033 — Upstream LLM error text streamed to client (chat.ts:255)

● **Medium** · implementation weakness · observed-practice

Practice sites:

- `routes/chat.ts:255`

Affected components: C-01

W-034 — Payment card numbers stored in plaintext (card.ts:38)

● **Medium** · implementation weakness · observed-practice · 1 finding

Practice sites:

- ● F-071 — Payment card numbers stored in plaintext

Affected components: C-02

Remediation:

- **Tactical** — ◑ M-076 — Encrypt or tokenize card numbers

W-035 — Unencrypted SQLite database file storage (index.ts:41)

● **Medium** · implementation weakness · observed-practice

Practice sites:

- `models/index.ts:41`

Affected components: C-08

W-036 — TOTP secrets stored in plaintext (user.ts:112)

● **Medium** · implementation weakness · observed-practice · 1 finding

Practice sites:

- ● F-072 — TOTP secrets stored in plaintext

Affected components: C-02

Remediation:

- **Tactical** — ◑ M-077 — Stop storing sensitive data in cleartext

W-037 — Unauthenticated encryption key directory exposure (keyServer.ts:14)

● **Medium** · implementation weakness · observed-practice · 1 finding

Practice sites:

- ● F-068 — Unauthenticated encryption key directory exposure

Affected components: C-01

Remediation:

- **Tactical** — ◑ M-073 — Remove public serving of the encryptionkeys directory

W-038 — Passwords sent in GET query string unencoded (user.service.ts:54)

● **Medium** · implementation weakness · observed-practice · 1 finding

Practice sites:

- ● F-074 — Passwords sent in GET query string unencoded

Affected components: C-05

Remediation:

- **Tactical** — ◑ M-079 — Send passwords in a POST body

W-039 — OAuth access token placed in query string (user.service.ts:68)

● **Medium** · implementation weakness · observed-practice

Practice sites:

- `frontend/src/app/Services/user.service.ts:68`

Affected components: C-05

W-040 — Substring redirect allowlist check (insecurity.ts:136)

● **Medium** · implementation weakness · observed-practice · 1 finding

Practice sites:

- ● F-046 — Substring redirect allowlist check

Affected components: C-09

Remediation:

- **Tactical** — ◑ M-055 — Validate redirect targets against an allowlist

W-041 — Security-sensitive data uses weak cryptographic primitives

● **Medium** · implementation weakness · observed-practice · 1 finding

Password, token, or integrity protection uses a weak hash, predictable random source, or insufficient work factor. The application may use a standard library, but the selected primitive does not provide the required security property.

Practice sites:

- ● F-032 — Passwords hashed with unsalted fast MD5

Affected components: C-02

Remediation:

- **Structural** — standardise on a password KDF, a CSPRNG for secrets, and modern authenticated cryptographic primitives with centrally reviewed parameters
- **Tactical** — ◕ M-043 — Hash passwords with a strong, salted algorithm

---

8. Findings Register

Findings are grouped by severity (Critical → High → Medium → Low); within a tier they are ordered by attack vektor (Repo-Read → Internet-Anon → Internet-User → Victim-Required). Every finding card shows **Severity · Component · Location**, **Issue**, **Fix**, and **Classification** (with external CWE / OWASP links). **Evidence**, **Root cause**, violated requirements, the weakness, and the trust boundary gap appear when they apply.

Risk Distribution: ● Critical: 4 · ● High: 25 · ● Medium: 53 · ● Low: n/a · **Total findings: 82** *This register counts every finding card below. The Management Summary's risk distribution (Total: 41) counts folded insecure-practice sites under their weakness and adds each design risk once.* **STRIDE Coverage:** Spoofing: 13 · Tampering: 27 · Repudiation: 0 · Information Disclosure: 21 · Denial of Service: 7 · Elevation of Privilege: 14

The root causes and the fixes to do first are summarized in **Remediation Priorities** in the Management Summary; evidence-backed weaknesses are documented in the Weakness Register.

Findings index:

● F-001 — Missing Object Ownership Checks
● F-007 — SQL injection
● F-009 — Broken authorization
● F-013 — SQL injection in login query
● F-002 — XSS via Trusted HTML Binding
● F-003 — Angular Sanitizer Bypass XSS
● F-004 — Session Token Stored in localStorage
● F-005 — Hardcoded Secrets in Source
● F-006 — Systemic IDOR Missing Ownership Checks
● F-015 — Chat identity taken from unverified JWT
● F-016 — JWT verify call missing algorithms allowlist
● F-017 — Password change without current password

- F-018 — Predictable password derived from OAuth email address
- F-021 — Password reset by security answer only
- F-022 — Path Traversal
- F-023 — Zip Slip path traversal during archive extraction
- F-025 — Input in executable NoSQL predicate
- F-026 — Input in executable NoSQL predicate
- F-027 — Input compiled as template source
- F-028 — DOM XSS
- F-030 — Unauthenticated log directory listing
- F-031 — XML external entity expansion in upload parser
- F-032 — Passwords hashed with unsalted fast MD5
- F-033 — Code injection
- F-034 — Sensitive Routes Registered Without Authentication Middleware
- F-036 — Missing authorization on product update route
- F-037 — Sensitive Routes Registered Without Authentication Middleware
- F-038 — User registration accepts privileged fields
- F-113 — Session token stored in web-accessible client storage
- F-039 — Unauthenticated review creation
- F-040 — Hard-coded cookie signing secret
- F-041 — Client-supplied review author identity
- F-042 — OAuth implicit flow without state or nonce
- F-043 — Unauthenticated WebSocket Channel
- F-044 — Weak 2FA code guessing bounds
- F-045 — No brute-force protection on login
- F-046 — Substring redirect allowlist check
- F-047 — Wallet top-up accepts unvalidated balance amount
- F-048 — Request data evaluated by notevil in vm sandbox
- F-049 — Cookie-authenticated profile update
- F-050 — Client-supplied coupon data accepted without server-side
- F-051 — Unvalidated client chat history passed to model
- F-052 — Product description stored without sanitization
- F-053 — User email stored without sanitization
- F-054 — Missing Container Image Signing
- F-055 — Remote install script piped to shell in deploy job
- F-056 — Dependency Install Scripts Run at Build Time
- F-057 — Unaudited npm Lifecycle Script
- F-058 — Unpinned Third-Party GitHub Action
- F-059 — Non-reproducible dependency installs in release builds
- F-060 — Unpinned floating container base images
- F-061 — Unsanitised login IP header stored for later
- F-062 — Null byte extension allowlist bypass in FTP file
- F-063 — JSONP and arbitrary field selection on current user
- F-064 — Unauthenticated memories listing joins user records
- F-065 — Security question disclosure
- F-066 — Unauthenticated metrics endpoint
- F-067 — Unauthenticated log directory listing and download
- F-068 — Unauthenticated encryption key directory exposure

- F-069 — Order history lookup by vowel-masked email
- F-070 — Server-side request forgery
- F-071 — Payment card numbers stored in plaintext
- F-072 — TOTP secrets stored in plaintext
- F-073 — Security answers HMAC with hardcoded key
- F-074 — Passwords sent in GET query string unencoded
- F-075 — Full user record with password hash in JWT
- F-076 — Session token cookie without security attributes
- F-077 — Blocking busy-wait sleep reachable from review
- F-078 — Unbounded zip extraction output size
- F-079 — Unbounded unauthenticated chat consumption
- F-080 — Disk upload without size limit
- F-081 — Unbounded disk upload processed before authentication
- F-082 — Unbounded wallet set growth without validation
- F-083 — Unhandled TypeError from non-string socket
- F-084 — Basket ID check bypass
- F-085 — Deluxe upgrade without payment for unrecognized payment
- F-086 — Chatbot coupon discount unbounded server-side
- F-087 — Container Runs as Root
- F-088 — Missing Workflow Permissions Block
- F-089 — Client-side role guards trust unverified JWT claims
- F-090 — Any authenticated user can list all accounts
- F-091 — Sessions cannot be revoked

● Critical (4)

F-001 · Missing Object Ownership Checks

Severity: ● Critical · **Component:** C-01 - Juice Shop Express Server · **Location:** Multiple locations (7)

Weakness: W-002 - Authorization is implemented route by route

Instances (7): `routes/basketItems.ts` : 68; `routes/order.ts` : 148, 150; `routes/orderHistory.ts` : 36; `routes/payment.ts` : 21, 41, 70

Issue: A signed-in attacker puts another user's ID into the owner field of the request (`req.body.UserId` and the like); the query trusts it instead of the session identity, so the attacker reads or changes every other user's records - horizontal privilege escalation across all accounts.

Evidence: ◌ ambiguous

```
routes/order.ts
  145 |
  146 |           if (req.body.UserId) {
  147 |             if (req.body.orderDetails && req.body.orderDetails.paymentId === 'wallet') {
→ 148 |               const wallet = await WalletModel.findOne({ where: { UserId: req.body.UserId } })
```

```
          149 |               if ((wallet != null) && wallet.balance >= totalPrice) {
          150 |                 await WalletModel.decrement({ balance: totalPrice }, { where: { UserId:
req.body.UserId } })
          151 |               } else {
```

Fix: Tie every object lookup to the requesting user's identity and reject cross-tenant references → ◑ M-001 — Manual review: verify Missing Object Ownership Checks · ● M-023 — Enforce object-level (ownership) authorization (● F-001)

Classification: Broken Access Control · STRIDE: Tampering · CWE-639 · OWASP A01:2025 · walkthrough Walkthrough §3.3

F-007 · SQL injection

Severity: ● Critical · **Component:** C-01 - Juice Shop Express Server · **Location:** `routes/search.ts:23`

Weakness: W-001 - Database access relies on concatenated queries

Issue: Interpolating request-controlled text into a SQL statement lets an attacker alter the query - exfiltrating or modifying arbitrary rows, bypassing authentication, or escalating to full database control.

Evidence: ✓ verified

```
          20 |   return (req: Request, res: Response, next: NextFunction) => {
          21 |     let criteria: any = req.query.q === 'undefined' ? '' : req.query.q ?? ''
          22 |     criteria = (criteria.length <= 200) ? criteria : criteria.substring(0, 200)
        → 23 |     models.sequelize.query(`SELECT * FROM Products WHERE ((name LIKE '%${criteria}%' OR
description LIKE '%${criteria}%') AND deletedAt IS NULL) ORDER BY name`) // vuln-code-snippet vuln-line
unionSqlInjectionChallenge dbSchemaChallenge
          24 |       .then(([products]: any) => {
          25 |         const dataString = JSON.stringify(products)
          26 |         if (challengeUtils.notSolved(challenges.unionSqlInjectionChallenge)) { // vuln-code-
snippet hide-start
```

Fix: Switch all SQL execution to parameterised queries or ORM-bound parameters → ● M-029 — Use parameterized database queries (● F-007)

Classification: Injection · STRIDE: Tampering · CWE-89 · OWASP A05:2025 · walkthrough Walkthrough §3.1

F-009 · Broken authorization

Severity: ● Critical · **Component:** C-01 - Juice Shop Express Server · **Location:** `routes/dataExport.ts:26`

Weakness: W-002 - Authorization is implemented route by route

Issue: A signed-in attacker puts another user's ID into the owner field of the request (`req.body.UserId` and the like); the query trusts it instead of the session identity, so the attacker reads or changes every other user's records - horizontal privilege escalation across all accounts.

Evidence: ✓ verified

```
  23 |
  24 |         let memories, orders, reviews
  25 |         try {
→ 26 |           memories = await MemoryModel.findAll({ where: { UserId: req.body.UserId } })
  27 |         } catch (error) {
  28 |           next(error)
  29 |           return
```

Fix: Tie every object lookup to the requesting user's identity and reject cross-tenant references → ● M-023 — Enforce object-level (ownership) authorization (● F-001)

Classification: Broken Access Control · STRIDE: Tampering · CWE-639 · OWASP A01:2025 · walkthrough Walkthrough §3.4

F-013 · SQL injection in login query

Severity: ● Critical · **Component:** C-09 - Authentication & Session Surface · **Location:** `routes/login.ts:34`

Weakness: W-001 - Database access relies on concatenated queries

Trust boundary gap: tb-2 🌐 External - external → Authentication & Session Surface · Validation: The finding's evidence line is the interface this boundary cites; CWE-89 breaks its validation condition.

Issue: An unauthenticated attacker posts an email such as admin' -- to `/rest/user/login`. The value is concatenated into raw SQL, so the password check is commented out and a valid admin token is issued.

Authentication bypass for any account including admin, and database read access through injected queries.

Evidence: ✓ verified - `req.body.email` is interpolated into a raw `sequelize.query` string on the unauthenticated login route.

```
  31 |
  32 |   return (req: Request, res: Response, next: NextFunction) => {
  33 |     verifyPreLoginChallenges(req) // vuln-code-snippet hide-line
→ 34 |     models.sequelize.query(`SELECT * FROM Users WHERE email = '${req.body.email || ''}' AND
password = '${security.hash(req.body.password || '')}' AND deletedAt IS NULL`, { model: UserModel,
plain: true }) // vuln-code-snippet vuln-line loginAdminChallenge loginBenderChallenge loginJimChallenge
  35 |       .then((authenticatedUser) => { // vuln-code-snippet neutral-line loginAdminChallenge
loginBenderChallenge loginJimChallenge
```

```
          36 |         const user = utils.queryResultToJson(authenticatedUser)
          37 |         if (user.data?.id && user.data.totpSecret !== '') {
```

Fix: Switch all SQL execution to parameterised queries or ORM-bound parameters → ● M-030 — Parameterize the login query with bind replacements

Classification: Injection · STRIDE: Tampering · CWE-89 · OWASP A05:2025 · walkthrough Walkthrough §3.2

● High (25)

F-005 · Hardcoded Secrets in Source

Severity: ● High - secret committed to the public source repo - extractable on clone, no prior access needed; elevated to Critical as a verified attack-chain keystone in AC-T-005 (Authentication Bypass via Exposed Secret Material); see §9 *(raw Critical)* · **Component:** C-09 - Authentication & Session Surface · **Location:** Multiple locations (3)

Weakness: W-007 - Secrets are committed to source instead of a managed store

Instances (3): `routes/checkKeys.ts`: 10; `lib/insecurity.ts`: 21; `routes/login.ts`: 60

Issue: The `RS256` signing key is a literal in source and the security-answer HMAC key is a literal at line 42. Anyone with the source can sign tokens for any user or role, or compute answer hashes offline. Complete forgery of sessions and roles.

Evidence: ✓ verified - `privateKey` is a string literal used by authorize and `deluxeToken`; exploitation needs source access.

```
        lib/insecurity.ts
          19 |
          20 | export const publicKey = fs ? fs.readFileSync('encryptionkeys/jwt.pub', 'utf8') : 'placeholder-
public-key'
        → 21 | const privateKey = '[PEM PRIVATE KEY — REDACTED] …
          22 |
          23 | interface ResponseWithUser {
```

Fix: Move the credential out of source control into a secret store and rotate it → ◕ M-027 — Load signing keys from a secret store

Classification: Cryptographic Failures · STRIDE: Information Disclosure · CWE-798 · OWASP A04:2025 · walkthrough Walkthrough §3.6

F-015 · Chat identity taken from unverified JWT

Severity: ● High - verified attack-chain contributor in AC-T-005 (Authentication Bypass via Exposed Secret Material); see §9 · **Component:** C-01 - Juice Shop Express Server · **Location:** `routes/chat.ts:45`

Weakness: W-008 - Authentication relies on hand-rolled token verification

Issue: ASI03: POST `/rest/chat` has no auth middleware and `getUserId` uses `security.decode`, which does not verify the signature. An attacker forges a token with another user's `data.id`; `getOrderById` then fetches that user's email and returns their order, and the username enters the system prompt.

Another customer's order data can be read through the chatbot; decode implementation was inferred from the supplied flow analysis.

Evidence: ✓ verified - `security.decode` result is used as the caller identity for order lookup without signature verification and without `isAuthorized` on the route.

```
  43 |   const token = utils.jwtFrom(req)
  44 |   if (!token) return undefined
→ 45 |   const decoded = security.decode(token) as { data?: { id?: number } } | undefined
  46 |   return decoded?.data?.id
  47 | }
```

Fix: Pin the signature algorithm explicitly and reject `alg:none` and unknown algorithms → ◕ M-031 — Verify JWT signature

Classification: Broken Authentication · STRIDE: Spoofing · CWE-347 · OWASP A07:2025

F-016 · JWT verify call missing algorithms allowlist

Severity: ● High *(raw Critical)* · **Component:** C-01 - Juice Shop Express Server · **Location:** Multiple locations (3)

Weakness: W-008 - Authentication relies on hand-rolled token verification

Instances (3): `lib/insecurity.ts`: 55, 189; `routes/verify.ts`: 120

Issue: Without an explicit algorithm allowlist, attackers can forge tokens with `alg:none` (older lib versions) or use the public key as an HMAC secret to mint valid signatures.

Evidence: ◌ ambiguous

```
routes/verify.ts
  118 |     }
  119 |
→ 120 |     jwt.verify(token, security.publicKey, (err: jwt.VerifyErrors | null) => {
  121 |       if (err === null) {
  122 |         challengeUtils.solveIf(challenge, () => {
```

Fix: Pin the signature algorithm explicitly and reject `alg:none` and unknown algorithms → ◑ M-003 — Manual review: verify JWT verify call missing algorithms allowlist · ◕ M-032 — Enforce JWT signature and algorithm verification (● F-016)

Classification: Broken Authentication · STRIDE: Spoofing · CWE-347 · OWASP A07:2025

F-017 · Password change without current password

Severity: ● High · **Component:** C-01 - Juice Shop Express Server · **Location:** `routes/changePassword.ts:39`

Issue: The current password is only checked when supplied (`currentPassword` && ...). Anyone holding a session token, e.g. a stolen or leaked one, omits the 'current' parameter and sets a new password, taking over the account permanently.

Passwords also travel in the query string. A stolen session token escalates to permanent account takeover.

Evidence: ✓ verified - Verification of the current password is skipped when the parameter is absent.

```
  37 |   }
  38 |
→ 39 |   if (currentPassword && security.hash(currentPassword) !== loggedInUser.data.password) {
  40 |     res.status(401).send(res.__('Current password is not correct.'))
  41 |     return
```

Fix: ◕ M-033 — Require current password for password change

Classification: Broken Authentication · STRIDE: Spoofing · CWE-620 · OWASP A07:2025

F-018 · Predictable password derived from OAuth email address

Severity: ● High · **Component:** C-05 - Angular Frontend · **Location:** `frontend/src/app/oauth/oauth.component.ts:30`

Issue: After Google sign-in, the client sets the account password to btoa of the reversed email, then registers and logs in with it. An attacker who knows a victim's email computes the same password and logs in through the normal login endpoint.

This takes over accounts created through OAuth. An attacker can authenticate as any OAuth-registered user and reach their orders, addresses and wallet.

Evidence: ✓ verified - Line 30 builds the password deterministically from the email. Line 46 reuses it for login. Server acceptance was not read.

```
  28 |   this.userService.oauthLogin(this.parseRedirectUrlParams().access_token).subscribe({
  29 |     next: (profile: any) => {
→ 30 |       const password = btoa(profile.email.split('').reverse().join(''))
  31 |       this.userService.save({ email: profile.email, password, passwordRepeat: password }).subscr
ibe({
  32 |         next: () => {
```

Fix: ◕ M-034 — Stop deriving OAuth account passwords from the email

Classification: OAuth / OIDC Misconfiguration · STRIDE: Spoofing · CWE-1391 · OWASP A07:2025

F-021 · Password reset by security answer only

Severity: ● High - verified attack-chain keystone in AC-B-003 (User takes over another account); see §9 · **Component:** C-09 - Authentication & Session Surface · **Location:** Multiple locations (2)

Instances (2): `routes/resetPassword.ts`: 41, 10

Issue: An unauthenticated attacker posts a victim's email, a guessed security answer and a new password to `/rest/user/reset-password`. A correct answer resets the password with no out-of-band check, no rate limit and no notice to the owner.

Full account takeover through guessable or researchable answers, with unlimited guesses.

Evidence: ✓ verified - The only proof is `hmac(answer)` equal to the stored answer, then `user.update` sets the new password; no attempt limit is applied.

```
routes/resetPassword.ts
  39 |         }]
  40 |       })
→ 41 |       if ((data != null) && security.hmac(answer) === data.answer) {
  42 |         const user = await UserModel.findByPk(data.UserId)
  43 |         if (user) {
```

Fix: ◕ M-035 — Require emailed one-time token and rate-limit reset

Classification: Broken Authentication · STRIDE: Spoofing · CWE-640 · OWASP A07:2025

F-022 · Path Traversal

Severity: ● High · **Component:** C-01 - Juice Shop Express Server · **Location:** Multiple locations (2)

Instances (2): `routes/dataErasure.ts`: 107, 104

Issue: A request-controlled path with `../` can read arbitrary files (`/etc/passwd`, source, secrets) or write outside the intended root.

Evidence: ✓ verified

```
routes/dataErasure.ts
  102 |
  103 |       if (req.body.layout) {
→ 104 |         const filePath: string = path.resolve(req.body.layout).toLowerCase()
  105 |         const isForbiddenFile: boolean = (filePath.includes('ftp') ||
```

```
filePath.includes('ctf.key') || filePath.includes('encryptionkeys'))
          106 |         if (!isForbiddenFile) {
```

Fix: Resolve and normalise every constructed path and reject anything that escapes the intended base directory → ◕ M-036 — Constrain file paths to a safe base directory (● F-022)

Classification: Insecure File Handling · STRIDE: Tampering · CWE-22 · OWASP A06:2025

F-023 · Zip Slip path traversal during archive extraction

Severity: ● High · **Component:** C-01 - Juice Shop Express Server · **Location:** `routes/fileUpload.ts:34`

Issue: An unauthenticated attacker uploads a .zip to POST `/file-upload` whose entry names contain ../ segments. The containment check only tests that the resolved path contains the working directory, so traversal inside the app root passes and the entry is written outside uploads/complaints, overwriting files such as `ftp/legal.md` or application assets.

Arbitrary file overwrite inside the application directory, enabling content tampering and potentially code replacement.

Evidence: ✓ verified - `createWriteStream` uses the raw entry name; the `includes(path.resolve('.'))` check at line 33 accepts any path under the cwd.

```
          32 |     challengeUtils.solveIf(challenges.fileWriteChallenge, () => { return absolutePath === path.res
olve('ftp/legal.md') })
          33 |     if (absolutePath.includes(path.resolve('.'))) {
        → 34 |       await pipeline(entry.stream(), fs.createWriteStream('uploads/complaints/' + fileName))

          35 |     }
          36 |   }
```

Fix: Resolve and normalise every constructed path and reject anything that escapes the intended base directory → ◕ M-037 — Constrain zip entries to the extraction directory

Classification: Insecure File Handling · STRIDE: Tampering · CWE-22 · OWASP A06:2025

F-025 · Input in executable NoSQL predicate

Severity: ● High · **Component:** C-01 - Juice Shop Express Server · **Location:** `routes/showProductReviews.ts:36`

Weakness: W-004 - Document-database queries take their structure from application values

Issue: Request input read reaches an executable expression. Observed path conditions: const id = ! `utils.isChallengeEnabled(challenges.noSqlCommandChallenge)` ? The unsafe path is observed; attacker control, access prerequisites, and active configuration require verification.

Evidence: ◌ ambiguous

```
   34 |     const t0 = new Date().getTime()
   35 |
 → 36 |     db.reviewsCollection.find({ $where: 'this.product == ' + id }).then((reviews: Review[]) => {
   37 |       const t1 = new Date().getTime()
   38 |       challengeUtils.solveIf(challenges.noSqlCommandChallenge, () => { return (t1 - t0) > 2000 })
```

Fix: Replace string concatenation in query operators with parameter binding → ◑ M-004 — Manual review: verify Input in executable NoSQL predicate · ◕ M-038 — Use typed query predicates instead of $where

Classification: Injection · STRIDE: Tampering · CWE-943 · OWASP A05:2025

F-026 · Input in executable NoSQL predicate

Severity: ● High · **Component:** C-01 - Juice Shop Express Server · **Location:** `routes/trackOrder.ts:18`

Weakness: W-004 - Document-database queries take their structure from application values

Issue: Request input read reaches an executable expression. Observed path conditions: const id = ! `utils.isChallengeEnabled(challenges.reflectedXssChallenge)` ? The unsafe path is observed; attacker control, access prerequisites, and active configuration require verification.

Evidence: ✓ verified

```
   16 |
   17 |     challengeUtils.solveIf(challenges.reflectedXssChallenge, () => { return utils.contains(id, '<iframe src="javascript:alert(`xss`)">') })
 → 18 |     db.ordersCollection.find({ $where: `this.orderId === '${id}'` }).then((order: any) => {
   19 |       const result = utils.queryResultToJson(order)
   20 |       challengeUtils.solveIf(challenges.noSqlOrdersChallenge, () => { return result.data.length > 1 })
```

Fix: Replace string concatenation in query operators with parameter binding → ◕ M-038 — Use typed query predicates instead of $where

Classification: Injection · STRIDE: Tampering · CWE-943 · OWASP A05:2025

F-030 · Unauthenticated log directory listing

Severity: ● High · **Component:** C-01 - Juice Shop Express Server · **Location:** `server.ts:281`

Issue: `/support/logs` is served with `serveIndex` and file serving, with no authentication middleware. An anonymous attacker can list and download access logs, which contain URLs, IPs and query strings. Disclosure of request logs aids reconnaissance and may expose user data.

Evidence: ✓ verified - The `serveIndex('logs')` and `serveLogFiles` routes are registered without `isAuthorized`.

```
          279 |
          280 |   /* /logs directory browsing */ // vuln-code-snippet neutral-line accessLogDisclosureChallenge

        → 281 |   app.use('/support/logs', serveIndexMiddleware, serveIndex('logs', { icons: true, view: 'details
' })) // vuln-code-snippet vuln-line accessLogDisclosureChallenge
          282 |   app.use('/support/logs', verify.accessControlChallenges()) // vuln-code-snippet hide-line

          283 |   app.use('/support/logs/:file', serveLogFiles()) // vuln-code-snippet vuln-line
accessLogDisclosureChallenge
```

Fix: ◕ M-041 — Disable public directory listings

Classification: Unauthenticated Management Plane · STRIDE: Information Disclosure · CWE-548 · OWASP A01:2025

F-031 · XML external entity expansion in upload parser

Severity: ● High · **Component:** C-01 - Juice Shop Express Server · **Location:** `routes/fileUpload.ts:76`

Issue: An attacker uploads an .xml file to POST `/file-upload` containing a DOCTYPE with an external entity pointing at `file:///etc/passwd`. The parser is configured with entity substitution and DTD loading and filesystem providers, and the parsed document is echoed back in the error message.

Local file disclosure and entity-expansion resource exhaustion from an unauthenticated endpoint.

Evidence: ✓ verified - `parseXmlString` enables NOENT and DTDLOAD with FS providers and the result is returned in the line 79 error.

```
          74 |       const data = file.buffer.toString()
          75 |       try {
        → 76 |         const xmlString = await parseXmlString(data)
          77 |         challengeUtils.solveIf(challenges.xxeFileDisclosureChallenge, () => { return (utils.matche
sEtcPasswdFile(xmlString) || utils.matchesSystemIniFile(xmlString)) })
          78 |         res.status(410)
```

Fix: Disable external entity resolution on every XML parser and reject DOCTYPE declarations → ◕ M-042 — Disable XML external entity (XXE) resolution

Classification: Insecure File Handling · STRIDE: Information Disclosure · CWE-611 · OWASP A06:2025

F-032 · Passwords hashed with unsalted fast MD5

Severity: ● High · **Component:** C-02 - Sequelize application data access · **Location:** Multiple locations (2)

Weakness: W-041 - Security-sensitive data uses weak cryptographic primitives

Instances (2): `models/user.ts` : 76; `lib/insecurity.ts` : 41

Issue: The User password setter stores `security.hash(clearTextPassword)`, which is an unsalted `MD5` digest. An attacker who obtains the SQLite file or a query result can crack the hashes quickly with rainbow tables or GPU brute force and recover plaintext passwords.

Cracked passwords give account takeover, including admin accounts, and enable credential reuse elsewhere.

Evidence: ✓ verified - The setter calls `security.hash`, which `lib/insecurity.ts:41` implements as an unsalted `MD5` digest; database read access is a precondition.

Fix: Replace the broken hash with a salted password-hashing function (bcrypt/Argon2id) → ◕ M-043 — Hash passwords with a strong, salted algorithm

Classification: Cryptographic Failures · STRIDE: Information Disclosure · CWE-916 · OWASP A04:2025

F-034 · Sensitive Routes Registered Without Authentication Middleware

Severity: ● High · **Component:** C-01 - Juice Shop Express Server · **Location:** Multiple locations (4)

Weakness: W-002 - Authorization is implemented route by route

Instances (4): `server.ts` : 309, 607, 676, 729

Issue: The state-changing / management route `POST /file-upload` (`server.ts:309`) has no authentication or authorization check in its handler body, so it is reachable by an unauthenticated caller.

Evidence: ✓ verified

```
        server.ts
          307 |   app.use(bodyParser.urlencoded({ extended: true }))
          308 |   /* File Upload */
        → 309 |   app.post('/file-upload', uploadToMemory.single('file'), ensureFileIsPassed, metrics.observeFile
UploadMetricsMiddleware(), checkUploadSize, checkFileType, handleZipFileUpload, handleXmlUpload,
handleYamlUpload)
          310 |   app.post('/profile/image/file', uploadToMemory.single('file'), ensureFileIsPassed, metrics.obse
rveFileUploadMetricsMiddleware(), utils.asyncHandler(profileImageFileUpload()))
          311 |   app.post('/profile/image/url', uploadToMemory.single('file'), utils.asyncHandler(profileImageUr
lUpload()))
```

Fix: *no mitigation mapped - see §10*

Classification: Broken Access Control · STRIDE: Elevation of Privilege · CWE-862 · OWASP A01:2025

F-036 · Missing authorization on product update route

Severity: ● High · **Component:** C-01 - Juice Shop Express Server · **Location:** `server.ts:370`

Weakness: W-002 - Authorization is implemented route by route

Issue: The `isAuthorized` guard for PUT `/api/Products/:id` is commented out, and finale generates a PUT endpoint for the Product model. An unauthenticated attacker can send PUT `/api/Products/1` with a modified body to change product names, prices and descriptions.

Anyone can alter catalog data, such as prices, which affects shop integrity.

Evidence: ✓ verified - Line 370 is commented out, while the Product resource is registered with finale at line 485 with no other guard.

```
          368 |   /* Products: Only GET is allowed in order to view products
*/ // vuln-code-snippet neutral-line changeProductChallenge
          369 |   app.post('/api/Products', security.isAuthorized()) // vuln-code-snippet neutral-line
changeProductChallenge
        → 370 |   // app.put('/api/Products/:id', security.isAuthorized()) // vuln-code-snippet vuln-line
changeProductChallenge
          371 |   app.delete('/api/Products/:id', security.denyAll())
          372 |   /* Challenges: GET list of challenges allowed. Everything else forbidden entirely */
```

Fix: ◕ M-045 — Enforce server-side authorization on every endpoint (● F-036)

Classification: Broken Access Control · STRIDE: Elevation of Privilege · CWE-862 · OWASP A01:2025

F-037 · Sensitive Routes Registered Without Authentication Middleware

Severity: ● High · **Component:** C-01 - Juice Shop Express Server · **Location:** Multiple locations (12)

Weakness: W-002 - Authorization is implemented route by route

Instances (12): `server.ts` : 598, 641, 667, 606, 612, 613, 614, 638 … (+4 more)

Issue: The route registration carries no authentication marker, so an anonymous attacker may call this state-changing operation on a sensitive resource directly, unless a downstream check the scanner cannot see stops it.

Evidence: ◌ ambiguous

```
server.ts
  596 |   app.post('/rest/user/login', login())
  597 |   app.get('/rest/user/change-password', utils.asyncHandler(changePassword()))
→ 598 |   app.post('/rest/user/reset-password', utils.asyncHandler(resetPassword()))
  599 |   app.get('/rest/user/security-question', utils.asyncHandler(securityQuestion()))
  600 |   app.get('/rest/user/whoami', utils.asyncHandler(retrieveLoggedInUser()))
```

Fix: ◑ M-006 — Manual review: verify Sensitive Routes Registered Without Authentication Middleware · ◕ M-046 — Enforce server-side authorization on every endpoint (● F-037)

Classification: Broken Access Control · STRIDE: Elevation of Privilege · CWE-862 · OWASP A01:2025

F-038 · User registration accepts privileged fields

Severity: ● High - reaches a privileged operation on an unauthenticated endpoint; elevated to Critical as a verified attack-chain keystone in AC-T-002 (Bulk Data Exfiltration via Broken Object Authorization), AC-T-004 (Privilege Escalation via Mass-Assignment on Registration); see §9 · **Component:** C-01 - Juice Shop Express Server · **Location:** Multiple locations (2)

Weakness: W-023 - Privileged attributes lack an enforced write allowlist

Instances (2): `server.ts` : 503, 408

Issue: POST `/api/Users` is passed to the generated finale User resource after only trimming; the body is not filtered, so an attacker supplies a role such as admin at registration. Admin account creation.

Evidence: ✓ verified - Middleware at lines 408-419 trims values and does not strip role; model constraints were not inspected.

```
server.ts
  501 |     const resource = finale.resource({
  502 |       model,
→ 503 |       endpoints: [`/api/${name}s`, `/api/${name}s/:id`],
  504 |       excludeAttributes: exclude,
  505 |       pagination: false,
```

Fix: ◕ M-047 — Allowlist client-controlled fields

Classification: Broken Access Control · STRIDE: Elevation of Privilege · CWE-915 · OWASP A01:2025 · walkthrough Walkthrough §3.5

F-004 · Session Token Stored in localStorage

Severity: ● High · **Component:** C-05 - Angular Frontend · **Location:** Multiple locations (7)

Weakness: W-005 - Browser scripts retain reusable session credentials

Instances (7): `frontend/src/app/Services/request.interceptor.ts` : 16, 13; `frontend/src/app/app.guard.ts` : 35; `frontend/src/app/login/login.component.ts` : 101; `frontend/src/app/oauth/oauth.component.ts` : 51; `frontend/src/app/payment/payment.component.ts` : 232; `frontend/src/app/two-factor-auth-enter/two-factor-auth-enter.component.ts` : 53

Issue: The interceptor reads the JWT from `localStorage` and attaches it to requests, so the token lives in script-readable storage. Any XSS in the page origin can read it and replay it to impersonate the user.

A script injected into the origin can steal the session JWT and take over the account, including admin accounts.

Evidence: ✓ verified - `Authorization` header is built from `localStorage.getItem('token')`, proving the JWT is stored in script-accessible storage.

Fix: ◕ M-026 — Move session token to HttpOnly cookie

Classification: Insecure Client-Side Storage · STRIDE: Information Disclosure · CWE-922 · OWASP A04:2025

F-006 · Systemic IDOR Missing Ownership Checks

Severity: ● High - verified attack-chain keystone in AC-T-002 (Bulk Data Exfiltration via Broken Object Authorization); see §9 · **Component:** C-01 - Juice Shop Express Server · **Location:** Multiple locations (9)

Weakness: W-002 - Authorization is implemented route by route

Instances (9): `routes/basket.ts` : 19; `server.ts` : 450, 426, 603, 604; `routes/coupon.ts` : 18; `routes/order.ts` : 35; `routes/updateProductReviews.ts` : 18 … (+1 more)

Issue: `retrieveBasket` loads the basket by `req.params.id` with no comparison to the authenticated user's basket; any logged-in user reads other baskets by changing the id. Cross-user basket disclosure.

Evidence: ✓ verified - `findOne` where {id} only; the check at line 23 only flags a challenge.

```
routes/basket.ts
  17 |     try {
  18 |       const id = req.params.id
→ 19 |       const basket = await BasketModel.findOne({ where: { id }, include: [{ model: ProductModel,
paranoid: false, as: 'Products' }] })
  20 |       /* jshint eqeqeq:false */
  21 |       challengeUtils.solveIf(challenges.basketAccessChallenge, () => {
```

Fix: Tie every object lookup to the requesting user's identity and reject cross-tenant references → ◕ M-028 — Check basket ownership before returning

Classification: Broken Access Control · STRIDE: Elevation of Privilege · CWE-639 · OWASP A01:2025

F-027 · Input compiled as template source

Severity: ● High · **Component:** C-01 - Juice Shop Express Server · **Location:** `routes/userProfile.ts:87`

Weakness: W-018 - Application data crosses into template source

Issue: Persisted record read reaches an executable expression. Observed path conditions: if (username) {. The unsafe path is observed; attacker control, access prerequisites, and active configuration require verification.

Evidence: ✓ verified

```
  85 |     try {
  86 |       const pug = (await import('pug')).default
→ 87 |       const fn = pug.compile(template)
  88 |       const CSP = `img-src 'self' ${user?.profileImage}; script-src 'self' 'unsafe-eval'`
  89 |
```

Fix: ◕ M-039 — Remove server-side evaluation of untrusted input (● F-027)

Classification: Injection · STRIDE: Tampering · CWE-1336 · OWASP A05:2025

F-033 · Code injection

Severity: ● High · **Component:** C-01 - Juice Shop Express Server · **Location:** `routes/userProfile.ts:61`

Weakness: W-003 - Application data crosses into executable code

Issue: POST `/profile` stores `req.body.username` unvalidated. GET `/profile` then evals any username of the form #{...} when the username XSS challenge is enabled, giving server-side JavaScript execution for the logged-in user; the username is also substituted into a Pug template before compilation.

Remote code execution in the Node process.

Evidence: ✓ verified - `eval(code)` runs a substring of the user-controlled username; a feature flag gating the branch is not a control.

```
  59 |         throw new Error('Username is null')
  60 |       }
→ 61 |       username = eval(code) // eslint-disable-line no-eval
  62 |     } catch (err) {
  63 |       username = '\\' + username
```

Fix: Replace runtime code generation (eval/Function/template render) with a data-only execution path → ◕ M-044 — Remove eval of username in profile rendering

Classification: Code Execution via Unsafe Deserialization or Eval · STRIDE: Elevation of Privilege · CWE-95 · OWASP A08:2025

F-113 · Session token stored in web-accessible client storage

Severity: ● High · **Component:** C-05 - Angular Frontend · **Location:** `frontend/src/app/Services/basket.service.ts:64`

Issue: Token exfiltrated from local/session storage via the Step 1 payload.

Evidence: ✓ verified

Fix: ◕ M-095 — Store session tokens in HttpOnly cookies

Classification: Insecure Client-Side Storage · STRIDE: Information Disclosure · CWE-922 · OWASP A04:2025

F-002 · XSS via Trusted HTML Binding

Severity: ● High · **Component:** C-05 - Angular Frontend · **Location:** Multiple locations (4)

Weakness: W-006 - Frontend rendering lacks enforced output encoding

Instances (4): `frontend/src/app/about/about.component.ts`: 119; `frontend/src/app/administration/administration.component.ts`: 73, 91; `frontend/src/app/search-result/search-result.component.ts`: 143

Issue: An attacker submits feedback whose comment contains script-bearing HTML. The about page concatenates the comment into markup, marks it trusted, and renders it through `innerHTML` for every visitor, so the script runs in the app origin and can read the JWT in `localStorage`.

Script execution in other users' sessions allows session token theft and actions as the victim.

Evidence: ✓ verified - Feedback comment is templated into HTML and passed to the Angular sanitizer bypass before being bound to `innerHTML`.

```
        frontend/src/app/about/about.component.ts
          117 |             feedbacks[i].comment
          118 |           }</p><div class="feedback-stars">(${this.stars[feedbacks[i].rating]})</div></
figcaption>`
        → 119 |           feedbacks[i].comment = this.sanitizer.bypassSecurityTrustHtml(
          120 |             feedbacks[i].comment
          121 |           )
```

Fix: Output-encode untrusted strings at every sink and remove all `bypassSecurityTrustHtml` calls → ◕ M-024 — Remove sanitizer bypass and render feedback as text

Classification: Cross-Site Scripting (XSS) · STRIDE: Tampering · CWE-79 · OWASP A05:2025

F-003 · Angular Sanitizer Bypass XSS

Severity: ● High · **Component:** C-05 - Angular Frontend · **Location:** Multiple locations (5)

Weakness: W-006 - Frontend rendering lacks enforced output encoding

Instances (5): `frontend/src/app/last-login-ip/last-login-ip.component.ts`: 39; `frontend/src/app/data-export/data-export.component.ts`: 57; `frontend/src/app/search-result/search-result.component.ts`: 110; `frontend/src/app/score-board/score-board.component.ts`: 82; `frontend/src/app/track-result/track-result.component.ts`: 48

Issue: The last-login-IP page decodes the JWT from `localStorage` and passes `payload.data.lastLoginIp` through the sanitizer bypass into an `innerHTML` binding. If an attacker can influence the stored `lastLoginIp` value, for example via the login request origin data, script runs in the victim's origin and can steal the token.

Script execution in the application origin allows theft of the `localStorage` JWT and actions as the victim. The source read does not show how the server populates `lastLoginIp`.

Evidence: ◌ ambiguous - Claim value is interpolated into HTML and marked trusted, then bound with [`innerHTML`] in the template at line 10.

```
frontend/src/app/last-login-ip/last-login-ip.component.ts
  37 |       if (payload.data.lastLoginIp) {
  38 |
→ 39 |         this.lastLoginIp = this.sanitizer.bypassSecurityTrustHtml(`<small>${payload.data.lastLogin
Ip}</small>`)
  40 |       }
  41 |     }
```

Fix: Output-encode untrusted strings at every sink and remove all `bypassSecurityTrustHtml` calls → ◑ M-002 — Manual review: verify Angular Sanitizer Bypass XSS · ◕ M-025 — Remove sanitizer bypass and render IP as text

Classification: Cross-Site Scripting (XSS) · STRIDE: Tampering · CWE-79 · OWASP A05:2025

F-028 · DOM XSS

Severity: ● High · **Component:** C-05 - Angular Frontend · **Location:** `frontend/src/app/data-export/data-export.component.ts:71`

Weakness: W-006 - Frontend rendering lacks enforced output encoding

Issue: The data export response field `userData` is written with `document.write` into a same-origin about:blank window. If an attacker stores markup in a field included in the victim's export (for example a username, review or feedback), the script runs in the app origin and can read the JWT held in `localStorage`.

Script execution in the application origin can steal the `localStorage` JWT and act as the victim.

Evidence: ◌ ambiguous - Line 71 passes the server-provided `userData` string directly to `document.write` with no escaping.

```
     69 |         this.confirmation = data.confirmation
     70 |         this.userData = data.userData
   → 71 |         window.open('', '_blank', 'width=500')?.document.write(this.userData)
     72 |         this.lastSuccessfulTry = new Date()
     73 |         localStorage.setItem('lstdtxprt', JSON.stringify(this.lastSuccessfulTry))
```

Fix: Output-encode untrusted strings at every sink and remove all `bypassSecurityTrustHtml` calls → ◑ M-005 — Manual review: verify DOM XSS via document.write of exported user data · ◕ M-040 — Encode output instead of bypassing the framework sanitizer (● F-028)

Classification: Cross-Site Scripting (XSS) · STRIDE: Tampering · CWE-79 · OWASP A05:2025

● Medium (53)

F-040 · Hard-coded cookie signing secret

Severity: ● Medium · **Component:** C-01 - Juice Shop Express Server · **Location:** `server.ts:289`

Weakness: W-007 - Secrets are committed to source instead of a managed store

Issue: `cookieParser` is initialised with the literal secret 'kekse'. An attacker who reads the public source can forge signed cookies, which can enable cookie-based identity spoofing wherever signed cookies are trusted.

Signed cookies can be forged. The consuming security decision was not shown in the slices.

Evidence: ◌ ambiguous - Line 289 passes a literal string as the cookie signing secret.

Fix: Move the credential out of source control into a secret store and rotate it → ◑ M-007 — Manual review: verify Hard-coded cookie signing secret · ◑ M-049 — Load the cookie secret from environment config

Classification: Cryptographic Failures · STRIDE: Spoofing · CWE-798 · OWASP A04:2025

F-072 · TOTP secrets stored in plaintext

Severity: ● Medium · **Component:** C-02 - Sequelize application data access · **Location:** Multiple locations (2)

Weakness: W-036 - TOTP secrets stored in plaintext (`user.ts:112`)

Instances (2): `models/user.ts` : 112; `routes/2fa.ts` : 129

Issue: The User model stores `totpSecret` as a plain string with no encryption. An attacker with database read access can generate valid second-factor codes and bypass 2FA for those users. The second authentication factor is defeated once the database is read.

Evidence: ✓ verified - `totpSecret` is a plain `DataTypes.STRING` column with no setter or encryption.

Fix: ◑ M-077 — Stop storing sensitive data in cleartext

Classification: Cryptographic Failures · STRIDE: Information Disclosure · CWE-312 · OWASP A04:2025

F-073 · Security answers HMAC with hardcoded key

Severity: ● Medium · **Component:** C-02 - Sequelize application data access · **Location:** `models/securityAnswer.ts:45`

Weakness: W-007 - Secrets are committed to source instead of a managed store

Issue: The SecurityAnswer setter hashes answers with `security.hmac`, which uses a static key embedded in source (`lib/insecurity.ts:42`). Anyone with the source and a copy of the database can brute-force the low-entropy answers offline.

Recovered answers allow password reset and account takeover for affected users.

Evidence: ✓ verified - The setter delegates to `security.hmac`, whose key is a string literal in source; the key is not secret and there is no per-record salt or work factor.

Fix: Move the cryptographic key out of source control into a managed secret store and rotate it → ◑ M-078 — Move cryptographic keys to a managed secret store

Classification: Cryptographic Failures · STRIDE: Information Disclosure · CWE-321 · OWASP A04:2025

F-039 · Unauthenticated review creation

Severity: ● Medium · **Component:** C-01 - Juice Shop Express Server · **Location:** `routes/createProductReviews.ts:26`

Issue: PUT `/rest/products/:id/reviews` has no `isAuthorized` guard and stores `req.body.author` as given, so an attacker posts reviews attributed to any user. Forged reviews impersonate other users and damage trust in review integrity.

Evidence: ✓ verified - The author is taken from `req.body.author`; the authenticated user is only compared for a challenge, not enforced.

Fix: ◑ M-048 — Derive review author from verified token identity

Classification: Broken Authentication · STRIDE: Spoofing · CWE-290 · OWASP A07:2025

F-043 · Unauthenticated WebSocket Channel

Severity: ● Medium - elevated to High because verified finding evidence ties the weakness to confirmed internet ingress tb-1 · **Component:** C-07 - Real-time WebSocket Channel · **Location:** Multiple locations (2)

Weakness: W-009 - Endpoints are reachable without enforced authentication

Trust boundary gap: tb-1 🌐 External - external → Real-time WebSocket Channel · Authentication: The finding's evidence line is the interface this boundary cites; CWE-306 breaks its authentication condition.
⬆ *Confirmed internet ingress at tb-1 raised this finding: Medium → High.*

Instances (2): `lib/startup/registerWebsocketEvents.ts`: 23, 36

Issue: An attacker on the internet opens a Socket.IO connection from a non-browser client, which ignores the CORS origin setting. The server registers no `io.use` middleware, so it accepts the client as a full peer. The attacker can emit any handler event and receive pushed events.

Evidence: ✓ verified

Fix: ◑ M-052 — Require authentication on every exposed endpoint (● F-043)

Classification: Unauthenticated Management Plane · STRIDE: Spoofing · CWE-306 · OWASP A01:2025

F-044 · Weak 2FA code guessing bounds

Severity: ● Medium · **Component:** C-09 - Authentication & Session Surface · **Location:** `routes/2fa.ts:31`

Weakness: W-011 - Weak 2FA code guessing bounds (`2fa.ts:31`)

Issue: An attacker who knows a password holds a `tmpToken` valid for 6 hours and can guess TOTP codes at `/rest/2fa/verify`. The limit is 100 per 5 minutes per client and `epochTolerance` 30 widens the accepted window, so guessing is feasible over time or across IPs.

Second factor can be bypassed by an attacker with the password.

Evidence: ✓ verified - Verify accepts TOTP with `epochTolerance` 30 and no per-user lockout; only an IP rate limit at `server.ts` line 459.

Fix: Apply rate limiting and lock-out thresholds on authentication endpoints → ◑ M-053 — Add per-user lockout and shrink the TOTP window

Classification: Broken Authentication · STRIDE: Spoofing · CWE-307 · OWASP A07:2025

F-045 · No brute-force protection on login

Severity: ● Medium · **Component:** C-09 - Authentication & Session Surface · **Location:** `routes/login.ts:32`

Weakness: W-012 - No brute-force protection on login (`login.ts:32`)

Issue: An attacker repeatedly posts credentials to `/rest/user/login`. The handler has no lockout or throttling, and the route registration carries no rate limiter, so guessing and credential stuffing are unrestricted. Account takeover of users with weak passwords.

Evidence: ◌ ambiguous - The handler and its registration at `server.ts` line 596 apply no attempt limit, and passwords are fast `MD5` hashes.

Fix: Apply rate limiting and lock-out thresholds on authentication endpoints → ◑ M-009 — Manual review: verify No brute-force protection on login · ◑ M-054 — Add per-account and per-IP throttling to login

Classification: Broken Authentication · STRIDE: Spoofing · CWE-307 · OWASP A07:2025

F-047 · Wallet top-up accepts unvalidated balance amount

Severity: ● Medium - verified attack-chain keystone in AC-B-005 (User exceeds a limit or a one-time entitlement); see §9 · **Component:** C-01 - Juice Shop Express Server · **Location:** `routes/wallet.ts:27`

Issue: PUT `/rest/wallet/balance` increments the wallet by `req.body.balance` after only checking that a card id belongs to the user. The amount is not validated and no charge is made, so a user can credit arbitrary or negative amounts without payment. Free store credit and wallet balance manipulation.

Evidence: ✓ verified - `WalletModel.increment` uses the raw body value with no type, range or payment verification.

Fix: ◑ M-056 — Validate wallet top-up amount and charge card

Classification: Broken Access Control · STRIDE: Tampering · CWE-20 · OWASP A01:2025

F-048 · Request data evaluated by notevil in vm sandbox

Severity: ● Medium - elevated to Critical as a verified attack-chain keystone in AC-T-006 (Remote Code Execution via Server-Side Injection); see §9 · **Component:** C-01 - Juice Shop Express Server · **Location:** `routes/b2bOrder.ts:23`

Weakness: W-003 - Application data crosses into executable code

Issue: The B2B order body field `orderLinesData` is evaluated with notevil inside a vm context. notevil/vm are not security boundaries; an authenticated caller can attempt sandbox escape or loop abuse to run code or occupy the process. Potential server-side code execution or process stalls.

Evidence: ✓ verified - User-controlled expression reaches an eval implementation; exploitability of a sandbox escape was not demonstrated.

Fix: Replace runtime code generation (eval/Function/template render) with a data-only execution path → ◑ M-057 — Remove dynamic evaluation of order lines

Classification: Code Execution via Unsafe Deserialization or Eval · STRIDE: Tampering · CWE-95 · OWASP A08:2025

F-050 · Client-supplied coupon data accepted without server-side

Severity: ● Medium · **Component:** C-01 - Juice Shop Express Server · **Location:** `routes/order.ts:196`

Issue: The checkout accepts a base64 `couponData` field from the request body and matches it against a hard-coded expired campaign by the client-supplied date. Any user can forge the string and obtain discounts of up to 75 percent without a server-issued coupon.

Evidence: ✓ verified

Fix: ◑ M-059 — Validate coupons server-side against current time

Classification: Broken Access Control · STRIDE: Tampering · CWE-602 · OWASP A01:2025

F-051 · Unvalidated client chat history passed to model

Severity: ● Medium · **Component:** C-01 - Juice Shop Express Server · **Location:** `routes/chat.ts:206`

Issue: LLM01/ASI01: messages from `req.body` are passed straight to `streamText` with no role or content validation. An unauthenticated caller injects system, assistant or tool messages, fabricating history, to override the coupon policy or alter tool behaviour.

Model behaviour and tool calls become attacker-steerable.

Evidence: ✓ verified - `req.body.messages` (line 191) is used verbatim as model messages.

Fix: ◑ M-060 — Restrict chat messages to validated user turns

Classification: Injection · STRIDE: Tampering · CWE-1427 · OWASP A05:2025

F-062 · Null byte extension allowlist bypass in FTP file

Severity: ● Medium · **Component:** C-01 - Juice Shop Express Server · **Location:** `routes/fileServer.ts:28`

Issue: `servePublicFiles` checks the .md/.pdf extension before `cutOffPoisonNullByte` truncates the name, so a name like `file.bak%2500.md` passes the check and `sendFile` serves the truncated non-allowlisted file from the ftp folder. Backup and sensitive files in ftp are downloadable by unauthenticated users.

Evidence: ✓ verified - Null byte stripping occurs after the allowlist check and before `res.sendFile`.

Fix: ◑ M-067 — Reject null bytes before validating file extension

Classification: Insecure File Handling · STRIDE: Information Disclosure · CWE-158 · OWASP A06:2025

F-066 · Unauthenticated metrics endpoint

Severity: ● Medium · **Component:** C-01 - Juice Shop Express Server · **Location:** `server.ts:676`

Weakness: W-009 - Endpoints are reachable without enforced authentication

Issue: GET `/metrics` is registered with no authentication, exposing internal Prometheus metrics to any network caller. Operational reconnaissance data exposed.

Evidence: ✓ verified - `app.get('/metrics', ...)` has no `isAuthorized` or network restriction in the inspected code.

Fix: ◑ M-071 — Restrict metrics endpoint access

Classification: Unauthenticated Management Plane · STRIDE: Information Disclosure · CWE-306 · OWASP A01:2025

F-067 · Unauthenticated log directory listing and download

Severity: ● Medium · **Component:** C-01 - Juice Shop Express Server · **Location:** `server.ts:281`

Issue: An anonymous client browses `/support/logs` and downloads access logs, which reveal request URLs, client IPs and any tokens or identifiers in query strings. Exposure of operational and user request data.

Evidence: ✓ verified - `serveIndex('logs')` and `serveLogFiles` are registered with no authentication middleware.

Fix: Strip secrets and PII from every log sink and rotate any token that already leaked → ◑ M-072 — Require authentication or remove public log serving

Classification: Missing Audit Logging & Accountability · STRIDE: Information Disclosure · CWE-532 · OWASP A09:2025

F-068 · Unauthenticated encryption key directory exposure

Severity: ● Medium · **Component:** C-01 - Juice Shop Express Server · **Location:** `routes/keyServer.ts:14`

Weakness: W-037 - Unauthenticated encryption key directory exposure (`keyServer.ts:14`)

Issue: Any anonymous client can list `/encryptionkeys` and fetch files from it. Key material stored there becomes readable. Disclosure of cryptographic key files; actual contents not inspected.

Evidence: ✓ verified - `sendFile` serves any slash-free name from encryptionkeys/ with `serveIndex` listing at `server.ts:277` and no auth.

Fix: ◑ M-073 — Remove public serving of the encryptionkeys directory

Classification: Unauthenticated Management Plane · STRIDE: Information Disclosure · CWE-538 · OWASP A01:2025

F-071 · Payment card numbers stored in plaintext

Severity: ● Medium · **Component:** C-02 - Sequelize application data access · **Location:** `models/card.ts:38`

Weakness: W-034 - Payment card numbers stored in plaintext (`card.ts:38`)

Issue: The Card model persists `cardNum` and expiry as plain integer columns with no encryption or tokenization. Anyone reading the SQLite file or any query result exposing the model obtains full card numbers. Disclosure of full card numbers enables payment fraud against users.

Evidence: ✓ verified - `cardNum` is a plain `DataTypes.INTEGER` column with only range validation and no encryption setter or getter.

Fix: Encrypt the data in transit and at rest with vetted primitives → ◑ M-076 — Encrypt or tokenize card numbers

Classification: Cryptographic Failures · STRIDE: Information Disclosure · CWE-311 · OWASP A04:2025

F-075 · Full user record with password hash in JWT

Severity: ● Medium · **Component:** C-09 - Authentication & Session Surface · **Location:** `routes/login.ts:24`

Weakness: W-030 - Full user record with password hash in JWT (`login.ts:24`)

Issue: Login signs the whole user object, including the `MD5` password hash and `totpSecret`, into the JWT. JWT payloads are only base64, so anyone holding or intercepting a token reads them. Hash and 2FA secret exposure via any token leak.

Evidence: ✓ verified - authorize receives `authenticatedUser` whose data is the full User model; `2fa.ts` line 42 does the same.

Fix: Restrict the response to the minimum fields needed and never echo secrets → ◑ M-080 — Stop exposing internal information to clients (● F-075)

Classification: Error Information Disclosure · STRIDE: Information Disclosure · CWE-200 · OWASP A02:2025

F-076 · Session token cookie without security attributes

Severity: ● Medium · **Component:** C-09 - Authentication & Session Surface · **Location:** `lib/insecurity.ts:192`

Weakness: W-027 - Sensitive cookies remain accessible to browser scripts

Issue: `res.cookie` sets the token with no `httpOnly`, secure or `sameSite` flags, so any script running in the page can read the JWT and cross-site requests carry it. XSS or network observers can steal the session token.

Evidence: ✓ verified - The cookie call passes no options object.

Fix: Set `HttpOnly` on every session cookie → ◑ M-081 — Set secure session-cookie attributes

Classification: Insecure Client-Side Storage · STRIDE: Information Disclosure · CWE-1004 · OWASP A04:2025

F-077 · Blocking busy-wait sleep reachable from review

Severity: ● Medium · **Component:** C-01 - Juice Shop Express Server · **Location:** `routes/showProductReviews.ts:23`

Weakness: W-013 - Blocking busy-wait sleep reachable from review query (`showProductReviews.ts:23`)

Issue: A global `sleep()` busy-loops the Node event loop for up to 2000 ms and is callable from the $where expression on the unauthenticated reviews route, so repeated requests stall the server. Event-loop starvation and service unavailability.

Evidence: ◌ ambiguous - Synchronous while loop capped at 2 s per call; reachability depends on the flag-dependent injection at line 36.

Fix: Bound the request rate and the per-request resource budget on this endpoint → ◑ M-014 — Manual review: verify Blocking busy-wait sleep reachable from review query · ◑ M-082 — Rate-limit expensive requests and bound input size

Classification: Denial of Service · STRIDE: Denial of Service · CWE-400 · OWASP A06:2025

F-078 · Unbounded zip extraction output size

Severity: ● Medium · **Component:** C-01 - Juice Shop Express Server · **Location:** `routes/fileUpload.ts:34`

Issue: An attacker uploads a small highly compressed zip with many or very large entries. Extraction streams each entry to disk with no cap on entry count, per-entry or total decompressed size, filling the disk. Disk exhaustion degrading availability of the shop.

Evidence: ✓ verified - Only the compressed upload is limited to 200KB; decompressed output is not bounded.

Fix: ◑ M-083 — Cap entry count and decompressed size during extraction

Classification: Insecure File Handling · STRIDE: Denial of Service · CWE-409 · OWASP A06:2025

F-079 · Unbounded unauthenticated chat consumption

Severity: ● Medium · **Component:** C-01 - Juice Shop Express Server · **Location:** `routes/chat.ts:203`

Weakness: W-014 - Unbounded unauthenticated chat consumption (`chat.ts:203`)

Issue: LLM10: `/rest/chat` is unauthenticated with no rate limit, no cap on message count/size and no output token cap; each request can run up to 10 tool steps and retries against a paid provider. Provider cost and availability exhaustion.

Evidence: ✓ verified - `streamText` has only `stepCountIs(10)` and `maxRetries`; no `maxOutputTokens` or input bounds.

Fix: Bound the request rate and the per-request resource budget on this endpoint → ◑ M-084 — Add rate limit and token caps to chat

Classification: Denial of Service · STRIDE: Denial of Service · CWE-770 · OWASP A06:2025

F-080 · Disk upload without size limit

Severity: ● Medium · **Component:** C-01 - Juice Shop Express Server · **Location:** `server.ts:698`

Weakness: W-015 - Disk upload without size limit (`server.ts:698`)

Issue: The multer disk storage used for uploads sets no `fileSize` limit, so a caller can upload very large files and fill the disk. Disk space exhaustion on the server host.

Evidence: ✓ verified - `multer({storage: diskStorage})` lacks limits, unlike `uploadToMemory`.

Fix: Bound the request rate and the per-request resource budget on this endpoint → ◑ M-085 — Set fileSize limit on disk uploads

Classification: Insecure File Handling · STRIDE: Denial of Service · CWE-770 · OWASP A06:2025

F-081 · Unbounded disk upload processed before authentication

Severity: ● Medium · **Component:** C-01 - Juice Shop Express Server · **Location:** `server.ts:698`

Weakness: W-016 - Unbounded disk upload processed before authentication (`server.ts:698`)

Issue: The memories upload uses disk storage with no file size limit, and multer runs before `appendUserId` rejects unauthenticated callers. An anonymous attacker repeatedly posts large png or jpeg files, filling the disk.

Evidence: ◌ ambiguous

Fix: Bound the request rate and the per-request resource budget on this endpoint → ◑ M-015 — Manual review: verify Unbounded disk upload processed before authentication · ◑ M-086 — Authenticate first and cap disk upload size

Classification: Denial of Service · STRIDE: Denial of Service · CWE-770 · OWASP A06:2025

F-082 · Unbounded wallet set growth without validation

Severity: ● Medium · **Component:** C-04 - Web3 / Wallet / NFT Surface · **Location:** `routes/web3Wallet.ts:16`

Issue: An attacker repeatedly POSTs distinct arbitrary `walletAddress` values to `walletExploitAddress` ; each is added to an in-memory Set with no type, length or count limit, exhausting process memory. Memory exhaustion can crash the shared application process, denying service to all users.

Evidence: ✓ verified - Line 16 adds the raw request value to a module-level Set; the route is registered without auth or rate limiting at `server.ts:645` .

Fix: Bound the request rate and the per-request resource budget on this endpoint → ◑ M-087 — Validate addresses and cap tracked wallet set

Classification: Denial of Service · STRIDE: Denial of Service · CWE-770 · OWASP A06:2025

F-083 · Unhandled TypeError from non-string socket

Severity: ● Medium · **Component:** C-07 - Real-time WebSocket Channel · **Location:** `lib/startup/registerWebsocketEvents.ts:46`

Issue: An attacker emits 'verifySvgInjectionChallenge' with a number, so data?.match is not a function and throws inside the event handler. The same applies to `utils.contains` with a truthy non-string.

With no try/catch, the exception can become uncaught and terminate the Node process while the challenge is unsolved.

Evidence: ◌ ambiguous

Fix: ◑ M-016 — Manual review: verify Unhandled TypeError from non-string socket payloads · ◑ M-088 — Validate payload types in socket handlers

Classification: Denial of Service · STRIDE: Denial of Service · CWE-248 · OWASP A06:2025

F-085 · Deluxe upgrade without payment for unrecognized payment

Severity: ● Medium · **Component:** C-01 - Juice Shop Express Server · **Location:** `routes/deluxe.ts:43`

Issue: POST `/rest/deluxe-membership` only charges when `paymentMode` is wallet or card; any other value skips payment and still promotes the customer to the deluxe role and issues a new token. The card branch also never charges. Free privileged membership and loss of revenue.

Evidence: ✓ verified - `user.update` to the deluxe role runs unconditionally after the optional payment branches.

Fix: ◑ M-090 — Require validated payment mode before deluxe upgrade

Classification: Broken Access Control · STRIDE: Elevation of Privilege · CWE-840 · OWASP A01:2025

F-086 · Chatbot coupon discount unbounded server-side

Severity: ● Medium · **Component:** C-01 - Juice Shop Express Server · **Location:** `routes/chat.ts:184`

Weakness: W-002 - Authorization is implemented route by route

Issue: LLM06/ASI02: `generateCoupon` accepts any number and issues a coupon; the 10% cap and eligibility exist only in the prompt. An unauthenticated user steers the model, via forged history or injection, to issue a 100% coupon. Financial loss via arbitrary discounts.

Evidence: ✓ verified - `z.number()` has no max and execute calls `generateCoupon` without checks or auth.

Fix: ◑ M-091 — Enforce coupon limits in server code

Classification: Broken Access Control · STRIDE: Elevation of Privilege · CWE-862 · OWASP A01:2025

F-089 · Client-side role guards trust unverified JWT claims

Severity: ● Medium · **Component:** C-05 - Angular Frontend · **Location:** `frontend/src/app/app.guard.ts:54`

Weakness: W-020 - Client-side role guards trust unverified JWT claims (`app.guard.ts:54`)

Issue: AdminGuard and AccountingGuard decode the `localStorage` JWT with `jwtDecode`, which does not verify the signature, and grant routes based on `data.role`. An attacker can write a forged token into `localStorage` to open admin or accounting views.

Attackers can reach admin and accounting UI and any client-exposed data if the server does not independently enforce roles.

Evidence: ✓ verified - The guard grants access from a role claim in a token that `jwtDecode` decoded without verifying the signature.

Fix: ◑ M-092 — Enforce authorization on the server (● F-089)

Classification: Broken Access Control · STRIDE: Elevation of Privilege · CWE-602 · OWASP A01:2025

F-091 · Sessions cannot be revoked

Severity: ● Medium · **Component:** C-09 - Authentication & Session Surface · **Location:** `lib/insecurity.ts:74`

Issue: Tokens are stored in a process map and never removed; there is no logout or revocation, so a stolen token works until it expires after 6 hours and stays in memory. Compromised sessions persist and the map grows unbounded.

Evidence: ✓ verified - put adds tokens to `tokenMap` and no code in this component deletes them.

Fix: ◑ M-094 — Add token revocation and expiry cleanup

Classification: Broken Authentication · STRIDE: Elevation of Privilege · CWE-613 · OWASP A07:2025

F-041 · Client-supplied review author identity

Severity: ● Medium · **Component:** C-05 - Angular Frontend · **Location:** `frontend/src/app/product-details/product-details.component.ts:73`

Issue: The review dialog sets the author from a client-side `whoAmI` result and posts it in the request body. An attacker can edit the request to submit reviews under another user's email.

Attackers could forge reviews attributed to other users, damaging trust and enabling impersonation in the product review feature.

Evidence: ◌ ambiguous - The review object includes author: `this.author`, a client-held value, sent to `productReviewService.create`.

Fix: ◑ M-008 — Manual review: verify Client-supplied review author identity · ◑ M-050 — Derive review author from the server session

Classification: Broken Authentication · STRIDE: Spoofing · CWE-290 · OWASP A07:2025

F-052 · Product description stored without sanitization

Severity: ● Medium · **Component:** C-02 - Sequelize application data access · **Location:** `models/product.ts:45`

Weakness: W-025 - Product description stored without sanitization (`product.ts:45`)

Issue: When `restfulXssChallenge` is enabled the Product description setter performs no sanitization and stores attacker-supplied HTML, including iframe javascript URLs. If a client renders the description unescaped, script runs in other users' browsers.

Stored script injection can steal session tokens and act as other users.

Evidence: ✓ verified - The enabled branch only calls `solveIf` and assigns the raw description; `sanitizeSecure` runs only in the else branch. The rendering sink and write route were not in scope.

Fix: ◑ M-061 — Always sanitize product description

Classification: Cross-Site Scripting (XSS) · STRIDE: Tampering · CWE-116 · OWASP A05:2025

F-053 · User email stored without sanitization

Severity: ● Medium · **Component:** C-02 - Sequelize application data access · **Location:** `models/user.ts:60`

Weakness: W-026 - User email stored without sanitization (`user.ts:60`)

Issue: When `persistedXssUserChallenge` is enabled the User email setter stores the raw value without sanitization, and username uses the weaker `sanitizeLegacy`. An attacker registering with an HTML payload can plant script that executes where the value is rendered unescaped.

Stored script injection can run in admin or other users' browsers.

Evidence: ✓ verified - The enabled branch only calls `solveIf`; `sanitizeSecure` at line 68 runs only in the else branch. The rendering sink was not in scope.

Fix: ◑ M-062 — Sanitize email and username unconditionally

Classification: Cross-Site Scripting (XSS) · STRIDE: Tampering · CWE-116 · OWASP A05:2025

F-063 · JSONP and arbitrary field selection on current user

Severity: ● Medium · **Component:** C-01 - Juice Shop Express Server · **Location:** `routes/currentUser.ts:58`

Weakness: W-028 - JSONP and arbitrary field selection on current user endpoint (`currentUser.ts:58`)

Issue: GET `/rest/user/whoami` accepts a callback parameter returning JSONP and a fields parameter selecting arbitrary properties of the session user data, allowing cross-origin pages to read user data and requesting sensitive fields such as the password hash. Victim's email, IP and potentially password hash are exposed to attacker-controlled origins.

Evidence: ✓ verified - `res.jsonp(response)` bypasses same-origin protection and fields are copied from `user.data` without an allowlist.

Fix: Restrict the response to the minimum fields needed and never echo secrets → ◑ M-068 — Remove JSONP and allowlist returned user fields

Classification: Error Information Disclosure · STRIDE: Information Disclosure · CWE-200 · OWASP A02:2025

F-064 · Unauthenticated memories listing joins user records

Severity: ● Medium · **Component:** C-01 - Juice Shop Express Server · **Location:** `routes/memory.ts:24`

Weakness: W-029 - Unauthenticated memories listing joins user records (`memory.ts:24`)

Issue: GET `/rest/memories` has no authentication and returns each memory with its included User model, which may expose user emails and other account attributes to anonymous callers. Possible disclosure of user PII to anonymous users.

Evidence: ◌ ambiguous - `findAll` with include UserModel returned verbatim; model attribute exclusions were not inspected.

Fix: Restrict the response to the minimum fields needed and never echo secrets → ◑ M-013 — Manual review: verify Unauthenticated memories listing joins user records · ◑ M-069 — Return only whitelisted user fields from memories

Classification: Error Information Disclosure · STRIDE: Information Disclosure · CWE-200 · OWASP A02:2025

F-065 · Security question disclosure

Severity: ● Medium · **Component:** C-01 - Juice Shop Express Server · **Location:** `routes/securityQuestion.ts:15`

Issue: GET `/rest/user/security-question` returns the user's security question for any supplied email without authentication, confirming account existence and helping password-reset attacks. Account enumeration and recovery-question disclosure.

Evidence: ✓ verified - Different responses for existing and non-existing accounts, with no auth or rate limit shown on the route.

Fix: ◑ M-070 — Return uniform response for security question lookup

Classification: Error Information Disclosure · STRIDE: Information Disclosure · CWE-204 · OWASP A02:2025

F-069 · Order history lookup by vowel-masked email

Severity: ● Medium · **Component:** C-01 - Juice Shop Express Server · **Location:** `routes/orderHistory.ts:17`

Weakness: W-002 - Authorization is implemented route by route

Issue: Orders are stored and queried by the email with vowels replaced by asterisks. Two different accounts whose emails differ only in vowels map to the same key. An attacker registers such a lookalike email and reads the other user's order history, including addresses and totals.

Evidence: ✓ verified

Fix: Tie every object lookup to the requesting user's identity and reject cross-tenant references → ◑ M-074 — Enforce object-level (ownership) authorization (● F-069)

Classification: Broken Access Control · STRIDE: Information Disclosure · CWE-639 · OWASP A01:2025

F-070 · Server-side request forgery

Severity: ● Medium · **Component:** C-01 - Juice Shop Express Server · **Location:** `routes/profileImageUrlUpload.ts:24`

Trust boundary gap: tb-10 - Juice Shop Express Server → external · Egress destination: The finding's evidence line is the interface this boundary cites; CWE-918 breaks its egress-destination condition.

Issue: A logged-in user posts an arbitrary `imageUrl` that the server fetches with no scheme, host or private-address validation. The attacker can make the server reach internal services or cloud metadata.

The response body is written under the public uploads directory and is readable, and on failure the raw URL is stored.

Evidence: ✓ verified

Fix: Validate the URL scheme + host against an explicit allow-list before issuing outbound requests → ◑ M-075 — Validate and allowlist outbound request targets

Classification: Server-Side Request Forgery · STRIDE: Information Disclosure · CWE-918 · OWASP A01:2025

F-074 · Passwords sent in GET query string unencoded

Severity: ● Medium · **Component:** C-05 - Angular Frontend · **Location:** `frontend/src/app/Services/user.service.ts:54`

Weakness: W-038 - Passwords sent in GET query string unencoded (`user.service.ts:54`)

Issue: `changePassword` builds a GET URL containing the current, new and repeat passwords without URL encoding. They can land in server logs, proxies, browser history and `Referer` headers, and special characters can alter other parameters.

Credentials persisted in logs or history can be recovered by anyone with access to them, leading to account takeover.

Evidence: ✓ verified - The password values are concatenated directly into the request URL of an HTTP GET call.

Fix: ◑ M-079 — Send passwords in a POST body

Classification: Cryptographic Failures · STRIDE: Information Disclosure · CWE-598 · OWASP A04:2025

F-084 · Basket ID check bypass

Severity: ● Medium · **Component:** C-01 - Juice Shop Express Server · **Location:** `routes/basketItems.ts:42`

Issue: `addBasketItem` validates the first BasketId parsed from the raw body (`basketIds[0]`) but saves the last (`basketIds[length-1]`). An attacker sends two BasketId keys, first their own, last the victim's, adding items to another basket. Writing to arbitrary baskets.

Evidence: ✓ verified - Check at line 37 uses index 0, persisted value at line 42 uses last index.

Fix: ◑ M-089 — Validate the same BasketId that is saved

Classification: Broken Access Control · STRIDE: Elevation of Privilege · CWE-235 · OWASP A01:2025

F-090 · Any authenticated user can list all accounts

Severity: ● Medium · **Component:** C-09 - Authentication & Session Surface · **Location:** `routes/authenticatedUsers.ts:12`

Weakness: W-002 - Authorization is implemented route by route

Issue: `/rest/user/authentication-details` only requires a valid token. The handler returns every user with email, role, last login IP and last login time with no role check. Disclosure of the full user directory enabling targeted attacks.

Evidence: ✓ verified - `findAll` returns all users to any bearer of a valid JWT; `server.ts` line 398 only applies `isAuthorized`.

Fix: Add explicit server-side authorisation checks on every protected route → ◑ M-093 — Enforce server-side authorization

Classification: Broken Access Control · STRIDE: Elevation of Privilege · CWE-285 · OWASP A01:2025

F-042 · OAuth implicit flow without state or nonce

Severity: ● Medium · **Component:** C-05 - Angular Frontend · **Location:** `frontend/src/app/login/login.component.ts:148`

Weakness: W-021 - OAuth sign-in requests are not bound to the user's browser session

Issue: The Google login builds a `response_type=token` request with no state, nonce or PKCE, and the `redirect_uri` is not URL-encoded. An attacker can feed a victim a token or redirect of their own, causing login CSRF or session confusion.

Attacker-controlled OAuth responses may be accepted, binding the victim to an attacker-chosen identity, and access tokens appear in URL fragments.

Evidence: ✓ verified - The authorization URL contains only `client_id`, `response_type=token`, scope and `redirect_uri`; no state parameter is generated or verified.

Fix: Enforce a same-origin or signed CSRF token on every state-changing endpoint → ◑ M-051 — Use authorization code flow with PKCE and state

Classification: Cross-Site Request Forgery (CSRF) · STRIDE: Spoofing · CWE-352 · OWASP A01:2025

F-046 · Substring redirect allowlist check

Severity: ● Medium · **Component:** C-09 - Authentication & Session Surface · **Location:** `lib/insecurity.ts:136`

Weakness: W-040 - Substring redirect allowlist check (`insecurity.ts:136`)

Issue: `isRedirectAllowed` uses `url.includes(allowedUrl)`, so an attacker URL that merely contains an allowed address in its query string passes and can be used for phishing redirects. Users can be sent to attacker sites from a trusted domain link.

Evidence: ✓ verified - The allowlist match is a substring test; the `/redirect` handler that calls it was not read.

Fix: ◑ M-055 — Validate redirect targets against an allowlist

Classification: Open Redirect · STRIDE: Spoofing · CWE-601 · OWASP A01:2025

F-049 · Cookie-authenticated profile update

Severity: ● Medium · **Component:** C-01 - Juice Shop Express Server · **Location:** `routes/updateUserProfile.ts:38`

Weakness: W-022 - Cookie-authenticated changes lack request authenticity checks

Issue: POST `/profile` authenticates solely from the token cookie and updates the username with no CSRF token or origin check, so a malicious page can submit a cross-site form to change a victim's username (and, via eval, inject code). Unauthorized profile modification on behalf of victims.

Evidence: ✓ verified - State change keyed on cookie; origin/referer is read only for challenge detection, and the cookie set at line 42 has no SameSite or HttpOnly flag.

Fix: Enforce a same-origin or signed CSRF token on every state-changing endpoint → ◑ M-058 — Add CSRF token and SameSite cookie flags

Classification: Cross-Site Request Forgery (CSRF) · STRIDE: Tampering · CWE-352 · OWASP A01:2025

F-061 · Unsanitised login IP header stored for later

Severity: ● Medium · **Component:** C-09 - Authentication & Session Surface · **Location:** `routes/saveLoginIp.ts:32`

Weakness: W-006 - Frontend rendering lacks enforced output encoding

Issue: An authenticated user sends a True-Client-IP header carrying HTML. When the challenge flag is enabled the sanitising branch is skipped, and the raw value is saved as `lastLoginIp` and returned to the page that displays it.

Stored script injection in the last-login-IP view; the browser sink was not read, so exploitation is unproven.

Evidence: ◌ ambiguous - `sanitizeSecure` only runs in the else branch of `isChallengeEnabled`; otherwise the raw header is persisted.

Fix: Output-encode untrusted strings at every sink and remove all `bypassSecurityTrustHtml` calls → ◑ M-012 — Manual review: verify Unsanitised login IP header stored for later rendering · ◑ M-066 — Encode output instead of bypassing the framework sanitizer (● F-061)

Classification: Cross-Site Scripting (XSS) · STRIDE: Tampering · CWE-79 · OWASP A05:2025

F-054 · Missing Container Image Signing

Severity: ● Medium · **Component:** C-03 - CI/CD Pipeline · **Location:** `.github/workflows/ci.yml:1`

Weakness: W-008 - Authentication relies on hand-rolled token verification

Issue: Missing Container Image Signing: Unsigned container images cannot be verified for provenance - any registry intermediary could substitute them.

Evidence: ✓ verified

Fix: Pin the signature algorithm explicitly and reject `alg:none` and unknown algorithms → ◑ M-017 — Sign and verify release artifacts

Classification: Broken Authentication · STRIDE: Tampering · CWE-347 · OWASP A07:2025

F-055 · Remote install script piped to shell in deploy job

Severity: ● Medium · **Component:** C-03 - CI/CD Pipeline · **Location:** `.github/workflows/ci.yml:358`

Issue: The heroku job downloads `install.sh` from cli-assets.heroku.com and pipes it straight into sh before deploying with `HEROKU_API_KEY`. A compromised or MITM-substituted script runs on the runner in the deploy job.

Attacker code runs in the production deploy job, which holds the Heroku API key and can alter the deployed application.

Evidence: ✓ verified - Line 358 runs curl `https://cli-assets.heroku.com/install.sh` | sh with no checksum or signature verification.

Fix: ◑ M-063 — Verify or remove curl-pipe Heroku CLI install

Classification: Supply-Chain Integrity · STRIDE: Tampering · CWE-494 · OWASP A03:2025

F-056 · Dependency Install Scripts Run at Build Time

Severity: ● Medium · **Component:** C-03 - CI/CD Pipeline · **Location:** `Dockerfile:4`

Issue: Dependency Install Scripts Run at Build Time: Without `--ignore-scripts`, any compromised dependency publisher can execute postinstall code at build time.

Evidence: ✓ verified

Fix: ◑ M-018 — Disable untrusted package install scripts (● F-056)

Classification: Supply-Chain Integrity · STRIDE: Tampering · CWE-506 · OWASP A03:2025

F-057 · Unaudited npm Lifecycle Script

Severity: ● Medium · **Component:** C-03 - CI/CD Pipeline · **Location:** `package.json:56`

Weakness: W-024 - Unaudited npm Lifecycle Script

Issue: Unaudited npm Lifecycle Script: postinstall hooks execute arbitrary code at install time - the attack surface for supply-chain compromise.

Evidence: ◌ ambiguous

Fix: ◑ M-010 — Manual review: verify Unaudited npm Lifecycle Script · ◑ M-019 — No postinstall scripts

Classification: Supply-Chain Integrity · STRIDE: Tampering · CWE-506 · OWASP A03:2025

F-058 · Unpinned Third-Party GitHub Action

Severity: ● Medium · **Component:** C-03 - CI/CD Pipeline · **Location:** Multiple locations (3)

Weakness: W-010 - Build pipeline trusts mutable third-party references

Instances (3): `.github/workflows/ci.yml`: 188; `.github/workflows/image_actions.yml`: 33; `.github/workflows/codeql-analysis.yml`: 23

Issue: Unpinned Third-Party GitHub Action: Tag-based action references (@v3, @main) are mutable - a compromised publisher can retroactively inject malicious code into an already-used tag.

Evidence: ✓ verified

Fix: ◑ M-020 — Third-party GitHub Actions pinned to commit SHA

Classification: Supply-Chain Integrity · STRIDE: Tampering · CWE-829 · OWASP A03:2025

F-059 · Non-reproducible dependency installs in release builds

Severity: ● Medium · **Component:** C-03 - CI/CD Pipeline · **Location:** `.github/workflows/release.yml:55`

Weakness: W-010 - Build pipeline trusts mutable third-party references

Issue: Release packaging runs npm install `--production` and installs global tools by semver range, instead of npm ci from a lockfile. Dependency resolution at release time can differ from reviewed state, so a malicious new version is pulled into released artifacts; `package.json` postinstall runs build scripts on install.

A compromised dependency version could be bundled into signed-off release archives and Docker images.

Evidence: ◌ ambiguous - Line 55 uses npm install while lines 38-39 install global tools from ranges; lockfile enforcement was not shown.

Fix: ◑ M-011 — Manual review: verify Non-reproducible dependency installs in release builds · ◑ M-064 — Commit lockfiles and install from them

Classification: Supply-Chain Integrity · STRIDE: Tampering · CWE-829 · OWASP A03:2025

F-060 · Unpinned floating container base images

Severity: ● Medium · **Component:** C-03 - CI/CD Pipeline · **Location:** Multiple locations (2)

Weakness: W-010 - Build pipeline trusts mutable third-party references

Instances (2): `Dockerfile`: 1; `test/smoke/Dockerfile`: 1

Issue: The production `Dockerfile` builds on `node:24` and a distroless tag without digests, and the smoke `Dockerfile` uses unpinned alpine. A replaced or compromised upstream image enters the published image built in release and master pipelines.

Backdoored base layers would ship in the public Docker Hub image to downstream users.

Evidence: ✓ verified - Line 1 uses the floating tag `node:24` and line 22 the distroless tag without @sha256 digests.

Fix: ◑ M-065 — Pin base images by digest

Classification: Supply-Chain Integrity · STRIDE: Tampering · CWE-829 · OWASP A03:2025

F-087 · Container Runs as Root

Severity: ● Medium · **Component:** C-03 - CI/CD Pipeline · **Location:** `test/smoke/Dockerfile:1`

Issue: Container Runs as Root: Containers running as root expand container-escape blast radius.

Evidence: ✓ verified

Fix: ◑ M-021 — Drop unnecessary privileges in build and runtime

Classification: Supply-Chain Integrity · STRIDE: Elevation of Privilege · CWE-250 · OWASP A03:2025

F-088 · Missing Workflow Permissions Block

Severity: ● Medium · **Component:** C-03 - CI/CD Pipeline · **Location:** Multiple locations (13)

Instances (13): `.github/workflows/ci.yml` : 25; `.github/workflows/frontend-bundle-analysis.yml` : 13; `.github/workflows/image_actions.yml` : 21; `.github/workflows/lint-fixer.yml` : 6; `.github/workflows/rebase.yml` : 8; `.github/workflows/release.yml` : 10; `.github/workflows/stale.yml` : 7; `.github/workflows/update-challenges-ebook.yml` : 11 … (+5 more)

Issue: Missing Workflow Permissions Block: Without an explicit permissions block at workflow or job level, the job inherits the repository default (commonly write-all) for `GITHUB_TOKEN` - any compromised step can push code, create releases, or approve PRs.

Evidence: ✓ verified

Fix: ◑ M-022 — Apply least-privilege permissions

Classification: Supply-Chain Integrity · STRIDE: Elevation of Privilege · CWE-732 · OWASP A03:2025

---

9. Abuse Cases

Abuse cases connect findings into attack scenarios. The verdict records whether repository evidence confirms the path. Combined risk is the highest policy-rated risk of the linked findings in the §8 Findings Register; verification adds no severity. Within each verification group, higher-risk cases appear first. Equal risks use the strongest finding's assessed reach, impact and likelihood. Case IDs remain stable and do not indicate priority.

Confirmed attack paths

#	Scenario	Actor	Combined Risk	Verdict
AC-B-003	User takes over another account	attacker	● High	▲ Fully viable
AC-T-002	Bulk Data Exfiltration via Broken Object Authorization	Internet Attacker	● High	▲ Fully viable
AC-T-004	Privilege Escalation via Mass-Assignment on Registration	Internet Attacker	● Critical	▲ Fully viable
AC-T-005	Authentication Bypass via Exposed Secret Material	Internet Attacker	● Critical	▲ Fully viable
AC-T-007	Privileged Action via Prompt Injection into a Tool-Calling Model	Internet Attacker	● Medium	▲ Fully viable
AC-T-006	Remote Code Execution via Server-Side Injection	Internet Attacker	● Critical	▲ Fully viable

#	Scenario	Actor	Combined Risk	Verdict
AC-B-005	User exceeds a limit or a one-time entitlement	attacker	● Medium	▲ Fully viable

Unresolved scenarios

#	Scenario	Actor	Combined Risk	Verdict
AC-T-003	Privilege Escalation to Admin via JWT Algorithm Confusion	Internet Attacker	● High	? Inconclusive
AC-T-001	Account Takeover via XSS + Token Hijacking	Internet Attacker	● High	? Inconclusive
AC-B-004	User bypasses a required approval	attacker	not rated (no linked finding)	? Inconclusive

Verdict: ▲ Fully viable - no effective control blocks this chain · ◐ Partially blocked - at least one step has a compensating control but the chain is not fully closed · ✓ Mitigated - chain is broken at a verified step · ? Inconclusive - could not be verified end-to-end. Step: ✗ Refuted - the matched pairing does not hold on the code.

Blocking mitigations: implementing any single mitigation listed under a case severs the chain at the named step, so the end-to-end abuse can no longer complete.

AC-B-003 — User takes over another account

> **Source:** business abuse case · **Actor:** attacker · **Combined Risk:** ● High · **Verdict:** ▲ Fully viable

Goal:

Attack chain

Step	Finding	Outcome
1	● F-021 — Password reset by security answer only	Check whether anyone other than the verified owner can change an account's credentials or recovery data, such as email, phone number, or reset tokens.

Blocking mitigations

- ◕ M-035 — Require emailed one-time token and rate-limit reset (**P2**): remediating ● F-021 — Password reset by security answer only breaks the chain at **Step 1**, removing the link the rest of the chain depends on.

AC-T-002 — Bulk Data Exfiltration via Broken Object Authorization

> **Source:** technical abuse case · **Actor:** Internet Attacker - authenticated low-privilege user · **Combined Risk:** ● High · **Verdict:** ▲ Fully viable

Goal: Enumerate and exfiltrate other users' records, then escalate own permissions via unguarded mass assignment.

Prerequisite: Attacker holds a valid, non-privileged user account.

Attack chain

Step	Finding	Outcome
1	● F-006 — Systemic IDOR Missing Ownership Checks	Attacker enumerates and retrieves records for arbitrary object IDs; no ownership comparison is performed.
2	● F-038 — User registration accepts privileged fields	Update endpoint persists an unfiltered `role` (or equivalent) field supplied in the request body.

Scenario impact

The ownership gap exposes every record, and the mass-assignment gap lets the same low-privilege actor self-elevate - together they turn a single compromised account into full tenant data access and role escalation.

Blocking mitigations

- ◕ M-028 — Check basket ownership before returning (**P2**): remediating ● F-006 — Systemic IDOR Missing Ownership Checks breaks the chain at **Step 1**, removing the link the rest of the chain depends on.
- ◕ M-047 — Allowlist client-controlled fields (**P2**): remediating ● F-038 — User registration accepts privileged fields breaks the chain at **Step 2**, removing the link the rest of the chain depends on.

AC-T-004 — Privilege Escalation via Mass-Assignment on Registration

> **Source:** technical abuse case · **Actor:** Internet Attacker - unauthenticated external attacker · **Combined Risk:** ● Critical · **Verdict:** ▲ Fully viable

Goal: Obtain an administrator account without any existing privilege.

Prerequisite: Self-registration is open (one unauthenticated POST).

Attack chain

Step	Finding	Outcome
1	● F-038 — User registration accepts privileged fields	The account-creation handler persists the request body wholesale, so a client-supplied `role` (or `isAdmin`) field is written verbatim.

Scenario impact

A single unauthenticated request mints an admin account when the registration handler trusts a client-supplied role field - the most direct full-compromise path in role-based apps with open sign-up.

Blocking mitigations

- ◕ M-047 — Allowlist client-controlled fields (**P2**): remediating ● F-038 — User registration accepts privileged fields breaks the chain at **Step 1**, removing the link the rest of the chain depends on.

AC-T-005 — Authentication Bypass via Exposed Secret Material

> **Source:** technical abuse case · **Actor:** Internet Attacker - unauthenticated external attacker · **Combined Risk:** ● Critical · **Verdict:** ▲ Fully viable

Goal: Forge trusted tokens / credentials and impersonate any user.

Prerequisite: Signing material or other secrets are reachable (committed to a public repo, served by an unauthenticated route, or in an exposed directory).

Attack chain

Step	Finding	Outcome
1	● F-005 — Hardcoded Secrets in Source	A private key, signing secret, or credential file is committed to the source repository

Step	Finding	Outcome
		or served without authentication.
2	● F-015 — Chat identity taken from unverified JWT	The exposed key/secret is the same one the server trusts, so a token signed with it (or the leaked credential) is accepted as authentic.

Scenario impact

Exposed signing material collapses the entire authentication boundary: any attacker who reads the key can mint a valid token for any identity or role, with no credential ever required.

Blocking mitigations

- ◕ M-027 — Load signing keys from a secret store (**P2**): remediating ● F-005 — Hardcoded Secrets in Source breaks the chain at **Step 1**, removing the link the rest of the chain depends on.
- ◕ M-031 — Verify JWT signature (**P2**): remediating ● F-015 — Chat identity taken from unverified JWT breaks the chain at **Step 2**, removing the link the rest of the chain depends on.

AC-T-007 — Privileged Action via Prompt Injection into a Tool-Calling Model

> **Source:** technical abuse case · **Actor:** Internet Attacker - unauthenticated external attacker · **Combined Risk:** ● Medium · **Verdict:** ▲ Fully viable

Goal: Make the model invoke a privileged tool or return data the attacker is not entitled to, using the application's own authority.

Prerequisite: Attacker can place text into a conversation, document, or record that the application later sends to a language model.

Attack chain

Step	Finding	Outcome
1	*no matching finding* `routes/chat.ts:83`	The handler builds the prompt by interpolating request or stored text into the same string as the system instructions,

Step	Finding	Outcome
		so attacker text is read as instruction.
2	● F-086 — Chatbot coupon discount unbounded server-side	Tools exposed to the model run with the application's authority; a manipulated model turn therefore performs actions the caller could not.

Scenario impact

Text an attacker can place anywhere the application later feeds to the model becomes instruction, and the tools behind that model run with the application's authority rather than the caller's - so a comment or a stored record reaches actions the attacker was never entitled to.

Blocking mitigations

- ◑ M-091 — Enforce coupon limits in server code (**P3**): remediating ● F-086 — Chatbot coupon discount unbounded server-side breaks the chain at **Step 2**, removing the link the rest of the chain depends on.

AC-T-006 — Remote Code Execution via Server-Side Injection

> **Source:** technical abuse case · **Actor:** Internet Attacker - unauthenticated external attacker · **Combined Risk:** ● Critical · **Verdict:** ▲ Fully viable

Goal: Execute arbitrary code in the application process.

Prerequisite: An input reaches a server-side interpreter / template / eval.

Attack chain

Step	Finding	Outcome
1	● F-048 — Request data evaluated by notevil in vm sandbox	Attacker-controlled input is passed to `eval`, a server-side template engine, an unsafe sandbox, or an unsafe deserializer.

Scenario impact

A single injection into a server-side interpreter yields code execution in the application process - the highest-impact outcome, granting full filesystem and network access from one unauthenticated request.

Blocking mitigations

- ◑ M-057 — Remove dynamic evaluation of order lines (**P3**): remediating ● F-048 — Request data evaluated by notevil in vm sandbox breaks the chain at **Step 1**, removing the link the rest of the chain depends on.

AC-B-005 — User exceeds a limit or a one-time entitlement

> **Source:** business abuse case · **Actor:** attacker · **Combined Risk:** ● Medium · **Verdict:** ▲ Fully viable

Goal:

Attack chain

Step	Finding	Outcome
1	● F-047 — Wallet top-up accepts unvalidated balance amount	Check whether a one-time or limited entitlement, such as a coupon, voucher, quota, or balance, can be used more often or for more value than allowed, including by repeating or racing the operation.

Blocking mitigations

- ◑ M-056 — Validate wallet top-up amount and charge card (**P3**): remediating ● F-047 — Wallet top-up accepts unvalidated balance amount breaks the chain at **Step 1**, removing the link the rest of the chain depends on.

AC-T-003 — Privilege Escalation to Admin via JWT Algorithm Confusion

> **Source:** technical abuse case · **Actor:** Internet Attacker - unauthenticated external attacker · **Combined Risk:** ● High · **Verdict:** ? Inconclusive

Goal: Forge an admin-role JWT without knowledge of the signing secret.

Prerequisite: Attacker can obtain any valid JWT issued by the system (e.g. by registering a free account).

Attack chain

Step	Finding	Outcome
1	● F-016 — JWT verify call missing algorithms allowlist	Verifier accepts attacker-chosen `alg` (e.g. `none` or HMAC-with-public-key), allowing token re-signing without the secret.
2	*no matching finding*	Forged `role: admin` claim is accepted as authoritative because the role is not re-fetched from the database per request.

Scenario impact

Algorithm confusion alone yields a forgeable token; trusting the in-token role claim turns that forgery into instant admin access - neither gap is Critical in isolation, but the chain is a full authentication bypass.

Blocking mitigations

- ◑ M-003 — Manual review: verify JWT verify call missing algorithms allowlist (**P3**): remediating ● F-016 — JWT verify call missing algorithms allowlist breaks the chain at **Step 1**, removing the link the rest of the chain depends on.
- ◕ M-032 — Enforce JWT signature and algorithm verification (● F-016) (**P2**): remediating ● F-016 — JWT verify call missing algorithms allowlist breaks the chain at **Step 1**, removing the link the rest of the chain depends on.

AC-T-001 — Account Takeover via XSS + Token Hijacking

> **Source:** technical abuse case · **Actor:** Internet Attacker - unauthenticated external attacker · **Combined Risk:** ● High · **Verdict:** ? Inconclusive

Goal: Obtain persistent authenticated access as an arbitrary user without valid credentials.

Prerequisite: Attacker can get markup rendered in a victim's session, through stored content (e.g. feedback, comments, profile fields) or a crafted link.

Attack chain

Step	Finding	Outcome
1	● F-002 — XSS via Trusted HTML Binding	

Step	Finding	Outcome
		Attacker JavaScript executes in the victim's browser session.
2	● F-113 — Session token stored in web-accessible client storage	Token exfiltrated from local/session storage via the Step 1 payload.
3	*no matching finding*	Exfiltrated token accepted for a new session; absence of token binding / PKCE removes the last server-side revocation opportunity.

Scenario impact

Individually the XSS sink and the web-readable token storage rate below Critical, but chained they form a repeatable credential-theft path: a single stored payload causes indefinite session compromise for every user who views the affected page.

Blocking mitigations

- ◕ M-024 — Remove sanitizer bypass and render feedback as text (**P2**): remediating ● F-002 — XSS via Trusted HTML Binding breaks the chain at **Step 1**, removing the link the rest of the chain depends on.
- ◕ M-095 — Store session tokens in HttpOnly cookies (**P2**): remediating ● F-113 — Session token stored in web-accessible client storage breaks the chain at **Step 2**, removing the link the rest of the chain depends on.

AC-B-004 — User bypasses a required approval

> **Source:** business abuse case · **Actor:** attacker · **Combined Risk:** not rated (no linked finding) · **Verdict:** ? Inconclusive

Goal:

Attack chain

Step	Finding	Outcome
1	*no matching finding*	Check whether a user can approve their own request where an independent approval is required, or change content after approval and keep the earlier approval.

Business-case coverage

- 4 business abuse case(s) were not performed: exceeds the limit of 3 business abuse cases at standard depth.

---

10. Mitigation Register

Each mitigation block lists the findings it **Addresses**, the CWEs it **Prevents**, and the **Priority** (P1 = before deployment, P2 = current sprint, P3 = next quarter, P4 = backlog). **Why**, **How**, and **Verification** appear when written; every P1 or P2 mitigation that changes code gives **How** and **Verification**. If a field is omitted, refer to the linked finding's *Evidence* line for file:line context and to the threat-category description in §8 Findings Register for the underlying weakness.

Mitigations index:

● M-023 — Enforce object-level (ownership) authorization (● F-001)
● M-029 — Use parameterized database queries (● F-007)
● M-030 — Parameterize the login query with bind replacements
◕ M-024 — Remove sanitizer bypass and render feedback as text
◕ M-025 — Remove sanitizer bypass and render IP as text
◕ M-026 — Move session token to HttpOnly cookie
◕ M-027 — Load signing keys from a secret store
◕ M-028 — Check basket ownership before returning
◕ M-031 — Verify JWT signature
◕ M-032 — Enforce JWT signature and algorithm verification (● F-016)
◕ M-033 — Require current password for password change
◕ M-034 — Stop deriving OAuth account passwords from the email
◕ M-035 — Require emailed one-time token and rate-limit reset
◕ M-036 — Constrain file paths to a safe base directory (● F-022)
◕ M-037 — Constrain zip entries to the extraction directory
◕ M-038 — Use typed query predicates instead of $where
◕ M-039 — Remove server-side evaluation of untrusted input (● F-027)
◕ M-040 — Encode output instead of bypassing the framework sanitizer (● F-028)
◕ M-041 — Disable public directory listings
◕ M-042 — Disable XML external entity (XXE) resolution
◕ M-043 — Hash passwords with a strong, salted algorithm
◕ M-044 — Remove eval of username in profile rendering
◕ M-045 — Enforce server-side authorization on every endpoint (● F-036)
◕ M-046 — Enforce server-side authorization on every endpoint (● F-037)
◕ M-047 — Allowlist client-controlled fields
◕ M-095 — Store session tokens in HttpOnly cookies
◑ M-001 — Manual review: verify Missing Object Ownership Checks
◑ M-002 — Manual review: verify Angular Sanitizer Bypass XSS
◑ M-003 — Manual review: verify JWT verify call missing algorithms allowlist
◑ M-004 — Manual review: verify Input in executable NoSQL predicate

◑ M-005 — Manual review: verify DOM XSS via document.write of exported user data
◑ M-006 — Manual review: verify Sensitive Routes Registered Without…
◑ M-007 — Manual review: verify Hard-coded cookie signing secret
◑ M-008 — Manual review: verify Client-supplied review author identity
◑ M-009 — Manual review: verify No brute-force protection on login
◑ M-010 — Manual review: verify Unaudited npm Lifecycle Script
◑ M-011 — Manual review: verify Non-reproducible dependency installs in release…
◑ M-012 — Manual review: verify Unsanitised login IP header stored for later…
◑ M-013 — Manual review: verify Unauthenticated memories listing joins user…
◑ M-014 — Manual review: verify Blocking busy-wait sleep reachable from review…
◑ M-015 — Manual review: verify Unbounded disk upload processed before…
◑ M-016 — Manual review: verify Unhandled TypeError from non-string socket…
◑ M-017 — Sign and verify release artifacts
◑ M-018 — Disable untrusted package install scripts (● F-056)
◑ M-019 — No postinstall scripts
◑ M-020 — Third-party GitHub Actions pinned to commit SHA
◑ M-021 — Drop unnecessary privileges in build and runtime
◑ M-022 — Apply least-privilege permissions
◑ M-048 — Derive review author from verified token identity
◑ M-049 — Load the cookie secret from environment config
◑ M-050 — Derive review author from the server session
◑ M-051 — Use authorization code flow with PKCE and state
◑ M-052 — Require authentication on every exposed endpoint (● F-043)
◑ M-053 — Add per-user lockout and shrink the TOTP window
◑ M-054 — Add per-account and per-IP throttling to login
◑ M-055 — Validate redirect targets against an allowlist
◑ M-056 — Validate wallet top-up amount and charge card
◑ M-057 — Remove dynamic evaluation of order lines
◑ M-058 — Add CSRF token and SameSite cookie flags
◑ M-059 — Validate coupons server-side against current time
◑ M-060 — Restrict chat messages to validated user turns
◑ M-061 — Always sanitize product description
◑ M-062 — Sanitize email and username unconditionally
◑ M-063 — Verify or remove curl-pipe Heroku CLI install
◑ M-064 — Commit lockfiles and install from them
◑ M-065 — Pin base images by digest
◑ M-066 — Encode output instead of bypassing the framework sanitizer (● F-061)
◑ M-067 — Reject null bytes before validating file extension
◑ M-068 — Remove JSONP and allowlist returned user fields
◑ M-069 — Return only whitelisted user fields from memories
◑ M-070 — Return uniform response for security question lookup
◑ M-071 — Restrict metrics endpoint access
◑ M-072 — Require authentication or remove public log serving
◑ M-073 — Remove public serving of the encryptionkeys directory
◑ M-074 — Enforce object-level (ownership) authorization (● F-069)
◑ M-075 — Validate and allowlist outbound request targets
◑ M-076 — Encrypt or tokenize card numbers

◑ M-077 — Stop storing sensitive data in cleartext
◑ M-078 — Move cryptographic keys to a managed secret store
◑ M-079 — Send passwords in a POST body
◑ M-080 — Stop exposing internal information to clients (● F-075)
◑ M-081 — Set secure session-cookie attributes
◑ M-082 — Rate-limit expensive requests and bound input size
◑ M-083 — Cap entry count and decompressed size during extraction
◑ M-084 — Add rate limit and token caps to chat
◑ M-085 — Set fileSize limit on disk uploads
◑ M-086 — Authenticate first and cap disk upload size
◑ M-087 — Validate addresses and cap tracked wallet set
◑ M-088 — Validate payload types in socket handlers
◑ M-089 — Validate the same BasketId that is saved
◑ M-090 — Require validated payment mode before deluxe upgrade
◑ M-091 — Enforce coupon limits in server code
◑ M-092 — Enforce authorization on the server (● F-089)
◑ M-093 — Enforce server-side authorization
◑ M-094 — Add token revocation and expiry cleanup

P1 — Immediate

M-023 — Enforce object-level (ownership) authorization (F-001)

Addresses:

- ● F-001 — Missing Object Ownership Checks
- ● F-009 — Broken authorization

Weaknesses addressed: W-002

Prevents CWEs:

- CWE-639 - Authorization Bypass Through User-Controlled Key (IDOR)

Priority: P1 - Immediate · **Effort:** Medium · **File:** `routes/order.ts:148`

How: Replace `req.body.UserId` / `userId` / `ownerId` with `req.user.id` (or equivalent session-derived identity) in every WHERE/filter clause.

1. Add a regression test (or CI check) that fails on the vulnerable pattern and passes once the fix is in place.

Generic CWE-639 pattern, not code from this repository:

```
        // Ownership check before touching a resource.
        const basket = await Basket.findByPk(req.params.id)
        if (!basket || basket.UserId !== req.user.id) return res.status(403).end()
```

Verification: Re-run the AUTHZ-001 scanner check; it reports no match at `routes/order.ts:148`, and the regression test passes.

M-029 — Use parameterized database queries (F-007)

Addresses:

- ● F-007 — SQL injection

Weaknesses addressed: W-001

Prevents CWEs:

- CWE-89 - SQL Injection

Priority: P1 - Immediate · **Effort:** Medium · **File:** `routes/search.ts:23`

How: Use parameterized / bound queries (placeholders + a values array, or an ORM's safe query builder); never interpolate request data into the SQL string.

1. Add a regression test (or CI check) that fails on the vulnerable pattern and passes once the fix is in place.

Generic CWE-89 pattern, not code from this repository:

```
// Reject string interpolation; use parameter binding.
await sequelize.query(
  'SELECT * FROM Users WHERE email = :email AND password = :password',
  { replacements: { email, password: hash(password) }, type: QueryTypes.SELECT }
)
```

Verification: Re-run the INJ-001 scanner check; it reports no match at `routes/search.ts:23`, and the regression test passes.

M-030 — Parameterize the login query with bind replacements

Addresses:

- ● F-013 — SQL injection in login query

Weaknesses addressed: W-001

Prevents CWEs:

- CWE-89 - SQL Injection

Priority: P1 - Immediate · **Effort:** Low · **File:** `routes/login.ts:34`

How:

1. Use `sequelize.query` with replacements or bind and a ? placeholder for email and password hash.
2. Add a test that logs in with a quote-containing email and expects 401.

Generic CWE-89 pattern, not code from this repository:

```
// Reject string interpolation; use parameter binding.
await sequelize.query(
  'SELECT * FROM Users WHERE email = :email AND password = :password',
  { replacements: { email, password: hash(password) }, type: QueryTypes.SELECT }
)
```

Verification: POST `/rest/user/login` with email "' OR 1=1--"; expect 401 and no token.

Reference: OWASP Cheat Sheet: SQL Injection Prevention Cheat Sheet

---

P2 — This Sprint

M-024 — Remove sanitizer bypass and render feedback as text

Addresses:

- ● F-002 — XSS via Trusted HTML Binding

Weaknesses addressed: W-006

Prevents CWEs:

- CWE-79 - Cross-site Scripting

Priority: P2 - This Sprint · **Effort:** Low · **File:** `frontend/src/app/about/about.component.ts:119`

How:

1. Render the comment with text interpolation and build the stars markup separately from a fixed list.
2. Add a unit test that a comment containing a script tag is displayed as inert text.

Generic CWE-79 pattern, not code from this repository:

```
// Never call bypassSecurityTrust*; let Angular sanitize.
// template:  <div [innerHTML]="product.description"></div>
// component: no DomSanitizer.bypassSecurityTrustHtml(...)
this.product.description = raw  // bound directly; Angular escapes
```

Verification: Submit feedback containing an img onerror payload and open the about page; no script executes and the payload shows as text.

Reference: CWE-79: Improper Neutralization of Input During Web Page Generation (XSS)

M-025 — Remove sanitizer bypass and render IP as text

Addresses:

- ● F-003 — Angular Sanitizer Bypass XSS

Weaknesses addressed: W-006

Prevents CWEs:

- CWE-79 - Cross-site Scripting

Priority: P2 - This Sprint · **Effort:** Low · **File:** `frontend/src/app/last-login-ip/last-login-ip.component.ts:39`

How:

1. Drop the sanitizer trust call and bind the value as plain text with interpolation.
2. Add a unit test that a claim containing an img onerror payload renders as inert text.

*Example implementation in `frontend/src/app/last-login-ip/last-login-ip.component.ts:39`: it applies **Remove sanitizer bypass and render IP as text**.*

```
        this.lastLoginIp = payload.data.lastLoginIp // template: <dd>{{ lastLoginIp }}</dd>
```

Verification: Set `lastLoginIp` to '`<img src=x onerror=alert(1)>`' in a test token and confirm no alert fires and the text is shown literally.

Reference: CWE-79: Improper Neutralization of Input During Web Page Generation (XSS)

M-026 — Move session token to HttpOnly cookie

Addresses:

- ● F-004 — Session Token Stored in localStorage

Weaknesses addressed: W-005

Prevents CWEs:

- CWE-922 - Insecure Storage of Sensitive Information

Priority: P2 - This Sprint · **Effort:** Medium · **File:** `frontend/src/app/Services/request.interceptor.ts:16`

How:

1. Issue the session token in a Secure, HttpOnly, SameSite cookie from the server and drop `localStorage` use in the interceptor.
2. Add a strict `Content-Security-Policy` to limit script injection impact.

Generic CWE-922 pattern, not code from this repository:

```
// Move the JWT out of localStorage into an httpOnly cookie.
res.cookie('session', token, {
  httpOnly: true, secure: true, sameSite: 'lax', maxAge: 3600_000
})
```

Verification: After login, run `localStorage.getItem('token')` in the browser console; it must return null and requests must still authenticate.

Reference: CWE-922: Insecure Storage of Sensitive Information

---

M-027 — Load signing keys from a secret store

Addresses:

- ● F-005 — Hardcoded Secrets in Source

Weaknesses addressed: W-007

Prevents CWEs:

- CWE-798 - Use of Hard-coded Credentials

Priority: P2 - This Sprint · **Effort:** Medium · **File:** `lib/insecurity.ts:21`

How:

1. Load the private key and HMAC key from environment or a vault and rotate them.
2. Remove the literals from the repository history.

Generic CWE-798 pattern, not code from this repository:

```
// Read secrets from env / vault, not the source.
const hmacKey = process.env.ORDER_HMAC_KEY
if (!hmacKey) throw new Error('ORDER_HMAC_KEY not set')
const sig = createHmac('sha256', hmacKey).update(payload).digest('hex')
```

Verification: Grep the repo for the key literals; expect zero hits.

Reference: CWE-798: Use of Hard-coded Credentials

M-028 — Check basket ownership before returning

Addresses:

- ● F-006 — Systemic IDOR Missing Ownership Checks

Weaknesses addressed: W-002

Prevents CWEs:

- CWE-639 - Authorization Bypass Through User-Controlled Key (IDOR)

Priority: P2 - This Sprint · **Effort:** Low · **File:** `routes/basket.ts:19`

How:

1. Add UserId/bid match to the query and return 403 on mismatch.
2. Test cross-user access is denied.

Generic CWE-639 pattern, not code from this repository:

```
// Ownership check before touching a resource.
const basket = await Basket.findByPk(req.params.id)
if (!basket || basket.UserId !== req.user.id) return res.status(403).end()
```

Verification: GET `/rest/basket/` ; expect 403.

Reference: CWE-639: Authorization Bypass Through User-Controlled Key

---

M-031 — Verify JWT signature

Addresses:

- ● F-015 — Chat identity taken from unverified JWT

Weaknesses addressed: W-008

Prevents CWEs:

- CWE-347 - Improper Verification of Cryptographic Signature

Priority: P2 - This Sprint · **Effort:** Low · **File:** `routes/chat.ts:45`

How:

1. Register `security.isAuthorized()` on `/rest/chat` or verify the token with `jwt.verify` before deriving user id.
2. Add tests that a forged unsigned token receives 401 and cannot read orders.

Generic CWE-347 pattern, not code from this repository:

```
        // Always verify on the public key; never trust the unsigned header.
        const decoded = jwt.verify(token, publicKey, { algorithms: ['RS256'] })
```

Verification: Send a forged alg-none JWT to POST `/rest/chat`; expect HTTP 401.

Reference: CWE-347: Improper Verification of Cryptographic Signature

---

M-032 — Enforce JWT signature and algorithm verification (F-016)

Addresses:

- ● F-016 — JWT verify call missing algorithms allowlist

Weaknesses addressed: W-008

Prevents CWEs:

- CWE-347 - Improper Verification of Cryptographic Signature

Priority: P2 - This Sprint · **Effort:** Medium · **File:** `routes/verify.ts:120`

How: Always pass `algorithms: ['RS256']` (or the project's chosen alg) as the third argument's options object to `jwt.verify`.

1. Add a regression test (or CI check) that fails on the vulnerable pattern and passes once the fix is in place.

Generic CWE-347 pattern, not code from this repository:

```
        // Always verify on the public key; never trust the unsigned header.
        const decoded = jwt.verify(token, publicKey, { algorithms: ['RS256'] })
```

Verification: Re-run the AUTHZ-005 scanner check; it reports no match at `routes/verify.ts:120`, and the regression test passes.

M-033 — Require current password for password change

Addresses:

- ● F-017 — Password change without current password

Prevents CWEs:

- CWE-620

Priority: P2 - This Sprint · **Effort:** Low · **File:** `routes/changePassword.ts:39`

How:

1. Reject requests where current is missing or does not match the stored hash; use POST with a body instead of query parameters.
2. Add a test that omitting current returns 401.

Verification: GET `/rest/user/change-password?new=a&repeat=a` with a valid token; expect HTTP 401.

Reference: CWE-620: Unverified Password Change

M-034 — Stop deriving OAuth account passwords from the email

Addresses:

- ● F-018 — Predictable password derived from OAuth email address

Prevents CWEs:

- CWE-1391

Priority: P2 - This Sprint · **Effort:** Medium · **File:** `frontend/src/app/oauth/oauth.component.ts:30`

How:

1. Verify the Google token on the server and issue the session there, with no client-computed password.
2. If a local credential is needed, generate a random one server-side and never derive it from public identifiers.

Verification: Log in with the password `btoa(reversed email)` for an OAuth account and confirm the server rejects it.

Reference: CWE-1391

M-035 — Require emailed one-time token and rate-limit reset

Addresses:

- ● F-021 — Password reset by security answer only

Prevents CWEs:

- CWE-640 - Weak Password Recovery Mechanism

Priority: P2 - This Sprint · **Effort:** Medium · **File:** `routes/resetPassword.ts:41`

How:

1. Send a single-use expiring reset link to the account email and require it.
2. Rate-limit attempts and notify the owner after a reset.

Verification: Post repeated wrong answers; expect 429, and a correct answer alone must not change the password.

Reference: OWASP Cheat Sheet: Forgot Password Cheat Sheet

M-036 — Constrain file paths to a safe base directory (F-022)

Addresses:

- ● F-022 — Path Traversal

Prevents CWEs:

- CWE-22

Priority: P2 - This Sprint · **Effort:** Medium · **File:** `routes/dataErasure.ts:104`

How: Resolve the path, assert it stays within an allowed base directory (`path.resolve` + `startsWith`), and use `path.basename` on user input.

1. Add a regression test (or CI check) that fails on the vulnerable pattern and passes once the fix is in place.

Verification: Re-run the INJ-004 scanner check; it reports no match at `routes/dataErasure.ts:104`, and the regression test passes.

M-037 — Constrain zip entries to the extraction directory

Addresses:

- ● F-023 — Zip Slip path traversal during archive extraction

Prevents CWEs:

- CWE-22

Priority: P2 - This Sprint · **Effort:** Low · **File:** `routes/fileUpload.ts:34`

How:

1. Resolve each entry against a fixed base directory and reject entries whose relative path starts with .. or is absolute; write to the resolved path.
2. Add a test uploading a zip with ../ entries and assert it is rejected.

Verification: Upload a zip containing ../../ftp/legal.md and confirm no file outside uploads/complaints changes.

Reference: CWE-22: Path Traversal

M-038 — Use typed query predicates instead of $where

Addresses:

- ● F-025 — Input in executable NoSQL predicate
- ● F-026 — Input in executable NoSQL predicate

Weaknesses addressed: W-004

Prevents CWEs:

- CWE-943 - Special-Element Injection in Data Query

Priority: P2 - This Sprint · **Effort:** Medium · **File:** routes/showProductReviews.ts:36

How: Use typed query predicates instead of $where

1. Add a regression test (or CI check) that fails on the vulnerable pattern and passes once the fix is in place.

Generic CWE-943 pattern, not code from this repository:

```
// Reject `$where` and operator keys from user-controlled input.
function safeQuery(filter) {
  for (const k of Object.keys(filter)) {
    if (k.startsWith('$')) throw new Error('operator not allowed')
  }
  return collection.find(filter)
}
```

Verification: Re-run the INJ-NODE-006 scanner check; it reports no match at routes/showProductReviews.ts:36, and the regression test passes.

M-039 — Remove server-side evaluation of untrusted input (F-027)

Addresses:

- ● F-027 — Input compiled as template source

Weaknesses addressed: W-018

Prevents CWEs:

- CWE-1336

Priority: P2 - This Sprint · **Effort:** Medium · **File:** `routes/userProfile.ts:87`

How: Keep input as data; remove dynamic code or template-source compilation

1. Add a regression test (or CI check) that fails on the vulnerable pattern and passes once the fix is in place.

Verification: Re-run the INJ-NODE-008 scanner check; it reports no match at `routes/userProfile.ts:87`, and the regression test passes.

M-040 — Encode output instead of bypassing the framework sanitizer (F-028)

Addresses:

- ● F-028 — DOM XSS

Weaknesses addressed: W-006

Prevents CWEs:

- CWE-79 - Cross-site Scripting

Priority: P2 - This Sprint · **Effort:** Low · **File:** `frontend/src/app/data-export/data-export.component.ts:71`

How:

1. Replace `document.write` with a Blob download of the JSON, or render it with `textContent` in a pre element
2. Add a unit test with a `userData` containing a script tag and assert it is not executed

Generic CWE-79 pattern, not code from this repository:

```
// Never call bypassSecurityTrust*; let Angular sanitize.
// template:  <div [innerHTML]="product.description"></div>
// component: no DomSanitizer.bypassSecurityTrustHtml(...)
this.product.description = raw  // bound directly; Angular escapes
```

Verification: Run a data export where a stored field contains an img onerror payload and confirm no script runs in the opened window.

Reference: CWE-79: Improper Neutralization of Input During Web Page Generation (XSS)

M-041 — Disable public directory listings

Addresses:

- ● F-030 — Unauthenticated log directory listing

Prevents CWEs:

- CWE-548

Priority: P2 - This Sprint · **Effort:** Low · **File:** `server.ts:281`

How:

1. Remove the public `serveIndex` mount for logs, or restrict it to an admin role.
2. Test that anonymous requests to `/support/logs` are rejected.

Verification: curl `/support/logs` without credentials and expect 401 or 403.

Reference: CWE-548: Exposure of Information Through Directory Listing

---

M-042 — Disable XML external entity (XXE) resolution

Addresses:

- ● F-031 — XML external entity expansion in upload parser

Prevents CWEs:

- CWE-611 - XML External Entity (XXE)

Priority: P2 - This Sprint · **Effort:** Low · **File:** `routes/fileUpload.ts:76`

How:

1. Remove `XML_PARSE_NOENT` and `XML_PARSE_DTDLOAD` and do not register filesystem input providers.
2. Do not reflect parsed content in error responses.

Generic CWE-611 pattern, not code from this repository:

```
        // libxmljs2 — disable external entities and network fetch.
        const doc = libxmljs.parseXml(xmlBuf, { noent: false, nonet: true, recover: false })
        if (doc.errors.length) throw new Error('xml rejected')
```

Verification: Upload an XML with an external entity for `/etc/passwd` and confirm no file content appears in the response.

Reference: OWASP Cheat Sheet: XML External Entity Prevention Cheat Sheet

---

M-043 — Hash passwords with a strong, salted algorithm

Addresses:

- ● F-032 — Passwords hashed with unsalted fast MD5

Weaknesses addressed: W-041

Prevents CWEs:

- CWE-916

Priority: P2 - This Sprint · **Effort:** Medium · **File:** `models/user.ts:76`

How:

1. Replace `security.hash` in the password setter with a salted adaptive hash such as argon2id or bcrypt.
2. Migrate existing hashes by rehashing on next successful login and add tests for the new format.

Generic CWE-916 pattern, not code from this repository:

```
// Replace md5/sha1 password hashes with bcrypt or argon2.
import bcrypt from 'bcrypt'
const hash = await bcrypt.hash(plaintext, 12)
const ok = await bcrypt.compare(plaintext, storedHash)
```

Verification: Create a user and inspect the `Users.password` column; it must be an argon2/bcrypt string, not a 32-char hex `MD5`.

Reference: OWASP Cheat Sheet: Password Storage Cheat Sheet

M-044 — Remove eval of username in profile rendering

Addresses:

- ● F-033 — Code injection

Weaknesses addressed: W-003

Prevents CWEs:

- CWE-95

Priority: P2 - This Sprint · **Effort:** Low · **File:** `routes/userProfile.ts:61`

How:

1. Delete the eval branch and pass the username to Pug as an escaped variable instead of string substitution
2. Add a test that a #{7*7} username renders literally

Verification: Set username #{7*7}, GET `/profile`, and confirm 49 is not rendered.

Reference: CWE-95

M-045 — Enforce server-side authorization on every endpoint (F-036)

Addresses:

- ● F-036 — Missing authorization on product update route

Weaknesses addressed: W-002

Prevents CWEs:

- CWE-862 - Missing Authorization

Priority: P2 - This Sprint · **Effort:** Low · **File:** `server.ts:370`

How:

1. Uncomment or add `app.put('/api/Products/:id', security.isAuthorized())` before `finale.initialize`, or use `denyAll`.
2. Add a test that an unauthenticated PUT returns 401.

Generic CWE-862 pattern, not code from this repository:

```
// Add server-side role check on every admin route.
router.use('/admin/*', (req, res, next) => {
  if (req.user?.role !== 'admin') return res.status(403).end()
  next()
})
```

Verification: curl -X PUT `/api/Products/1` without a token and expect HTTP 401.

Reference: CWE-862: Missing Authorization

M-046 — Enforce server-side authorization on every endpoint (F-037)

Addresses:

- ● F-037 — Sensitive Routes Registered Without Authentication Middleware

Weaknesses addressed: W-002

Prevents CWEs:

- CWE-862 - Missing Authorization

Priority: P2 - This Sprint · **Effort:** Medium · **File:** `server.ts:598`

How: Add an explicit auth middleware (`isAuthorized()` / `passport.authenticate()` / `requireAuth`) to the route registration, or mark the route intentionally public via `app.get(...)` for read-only access only.

1. Add a regression test (or CI check) that fails on the vulnerable pattern and passes once the fix is in place.

Generic CWE-862 pattern, not code from this repository:

```
// Add server-side role check on every admin route.
router.use('/admin/*', (req, res, next) => {
  if (req.user?.role !== 'admin') return res.status(403).end()
  next()
})
```

Verification: Re-run the AUTHZ-008 scanner check; it reports no match at `server.ts:598`, and the regression test passes.

M-047 — Allowlist client-controlled fields

Addresses:

- ● F-038 — User registration accepts privileged fields

Weaknesses addressed: W-023

Prevents CWEs:

- CWE-915

Priority: P2 - This Sprint · **Effort:** Low · **File:** `server.ts:503`

How:

1. Strip role and other privileged fields from the body before creation.
2. Test registration with role admin yields customer.

Generic CWE-915 pattern, not code from this repository:

```
// Whitelist mutable fields server-side; never trust the body.
const ALLOWED = ['email', 'username'] as const
const patch = Object.fromEntries(
  Object.entries(req.body).filter(([k]) => ALLOWED.includes(k as any))
```

```
        )
        await user.update(patch)
```

Verification: POST `/api/Users` with role admin; response role is customer.

Reference: CWE-915: Improperly Controlled Modification of Dynamically-Determined Object Attributes (Mass Assignment)

M-095 — Store session tokens in HttpOnly cookies

Addresses:

- ● F-113 — Session token stored in web-accessible client storage

Prevents CWEs:

- CWE-922 - Insecure Storage of Sensitive Information

Priority: P2 - This Sprint · **Effort:** Medium · **File:** `frontend/src/app/Services/basket.service.ts:64`

How: Store session tokens in HttpOnly cookies

1. Add a regression test (or CI check) that fails on the vulnerable pattern and passes once the fix is in place.

Generic CWE-922 pattern, not code from this repository:

```
        // Move the JWT out of localStorage into an httpOnly cookie.
        res.cookie('session', token, {
          httpOnly: true, secure: true, sameSite: 'lax', maxAge: 3600_000
        })
```

Verification: Re-run the security scan; the CWE-922 finding at `frontend/src/app/Services/basket.service.ts:64` is cleared, and the regression test passes.

P3 — Next Quarter

M-001 — Manual review: verify Missing Object Ownership Checks at order.ts: 148

Addresses:

- ● F-001 — Missing Object Ownership Checks

Weaknesses addressed: W-002

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `routes/order.ts:148`

How: The evidence-verifier sample could not confirm or refute the claim from the cited snippet alone. Have a developer familiar with this code path read ±20 lines around the cited location (`routes/order.ts:148`) and decide whether to keep, downgrade, or remove this finding.

1. Replace `req.body.UserId` / `userId` / `ownerId` with `req.user.id` (or equivalent session-derived identity) in every WHERE/filter clause.
2. Add a regression test (or CI check) that fails on the vulnerable pattern and passes once the fix is in place.

Generic CWE-639 pattern, not code from this repository:

```
        // Ownership check before touching a resource.
        const basket = await Basket.findByPk(req.params.id)
        if (!basket || basket.UserId !== req.user.id) return res.status(403).end()
```

Verification: Re-run the AUTHZ-001 scanner check; it reports no match at `routes/order.ts:148` , and the regression test passes.

M-002 — Manual review: verify Angular Sanitizer Bypass XSS

Addresses:

- ● F-003 — Angular Sanitizer Bypass XSS

Weaknesses addressed: W-006

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `frontend/src/app/last-login-ip/last-login-ip.component.ts:39`

How: The evidence-verifier sample could not confirm or refute the claim from the cited snippet alone. Have a developer familiar with this code path read ±20 lines around the cited location (`frontend/src/app/last-login-ip/last-login-ip.component.ts:39`) and decide whether to keep, downgrade, or remove this finding.

1. Drop the sanitizer trust call and bind the value as plain text with interpolation.
2. Add a unit test that a claim containing an img onerror payload renders as inert text.

*Example implementation in `frontend/src/app/last-login-ip/last-login-ip.component.ts:39` : it applies **Manual review: verify Angular Sanitizer Bypass XSS**.*

```
        this.lastLoginIp = payload.data.lastLoginIp // template: <dd>{{ lastLoginIp }}</dd>
```

Verification: Set `lastLoginIp` to '`<img src=x onerror=alert(1)>`' in a test token and confirm no alert fires and the text is shown literally.

Reference: CWE-79: Improper Neutralization of Input During Web Page Generation (XSS)

M-003 — Manual review: verify JWT verify call missing algorithms allowlist

Addresses:

- ● F-016 — JWT verify call missing algorithms allowlist

Weaknesses addressed: W-008

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `routes/verify.ts:120`

How: The evidence-verifier sample could not confirm or refute the claim from the cited snippet alone. Have a developer familiar with this code path read ±20 lines around the cited location (`routes/verify.ts:120`) and decide whether to keep, downgrade, or remove this finding.

1. Always pass `algorithms: ['RS256']` (or the project's chosen alg) as the third argument's options object to `jwt.verify`.
2. Add a regression test (or CI check) that fails on the vulnerable pattern and passes once the fix is in place.

Generic CWE-347 pattern, not code from this repository:

```
        // Always verify on the public key; never trust the unsigned header.
        const decoded = jwt.verify(token, publicKey, { algorithms: ['RS256'] })
```

Verification: Re-run the AUTHZ-005 scanner check; it reports no match at `routes/verify.ts:120`, and the regression test passes.

M-004 — Manual review: verify Input in executable NoSQL predicate

Addresses:

- ● F-025 — Input in executable NoSQL predicate

Weaknesses addressed: W-004

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `routes/showProductReviews.ts:36`

How: The evidence-verifier sample could not confirm or refute the claim from the cited snippet alone. Have a developer familiar with this code path read ±20 lines around the cited location (`routes/showProductReviews.ts:36`) and decide whether to keep, downgrade, or remove this finding.

1. Use typed query predicates instead of $where
2. Add a regression test (or CI check) that fails on the vulnerable pattern and passes once the fix is in place.

Generic CWE-943 pattern, not code from this repository:

```
// Reject `$where` and operator keys from user-controlled input.
function safeQuery(filter) {
  for (const k of Object.keys(filter)) {
    if (k.startsWith('$')) throw new Error('operator not allowed')
  }
  return collection.find(filter)
}
```

Verification: Re-run the INJ-NODE-006 scanner check; it reports no match at `routes/showProductReviews.ts:36`, and the regression test passes.

M-005 — Manual review: verify DOM XSS via document.write of exported user data

Addresses:

- ● F-028 — DOM XSS

Weaknesses addressed: W-006

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `frontend/src/app/data-export/data-export.component.ts:71`

How: The evidence-verifier sample could not confirm or refute the claim from the cited snippet alone. Have a developer familiar with this code path read ±20 lines around the cited location (`frontend/src/app/data-export/data-export.component.ts:71`) and decide whether to keep, downgrade, or remove this finding.

1. Replace `document.write` with a Blob download of the JSON, or render it with `textContent` in a pre element
2. Add a unit test with a `userData` containing a script tag and assert it is not executed

Generic CWE-79 pattern, not code from this repository:

```
// Never call bypassSecurityTrust*; let Angular sanitize.
// template:  <div [innerHTML]="product.description"></div>
// component: no DomSanitizer.bypassSecurityTrustHtml(...)
this.product.description = raw  // bound directly; Angular escapes
```

Verification: Run a data export where a stored field contains an img onerror payload and confirm no script runs in the opened window.

Reference: CWE-79: Improper Neutralization of Input During Web Page Generation (XSS)

M-006 — Manual review: verify Sensitive Routes Registered Without Authentication Middleware

Addresses:

- ● F-037 — Sensitive Routes Registered Without Authentication Middleware

Weaknesses addressed: W-002

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `server.ts:598`

How: The evidence-verifier sample could not confirm or refute the claim from the cited snippet alone. Have a developer familiar with this code path read ±20 lines around the cited location (`server.ts:598`) and decide whether to keep, downgrade, or remove this finding.

1. Add an explicit auth middleware (`isAuthorized()` / `passport.authenticate()` / `requireAuth`) to the route registration, or mark the route intentionally public via `app.get(...)` for read-only access only.
2. Add a regression test (or CI check) that fails on the vulnerable pattern and passes once the fix is in place.

Generic CWE-862 pattern, not code from this repository:

```
// Add server-side role check on every admin route.
router.use('/admin/*', (req, res, next) => {
  if (req.user?.role !== 'admin') return res.status(403).end()
  next()
})
```

Verification: Re-run the AUTHZ-008 scanner check; it reports no match at `server.ts:598` , and the regression test passes.

M-007 — Manual review: verify Hard-coded cookie signing secret at server.ts: 289

Addresses:

- ● F-040 — Hard-coded cookie signing secret

Weaknesses addressed: W-007

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `server.ts:289`

How: The evidence-verifier sample could not confirm or refute the claim from the cited snippet alone. Have a developer familiar with this code path read ±20 lines around the cited location (`server.ts:289`) and decide whether to keep, downgrade, or remove this finding.

1. Read the cookie secret from an environment variable or secret store.
2. Rotate the secret and add a startup check that it is set.

Generic CWE-798 pattern, not code from this repository:

```
        // Read secrets from env / vault, not the source.
        const hmacKey = process.env.ORDER_HMAC_KEY
        if (!hmacKey) throw new Error('ORDER_HMAC_KEY not set')
        const sig = createHmac('sha256', hmacKey).update(payload).digest('hex')
```

Verification: grep the source for 'kekse' and expect no hits.

Reference: CWE-798: Use of Hard-coded Credentials

M-008 — Manual review: verify Client-supplied review author identity

Addresses:

- ● F-041 — Client-supplied review author identity

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `frontend/src/app/product-details/product-details.component.ts:73`

How: The evidence-verifier sample could not confirm or refute the claim from the cited snippet alone. Have a developer familiar with this code path read ±20 lines around the cited location (`frontend/src/app/product-details/product-details.component.ts:73`) and decide whether to keep, downgrade, or remove this finding.

1. Remove the author field from the client request and derive it from the verified token on the server.
2. Add a test that a forged author in the body is ignored or rejected.

Verification: Send a review request with a different author value and confirm the stored author equals the authenticated user.

Reference: CWE-290: Authentication Bypass by Spoofing

M-009 — Manual review: verify No brute-force protection on login at login.ts:32

Addresses:

- ● F-045 — No brute-force protection on login

Weaknesses addressed: W-012

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `routes/login.ts:32`

How: The evidence-verifier sample could not confirm or refute the claim from the cited snippet alone. Have a developer familiar with this code path read ±20 lines around the cited location (`routes/login.ts:32`) and decide whether to keep, downgrade, or remove this finding.

1. Apply `express-rate-limit` and an account lockout or backoff to POST `/rest/user/login`.
2. Add a test that the sixth failed attempt returns 429.

Generic CWE-307 pattern, not code from this repository:

```
// Per-IP + per-account rate limiting on auth endpoints.
import rateLimit from 'express-rate-limit'
app.use('/rest/user/login',
  rateLimit({ windowMs: 60_000, max: 5, standardHeaders: true }))
```

Verification: Send 20 bad logins quickly; expect 429 after the configured limit.

Reference: CWE-307: Improper Restriction of Excessive Authentication Attempts

---

M-010 — Manual review: verify Unaudited npm Lifecycle Script at package.json: 56

Addresses:

- ● F-057 — Unaudited npm Lifecycle Script

Weaknesses addressed: W-024

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `package.json:56`

How: The evidence-verifier sample could not confirm or refute the claim from the cited snippet alone. Have a developer familiar with this code path read ±20 lines around the cited location (`package.json:56`) and decide whether to keep, downgrade, or remove this finding.

1. Remove postinstall if possible; else document why required and use `--ignore-scripts` in CI and audit the hook content
2. Add a regression test (or CI check) that fails on the vulnerable pattern and passes once the fix is in place.

Verification: Re-run the IAC-051 scanner check; it reports no match at `package.json:56`, and the regression test passes.

M-011 — Manual review: verify Non-reproducible dependency installs in release builds

Addresses:

- ● F-059 — Non-reproducible dependency installs in release builds

Weaknesses addressed: W-010

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `.github/workflows/release.yml:55`

How: The evidence-verifier sample could not confirm or refute the claim from the cited snippet alone. Have a developer familiar with this code path read ±20 lines around the cited location (`.github/workflows/release.yml:55`) and decide whether to keep, downgrade, or remove this finding.

1. Commit the lockfile and use npm ci in `release.yml`, the `Dockerfile` and `ci.yml` jobs.
2. Pin global CLI tools to exact versions and consider a minimum release age setting.

Verification: grep -rn `'npm install'` `.github/workflows/release.yml` `Dockerfile` returns only exact-version global installs.

Reference: docs.npmjs.com: Npm Ci

M-012 — Manual review: verify Unsanitised login IP header stored for later rendering

Addresses:

- ● F-061 — Unsanitised login IP header stored for later

Weaknesses addressed: W-006

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `routes/saveLoginIp.ts:32`

How: The evidence-verifier sample could not confirm or refute the claim from the cited snippet alone. Have a developer familiar with this code path read ±20 lines around the cited location (`routes/saveLoginIp.ts:32`) and decide whether to keep, downgrade, or remove this finding.

1. Validate `lastLoginIp` as an IP address and drop the header otherwise, in every configuration.
2. Test that a header with HTML is rejected or escaped.

Generic CWE-79 pattern, not code from this repository:

```
        // Never call bypassSecurityTrust*; let Angular sanitize.
        // template:  <div [innerHTML]="product.description"></div>
```

```
        // component: no DomSanitizer.bypassSecurityTrustHtml(...)
        this.product.description = raw  // bound directly; Angular escapes
```

Verification: Send True-Client-IP with an iframe payload; the stored `lastLoginIp` must not contain markup.

Reference: CWE-79: Improper Neutralization of Input During Web Page Generation (XSS)

M-013 — Manual review: verify Unauthenticated memories listing joins user records

Addresses:

- ● F-064 — Unauthenticated memories listing joins user records

Weaknesses addressed: W-029

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `routes/memory.ts:24`

How: The evidence-verifier sample could not confirm or refute the claim from the cited snippet alone. Have a developer familiar with this code path read ±20 lines around the cited location (`routes/memory.ts:24`) and decide whether to keep, downgrade, or remove this finding.

1. Select only needed attributes for the included User.
2. Add a test asserting no email or password fields appear.

Generic CWE-200 pattern, not code from this repository:

```
        // Remove directory-listing middleware; require auth on management endpoints.
        // app.use('/ftp', serveIndex(...))   // delete
        app.use('/metrics', requireRole('admin'), promBundle())
```

Verification: GET `/rest/memories` and confirm user objects contain only the whitelisted fields.

Reference: CWE-200: Exposure of Sensitive Information to an Unauthorized Actor

---

M-014 — Manual review: verify Blocking busy-wait sleep reachable from review query

Addresses:

- ● F-077 — Blocking busy-wait sleep reachable from review

Weaknesses addressed: W-013

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `routes/showProductReviews.ts:23`

How: The evidence-verifier sample could not confirm or refute the claim from the cited snippet alone. Have a developer familiar with this code path read ±20 lines around the cited location (`routes/showProductReviews.ts:23`) and decide whether to keep, downgrade, or remove this finding.

1. Delete `global.sleep` and the $where query
2. Add rate limiting to the reviews route

Verification: Send repeated `sleep(2000)` injection requests and confirm other requests stay responsive.

Reference: CWE-400: Uncontrolled Resource Consumption

M-015 — Manual review: verify Unbounded disk upload processed before authentication

Addresses:

- ● F-081 — Unbounded disk upload processed before authentication

Weaknesses addressed: W-016

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `server.ts:698`

How: The evidence-verifier sample could not confirm or refute the claim from the cited snippet alone. Have a developer familiar with this code path read ±20 lines around the cited location (`server.ts:698`) and decide whether to keep, downgrade, or remove this finding.

1. Add `limits.fileSize` to `uploadToDisk` and run authorization before multer.
2. Add per-user quotas and rate limiting.

Verification: POST an oversized image unauthenticated to `/rest/memories` and expect 401 or 413.

Reference: CWE-770

M-016 — Manual review: verify Unhandled TypeError from non-string socket payloads

Addresses:

- ● F-083 — Unhandled TypeError from non-string socket

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `lib/startup/registerWebsocketEvents.ts:46`

How: The evidence-verifier sample could not confirm or refute the claim from the cited snippet alone. Have a developer familiar with this code path read ±20 lines around the cited location (`lib/startup/registerWebsocketEvents.ts:46`) and decide whether to keep, downgrade, or remove this finding.

1. Check typeof data === 'string' before match and contains calls, and wrap handlers in try/catch.
2. Add tests emitting numbers and objects to each handler.

Verification: Emit `verifySvgInjectionChallenge` with the number 1; the server must stay running.

Reference: CWE-248

M-017 — Sign and verify release artifacts

Addresses:

- ● F-054 — Missing Container Image Signing

Weaknesses addressed: W-008

Prevents CWEs:

- CWE-347 - Improper Verification of Cryptographic Signature

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `.github/workflows/ci.yml:1`

How: Add cosign signing step to release workflow; or use actions/attest-build-provenance - Unsigned container images cannot be verified for provenance - any registry intermediary could substitute them.

1. Add cosign signing step to release workflow; or use actions/attest-build-provenance
2. Add a regression test (or CI check) that fails on the vulnerable pattern and passes once the fix is in place.

Generic CWE-347 pattern, not code from this repository:

```
        // Always verify on the public key; never trust the unsigned header.
        const decoded = jwt.verify(token, publicKey, { algorithms: ['RS256'] })
```

Verification: Re-run the IAC-040 scanner check; it reports no match at `.github/workflows/ci.yml:1` , and the regression test passes.

M-018 — Disable untrusted package install scripts (F-056)

Addresses:

- ● F-056 — Dependency Install Scripts Run at Build Time

Prevents CWEs:

- CWE-506

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `Dockerfile:4`

How: Use `npm ci --ignore-scripts` and audit required postinstalls separately - Without `--ignore-scripts`, any compromised dependency publisher can execute postinstall code at build time.

1. Use `npm ci --ignore-scripts` and audit required postinstalls separately
2. Add a regression test (or CI check) that fails on the vulnerable pattern and passes once the fix is in place.

Verification: Re-run the IAC-005 scanner check; it reports no match at `Dockerfile:4`, and the regression test passes.

M-019 — No postinstall scripts

Addresses:

- ● F-057 — Unaudited npm Lifecycle Script

Weaknesses addressed: W-024

Prevents CWEs:

- CWE-506

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `package.json:56`

How: Remove postinstall if possible; else document why required and use `--ignore-scripts` in CI and audit the hook content - postinstall hooks execute arbitrary code at install time - the attack surface for supply-chain compromise.

1. Remove postinstall if possible; else document why required and use `--ignore-scripts` in CI and audit the hook content
2. Add a regression test (or CI check) that fails on the vulnerable pattern and passes once the fix is in place.

Verification: Re-run the IAC-051 scanner check; it reports no match at `package.json:56`, and the regression test passes.

M-020 — Third-party GitHub Actions pinned to commit SHA

Addresses:

- ● F-058 — Unpinned Third-Party GitHub Action

Weaknesses addressed: W-010

Prevents CWEs:

- CWE-829

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `.github/workflows/ci.yml:188`

How: Pin every third-party action to a full 40-char commit SHA - Tag-based action references (@v3, @main) are mutable - a compromised publisher can retroactively inject malicious code into an already-used tag.

1. Pin every third-party action to a full 40-char commit SHA
2. Add a regression test (or CI check) that fails on the vulnerable pattern and passes once the fix is in place.

Verification: Re-run the IAC-011 scanner check; it reports no match at `.github/workflows/ci.yml:188`, and the regression test passes.

M-021 — Drop unnecessary privileges in build and runtime

Addresses:

- ● F-087 — Container Runs as Root

Prevents CWEs:

- CWE-250

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `test/smoke/Dockerfile:1`

How: Add USER `<non-root-uid>` - Containers running as root expand container-escape blast radius.

1. Add USER `<non-root-uid>`
2. Add a regression test (or CI check) that fails on the vulnerable pattern and passes once the fix is in place.

Verification: Re-run the IAC-002 scanner check; it reports no match at `test/smoke/Dockerfile:1`, and the regression test passes.

M-022 — Apply least-privilege permissions

Addresses:

- ● F-088 — Missing Workflow Permissions Block

Prevents CWEs:

- CWE-732

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `.github/workflows/ci.yml:25`

How: Add `permissions: { contents: read }` at workflow root; elevate per-job only where needed - Without an explicit permissions block at workflow or job level, the job inherits the repository default (commonly write-all) for `GITHUB_TOKEN` - any compromised step can push code, create releases, or approve PRs.

1. Add `permissions: { contents: read }` at workflow root; elevate per-job only where needed
2. Add a regression test (or CI check) that fails on the vulnerable pattern and passes once the fix is in place.

Verification: Re-run the IAC-010 scanner check; it reports no match at `.github/workflows/ci.yml:25`, and the regression test passes.

M-048 — Derive review author from verified token identity

Addresses:

- ● F-039 — Unauthenticated review creation

Prevents CWEs:

- CWE-290

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `routes/createProductReviews.ts:26`

How:

1. Register `security.isAuthorized()` on the route and set author from the verified user email, ignoring `body.author`
2. Add a test that a mismatched author is rejected or overwritten

Verification: PUT a review with a different author in the body; expect stored author equals caller or HTTP 401 without a token.

Reference: CWE-290: Authentication Bypass by Spoofing

M-049 — Load the cookie secret from environment config

Addresses:

- ● F-040 — Hard-coded cookie signing secret

Weaknesses addressed: W-007

Prevents CWEs:

- CWE-798 - Use of Hard-coded Credentials

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `server.ts:289`

How:

1. Read the cookie secret from an environment variable or secret store.
2. Rotate the secret and add a startup check that it is set.

Generic CWE-798 pattern, not code from this repository:

```
// Read secrets from env / vault, not the source.
const hmacKey = process.env.ORDER_HMAC_KEY
if (!hmacKey) throw new Error('ORDER_HMAC_KEY not set')
const sig = createHmac('sha256', hmacKey).update(payload).digest('hex')
```

Verification: grep the source for 'kekse' and expect no hits.

Reference: CWE-798: Use of Hard-coded Credentials

M-050 — Derive review author from the server session

Addresses:

- ● F-041 — Client-supplied review author identity

Prevents CWEs:

- CWE-290

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `frontend/src/app/product-details/product-details.component.ts:73`

How:

1. Remove the author field from the client request and derive it from the verified token on the server.
2. Add a test that a forged author in the body is ignored or rejected.

Verification: Send a review request with a different author value and confirm the stored author equals the authenticated user.

Reference: CWE-290: Authentication Bypass by Spoofing

M-051 — Use authorization code flow with PKCE and state

Addresses:

- ● F-042 — OAuth implicit flow without state or nonce

Weaknesses addressed: W-021

Prevents CWEs:

- CWE-352 - Cross-Site Request Forgery (CSRF)

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `frontend/src/app/login/login.component.ts:148`

How:

1. Switch to authorization code flow with PKCE and generate/verify a random state value.
2. URL-encode `redirect_uri` and add a test rejecting responses with a mismatched state.

Generic CWE-352 pattern, not code from this repository:

```
        // Use double-submit cookie / SameSite=Strict for state-changing routes.
        app.use(csrf({ cookie: { sameSite: 'strict', httpOnly: true, secure: true } }))
```

Verification: Replay a callback with a wrong state and confirm login is refused.

Reference: datatracker.ietf.org: Rfc9700

M-052 — Require authentication on every exposed endpoint (F-043)

Addresses:

- ● F-043 — Unauthenticated WebSocket Channel

Weaknesses addressed: W-009

Prevents CWEs:

- CWE-306

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `lib/startup/registerWebsocketEvents.ts:23`

How:

1. Add `io.use` middleware that verifies a session token or JWT in the handshake and rejects unauthenticated sockets.
2. Add tests that unauthenticated connections are refused.

Verification: Connect with a socket.io client without a token; the connection must fail with an auth error.

Reference: CWE-306: Missing Authentication for Critical Function

---

M-053 — Add per-user lockout and shrink the TOTP window

Addresses:

- ● F-044 — Weak 2FA code guessing bounds

Weaknesses addressed: W-011

Prevents CWEs:

- CWE-307 - Improper Restriction of Excessive Authentication Attempts

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `routes/2fa.ts:31`

How:

1. Count failed codes per `userId` and lock or back off after a few failures.
2. Reduce `epochTolerance` and shorten `tmpToken` lifetime.

Generic CWE-307 pattern, not code from this repository:

```
// Per-IP + per-account rate limiting on auth endpoints.
import rateLimit from 'express-rate-limit'
app.use('/rest/user/login',
  rateLimit({ windowMs: 60_000, max: 5, standardHeaders: true }))
```

Verification: Submit 10 wrong codes for one user from different IPs; expect lockout.

Reference: CWE-307: Improper Restriction of Excessive Authentication Attempts

---

M-054 — Add per-account and per-IP throttling to login

Addresses:

- ● F-045 — No brute-force protection on login

Weaknesses addressed: W-012

Prevents CWEs:

- CWE-307 - Improper Restriction of Excessive Authentication Attempts

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `routes/login.ts:32`

How:

1. Apply `express-rate-limit` and an account lockout or backoff to POST `/rest/user/login`.
2. Add a test that the sixth failed attempt returns 429.

Generic CWE-307 pattern, not code from this repository:

```
// Per-IP + per-account rate limiting on auth endpoints.
import rateLimit from 'express-rate-limit'
app.use('/rest/user/login',
  rateLimit({ windowMs: 60_000, max: 5, standardHeaders: true }))
```

Verification: Send 20 bad logins quickly; expect 429 after the configured limit.

Reference: CWE-307: Improper Restriction of Excessive Authentication Attempts

---

M-055 — Validate redirect targets against an allowlist

Addresses:

- ● F-046 — Substring redirect allowlist check

Weaknesses addressed: W-040

Prevents CWEs:

- CWE-601

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `lib/insecurity.ts:136`

How:

1. Compare the full URL with `Set.has`, or parse it and compare origin and path exactly.
2. Add a test with `https://evil.example/?https://github.com/juice-shop/juice-shop`.

Verification: Call `isRedirectAllowed` with an evil URL embedding an allowed URL; expect false.

Reference: CWE-601: URL Redirection to Untrusted Site (Open Redirect)

M-056 — Validate wallet top-up amount and charge card

Addresses:

- ● F-047 — Wallet top-up accepts unvalidated balance amount

Prevents CWEs:

- CWE-20

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `routes/wallet.ts:27`

How:

1. Require a positive finite number within limits and reject others with 400
2. Process the card charge before incrementing, with tests for negative and huge values

Verification: PUT balance -100 and confirm a 400 response with no balance change.

Reference: CWE-20: Improper Input Validation

M-057 — Remove dynamic evaluation of order lines

Addresses:

- ● F-048 — Request data evaluated by notevil in vm sandbox

Weaknesses addressed: W-003

Prevents CWEs:

- CWE-95

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `routes/b2bOrder.ts:23`

How:

1. Parse `orderLinesData` as JSON and validate with a schema instead of evaluating it.
2. Add tests that expressions are rejected.

Verification: POST `orderLinesData` containing a JS expression; expect HTTP 400.

Reference: CWE-95

M-058 — Add CSRF token and SameSite cookie flags

Addresses:

- ● F-049 — Cookie-authenticated profile update

Weaknesses addressed: W-022

Prevents CWEs:

- CWE-352 - Cross-Site Request Forgery (CSRF)

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `routes/updateUserProfile.ts:38`

How:

1. Require an anti-CSRF token and set `SameSite=Strict`, HttpOnly on the token cookie
2. Add a test that cross-origin form posts are rejected

Generic CWE-352 pattern, not code from this repository:

```
// Use double-submit cookie / SameSite=Strict for state-changing routes.
app.use(csrf({ cookie: { sameSite: 'strict', httpOnly: true, secure: true } }))
```

Verification: Submit a cross-origin POST `/profile` without the token and confirm 403.

Reference: CWE-352: Cross-Site Request Forgery (CSRF)

M-059 — Validate coupons server-side against current time

Addresses:

- ● F-050 — Client-supplied coupon data accepted without server-side

Prevents CWEs:

- CWE-602

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `routes/order.ts:196`

How:

1. Remove the `couponData` path or require a server-issued, stored coupon and compare against the current server time.
2. Add tests that expired campaign codes are rejected.

Verification: Submit `couponData` for WMNSDY2019 and expect no discount.

Reference: CWE-602: Client-Side Enforcement of Server-Side Security

---

M-060 — Restrict chat messages to validated user turns

Addresses:

- ● F-051 — Unvalidated client chat history passed to model

Prevents CWEs:

- CWE-1427

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `routes/chat.ts:206`

How:

1. Validate messages with zod: only role 'user' (and server-stored assistant turns), bounded length and count.
2. Test that role 'system' or 'tool' messages are rejected.

Verification: POST `/rest/chat` with a system role message; expect HTTP 400.

Reference: OWASP GenAI: LLM01 Prompt Injection

M-061 — Always sanitize product description

Addresses:

- ● F-052 — Product description stored without sanitization

Weaknesses addressed: W-025

Prevents CWEs:

- CWE-116

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `models/product.ts:45`

How:

1. Apply `security.sanitizeSecure` unconditionally in the description setter and keep challenge detection separate.
2. Add a test that an iframe javascript payload is stripped on save.

Verification: Create a product with an iframe javascript: description; the stored value must have the iframe removed.

Reference: OWASP Cheat Sheet: Cross Site Scripting Prevention Cheat Sheet

---

M-062 — Sanitize email and username unconditionally

Addresses:

- ● F-053 — User email stored without sanitization

Weaknesses addressed: W-026

Prevents CWEs:

- CWE-116

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `models/user.ts:60`

How:

1. Call `security.sanitizeSecure` on email and username regardless of challenge flags.
2. Add tests that HTML payloads in email and username are neutralized.

Verification: Register with an iframe payload in the email; the stored email must not contain the iframe.

Reference: OWASP Cheat Sheet: Cross Site Scripting Prevention Cheat Sheet

---

M-063 — Verify or remove curl-pipe Heroku CLI install

Addresses:

- ● F-055 — Remote install script piped to shell in deploy job

Prevents CWEs:

- CWE-494

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `.github/workflows/ci.yml:358`

How:

1. Drop the CLI install step since the pinned heroku-deploy action does not need it, or install a pinned version with checksum verification.
2. Run deploy steps with least-privilege tokens.

Verification: grep -n `'curl .*| *sh'` `.github/workflows/ci.yml` returns no match.

Reference: CWE-494

M-064 — Commit lockfiles and install from them

Addresses:

- ● F-059 — Non-reproducible dependency installs in release builds

Weaknesses addressed: W-010

Prevents CWEs:

- CWE-829

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `.github/workflows/release.yml:55`

How:

1. Commit the lockfile and use npm ci in `release.yml`, the `Dockerfile` and `ci.yml` jobs.
2. Pin global CLI tools to exact versions and consider a minimum release age setting.

Verification: grep -rn `'npm install'` `.github/workflows/release.yml` `Dockerfile` returns only exact-version global installs.

Reference: docs.npmjs.com: Npm Ci

M-065 — Pin base images by digest

Addresses:

- ● F-060 — Unpinned floating container base images

Weaknesses addressed: W-010

Prevents CWEs:

- CWE-829

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `Dockerfile:1`

How:

1. Pin node, distroless and alpine base images with sha256 digests and update them via Renovate or Dependabot.
2. Scan built images in CI before pushing.

Verification: grep -nE '^FROM ' `Dockerfile` `test/smoke/Dockerfile` shows every line containing @sha256.

Reference: OWASP: Www Project Top 10 Ci Cd Security Risks

M-066 — Encode output instead of bypassing the framework sanitizer (F-061)

Addresses:

- ● F-061 — Unsanitised login IP header stored for later

Weaknesses addressed: W-006

Prevents CWEs:

- CWE-79 - Cross-site Scripting

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `routes/saveLoginIp.ts:32`

How:

1. Validate `lastLoginIp` as an IP address and drop the header otherwise, in every configuration.
2. Test that a header with HTML is rejected or escaped.

Generic CWE-79 pattern, not code from this repository:

```
// Never call bypassSecurityTrust*; let Angular sanitize.
// template:  <div [innerHTML]="product.description"></div>
// component: no DomSanitizer.bypassSecurityTrustHtml(...)
this.product.description = raw  // bound directly; Angular escapes
```

Verification: Send True-Client-IP with an iframe payload; the stored `lastLoginIp` must not contain markup.

Reference: CWE-79: Improper Neutralization of Input During Web Page Generation (XSS)

M-067 — Reject null bytes before validating file extension

Addresses:

- ● F-062 — Null byte extension allowlist bypass in FTP file

Prevents CWEs:

- CWE-158

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `routes/fileServer.ts:28`

How:

1. Reject names containing null bytes or percent-encoded nulls and validate the extension on the final name
2. Add a test requesting a .bak file with an appended null byte and .md suffix

Verification: GET `/ftp/package.json.bak%2500.md`; expect HTTP 403.

Reference: CWE-158

M-068 — Remove JSONP and allowlist returned user fields

Addresses:

- ● F-063 — JSONP and arbitrary field selection on current user

Weaknesses addressed: W-028

Prevents CWEs:

- CWE-200 - Exposure of Sensitive Information

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `routes/currentUser.ts:58`

How:

1. Remove the callback/res.jsonp branch and restrict fields to an explicit allowlist excluding password
2. Add tests that `fields=password` returns no hash

Generic CWE-200 pattern, not code from this repository:

```
        // Remove directory-listing middleware; require auth on management endpoints.
        // app.use('/ftp', serveIndex(...))   // delete
        app.use('/metrics', requireRole('admin'), promBundle())
```

Verification: Request `/rest/user/whoami?fields=password` and ?`callback=x`; expect no password and a plain JSON response.

Reference: CWE-200: Exposure of Sensitive Information to an Unauthorized Actor

---

M-069 — Return only whitelisted user fields from memories

Addresses:

- ● F-064 — Unauthenticated memories listing joins user records

Weaknesses addressed: W-029

Prevents CWEs:

- CWE-200 - Exposure of Sensitive Information

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `routes/memory.ts:24`

How:

1. Select only needed attributes for the included User.
2. Add a test asserting no email or password fields appear.

Generic CWE-200 pattern, not code from this repository:

```
        // Remove directory-listing middleware; require auth on management endpoints.
        // app.use('/ftp', serveIndex(...))   // delete
        app.use('/metrics', requireRole('admin'), promBundle())
```

Verification: GET `/rest/memories` and confirm user objects contain only the whitelisted fields.

Reference: CWE-200: Exposure of Sensitive Information to an Unauthorized Actor

---

M-070 — Return uniform response for security question lookup

Addresses:

- ● F-065 — Security question disclosure

Prevents CWEs:

- CWE-204

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `routes/securityQuestion.ts:15`

How:

1. Return an identical response regardless of account existence
2. Add rate limiting to the endpoint

Verification: Query a valid and an invalid email and confirm identical responses.

Reference: CWE-204

M-071 — Restrict metrics endpoint access

Addresses:

- ● F-066 — Unauthenticated metrics endpoint

Weaknesses addressed: W-009

Prevents CWEs:

- CWE-306

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `server.ts:676`

How:

1. Require authentication or bind metrics to an internal interface
2. Test that anonymous requests receive 401/403

Verification: curl `/metrics` without credentials returns 401 or 403.

Reference: CWE-306: Missing Authentication for Critical Function

---

M-072 — Require authentication or remove public log serving

Addresses:

- ● F-067 — Unauthenticated log directory listing and download

Prevents CWEs:

- CWE-532

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `server.ts:281`

How:

1. Remove the `/support/logs` routes or restrict to an admin role.
2. Test anonymous access is denied.

Verification: Anonymous GET `/support/logs` returns 401/404.

Reference: CWE-532: Insertion of Sensitive Information into Log File

---

M-073 — Remove public serving of the encryptionkeys directory

Addresses:

- ● F-068 — Unauthenticated encryption key directory exposure

Weaknesses addressed: W-037

Prevents CWEs:

- CWE-538

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `routes/keyServer.ts:14`

How:

1. Remove the `serveIndex` and file routes for `/encryptionkeys`.
2. Move key material outside web-reachable paths.

Verification: GET `/encryptionkeys` returns 404 and no key file is retrievable.

Reference: CWE-538

M-074 — Enforce object-level (ownership) authorization (F-069)

Addresses:

- ● F-069 — Order history lookup by vowel-masked email

Weaknesses addressed: W-002

Prevents CWEs:

- CWE-639 - Authorization Bypass Through User-Controlled Key (IDOR)

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `routes/orderHistory.ts:17`

How:

1. Store the user id with each order and query by it from the authenticated token.
2. Add a test with two users whose emails differ only in vowels.

Generic CWE-639 pattern, not code from this repository:

```
// Ownership check before touching a resource.
const basket = await Basket.findByPk(req.params.id)
if (!basket || basket.UserId !== req.user.id) return res.status(403).end()
```

Verification: Log in as the lookalike account and expect an empty order history.

Reference: CWE-639: Authorization Bypass Through User-Controlled Key

---

M-075 — Validate and allowlist outbound request targets

Addresses:

- ● F-070 — Server-side request forgery

Prevents CWEs:

- CWE-918 - Server-Side Request Forgery (SSRF)

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `routes/profileImageUrlUpload.ts:24`

How:

1. Parse the URL and allow only https to an allow-listed host set; resolve DNS and reject private, loopback and link-local addresses.
2. Validate the fetched content is an image and stop storing the raw URL on failure.

Generic CWE-918 pattern, not code from this repository:

```
// Allowlist external image hosts; block private IP ranges.
const ALLOWED_HOSTS = new Set(['images.example.com', 'cdn.example.com'])
const url = new URL(input)
if (!ALLOWED_HOSTS.has(url.hostname)) throw new Error('host not allowed')
if (/^(10\.|172\.(1[6-9]|2[0-9]|3[01])\.|192\.168\.|127\.)/.test(url.hostname))
  throw new Error('private range blocked')
```

Verification: POST `imageUrl` = `http://127.0.0.1/` and a metadata address and expect rejection with no fetch.

Reference: OWASP: Server Side Request Forgery

M-076 — Encrypt or tokenize card numbers

Addresses:

- ● F-071 — Payment card numbers stored in plaintext

Weaknesses addressed: W-034

Prevents CWEs:

- CWE-311

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `models/card.ts:38`

How:

1. Store a payment-provider token or an encrypted value plus last four digits instead of the full number.
2. Change the column to a string type and add tests ensuring raw numbers are never persisted.

Verification: Insert a card and read the SQLite row; the `cardNum` column must not contain the plain number.

Reference: CWE-311: Missing Encryption of Sensitive Data

M-077 — Stop storing sensitive data in cleartext

Addresses:

- ● F-072 — TOTP secrets stored in plaintext

Weaknesses addressed: W-036

Prevents CWEs:

- CWE-312

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `models/user.ts:112`

How:

1. Add a setter and getter encrypting `totpSecret` with AES-GCM using a key held outside the database.

2. Add a migration encrypting existing values and a test confirming ciphertext at rest.

Verification: Enable 2FA for a user and inspect `Users.totpSecret`; it must be ciphertext, not a base32 secret.

Reference: CWE-312

M-078 — Move cryptographic keys to a managed secret store

Addresses:

- ● F-073 — Security answers HMAC with hardcoded key

Weaknesses addressed: W-007

Prevents CWEs:

- CWE-321 - Use of Hard-coded Cryptographic Key

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `models/securityAnswer.ts:45`

How:

1. Store answers using a salted adaptive hash (argon2id/bcrypt) after normalization instead of a static-key HMAC.
2. Remove the hardcoded key from source and add a test that equal answers yield different stored values.

Generic CWE-321 pattern, not code from this repository:

```
// Load the RSA private key from an environment variable / KMS — never
// the source tree. Rotate the prior key and revoke outstanding tokens.
const privateKey = process.env.JWT_PRIVATE_KEY
if (!privateKey) throw new Error('JWT_PRIVATE_KEY not set')
const token = jwt.sign(claims, privateKey, { algorithm: 'RS256' })
```

Verification: Save the same answer for two users; the stored values must differ.

Reference: CWE-321: Use of Hard-coded Cryptographic Key

M-079 — Send passwords in a POST body

Addresses:

- ● F-074 — Passwords sent in GET query string unencoded

Weaknesses addressed: W-038

Prevents CWEs:

- CWE-598

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `frontend/src/app/Services/user.service.ts:54`

How:

1. Change the call to POST with the passwords in the JSON body and update the server route accordingly.
2. Add a test asserting no credentials appear in request URLs.

Verification: Inspect the network request for password change; the URL must contain no password values.

Reference: CWE-598: Use of GET Request Method With Sensitive Query Strings

---

M-080 — Stop exposing internal information to clients (F-075)

Addresses:

- ● F-075 — Full user record with password hash in JWT

Weaknesses addressed: W-030

Prevents CWEs:

- CWE-200 - Exposure of Sensitive Information

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `routes/login.ts:24`

How:

1. Sign a minimal claim set and look up the user server-side.
2. Test that decoded tokens contain no password or `totpSecret`.

Generic CWE-200 pattern, not code from this repository:

```
        // Remove directory-listing middleware; require auth on management endpoints.
        // app.use('/ftp', serveIndex(...))   // delete
        app.use('/metrics', requireRole('admin'), promBundle())
```

Verification: Decode a fresh token; expect no password or `totpSecret` field.

Reference: CWE-200: Exposure of Sensitive Information to an Unauthorized Actor

---

M-081 — Set secure session-cookie attributes

Addresses:

- ● F-076 — Session token cookie without security attributes

Weaknesses addressed: W-027

Prevents CWEs:

- CWE-1004

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `lib/insecurity.ts:192`

How:

1. Call `res.cookie` with `httpOnly` true, secure true and `sameSite` strict.
2. Test that `Set-Cookie` includes all three flags.

Verification: Inspect the `Set-Cookie` header; expect HttpOnly, Secure and SameSite.

Reference: CWE-1004

M-082 — Rate-limit expensive requests and bound input size

Addresses:

- ● F-077 — Blocking busy-wait sleep reachable from review

Weaknesses addressed: W-013

Prevents CWEs:

- CWE-400 - Uncontrolled Resource Consumption

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `routes/showProductReviews.ts:23`

How:

1. Delete `global.sleep` and the $where query
2. Add rate limiting to the reviews route

Verification: Send repeated `sleep(2000)` injection requests and confirm other requests stay responsive.

Reference: CWE-400: Uncontrolled Resource Consumption

M-083 — Cap entry count and decompressed size during extraction

Addresses:

- ● F-078 — Unbounded zip extraction output size

Prevents CWEs:

- CWE-409

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `routes/fileUpload.ts:34`

How:

1. Track cumulative bytes written and abort past a fixed limit; cap the number of entries.
2. Add a zip-bomb test asserting rejection.

Verification: Upload a zip expanding beyond the limit and confirm extraction aborts with an error.

Reference: CWE-409

M-084 — Add rate limit and token caps to chat

Addresses:

- ● F-079 — Unbounded unauthenticated chat consumption

Weaknesses addressed: W-014

Prevents CWEs:

- CWE-770

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `routes/chat.ts:203`

How:

1. Add `rateLimit`, body size and message-count limits, and `maxOutputTokens`.
2. Test oversized histories are rejected.

Verification: Send 100 requests quickly; expect 429.

Reference: OWASP GenAI: LLM102025 Unbounded Consumption

M-085 — Set fileSize limit on disk uploads

Addresses:

- ● F-080 — Disk upload without size limit

Weaknesses addressed: W-015

Prevents CWEs:

- CWE-770

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `server.ts:698`

How:

1. Add limits: { `fileSize` } to `uploadToDisk`.
2. Test oversized upload is rejected.

Verification: Upload a file above the limit; expect HTTP 413.

Reference: CWE-770

M-086 — Authenticate first and cap disk upload size

Addresses:

- ● F-081 — Unbounded disk upload processed before authentication

Weaknesses addressed: W-016

Prevents CWEs:

- CWE-770

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `server.ts:698`

How:

1. Add `limits.fileSize` to `uploadToDisk` and run authorization before multer.
2. Add per-user quotas and rate limiting.

Verification: POST an oversized image unauthenticated to `/rest/memories` and expect 401 or 413.

Reference: CWE-770

M-087 — Validate addresses and cap tracked wallet set

Addresses:

- ● F-082 — Unbounded wallet set growth without validation

Prevents CWEs:

- CWE-770

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `routes/web3Wallet.ts:16`

How:

1. Validate `walletAddress` with ethers `isAddress` and reject others
2. Cap the set size and add rate limiting to the route

Verification: Send more than the cap of distinct addresses; expect 429 or 400 and bounded memory.

Reference: CWE-770

M-088 — Validate payload types in socket handlers

Addresses:

- ● F-083 — Unhandled TypeError from non-string socket

Prevents CWEs:

- CWE-248

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `lib/startup/registerWebsocketEvents.ts:46`

How:

1. Check typeof data === 'string' before match and contains calls, and wrap handlers in try/catch.
2. Add tests emitting numbers and objects to each handler.

Verification: Emit `verifySvgInjectionChallenge` with the number 1; the server must stay running.

Reference: CWE-248

M-089 — Validate the same BasketId that is saved

Addresses:

- ● F-084 — Basket ID check bypass

Prevents CWEs:

- CWE-235

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `routes/basketItems.ts:42`

How:

1. Reject duplicate keys and use one parsed value for both check and save.
2. Test duplicate BasketId payload is rejected.

Verification: Send duplicate BasketId keys; expect 401/400.

Reference: CWE-235

M-090 — Require validated payment mode before deluxe upgrade

Addresses:

- ● F-085 — Deluxe upgrade without payment for unrecognized payment

Prevents CWEs:

- CWE-840

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `routes/deluxe.ts:43`

How:

1. Reject any `paymentMode` other than wallet or card and complete payment before role update
2. Add a test that an unknown `paymentMode` returns 400 and leaves role unchanged

Verification: POST `paymentMode`=free; expect HTTP 400 and role still customer.

Reference: CWE-840

M-091 — Enforce coupon limits in server code

Addresses:

- ● F-086 — Chatbot coupon discount unbounded server-side

Weaknesses addressed: W-002

Prevents CWEs:

- CWE-862 - Missing Authorization

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `routes/chat.ts:184`

How:

1. Use `z.number().max(10)`, require authenticated user and verified order in execute.
2. Test discount 50 is rejected.

Generic CWE-862 pattern, not code from this repository:

```
// Add server-side role check on every admin route.
router.use('/admin/*', (req, res, next) => {
  if (req.user?.role !== 'admin') return res.status(403).end()
  next()
})
```

Verification: Call tool with discount 50; expect validation error.

Reference: OWASP GenAI: LLM062025 Excessive Agency

M-092 — Enforce authorization on the server (F-089)

Addresses:

- ● F-089 — Client-side role guards trust unverified JWT claims

Weaknesses addressed: W-020

Prevents CWEs:

- CWE-602

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `frontend/src/app/app.guard.ts:54`

How:

1. Verify on the server that every admin and accounting API validates the JWT signature and role.
2. Treat Angular guards as UX only, and add tests that call the APIs directly with a forged token.

Verification: Send a request with a forged-role token to the admin APIs and confirm a 401 or 403 response.

Reference: CWE-602: Client-Side Enforcement of Server-Side Security

---

M-093 — Enforce server-side authorization

Addresses:

- ● F-090 — Any authenticated user can list all accounts

Weaknesses addressed: W-002

Prevents CWEs:

- CWE-285

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `routes/authenticatedUsers.ts:12`

How:

1. Add a role check for admin before the handler and narrow returned fields.
2. Test that a customer token gets 403.

Verification: Call with a customer token; expect 403.

Reference: CWE-285: Improper Authorization

M-094 — Add token revocation and expiry cleanup

Addresses:

- ● F-091 — Sessions cannot be revoked

Prevents CWEs:

- CWE-613

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `lib/insecurity.ts:74`

How:

1. Add a logout route that deletes the token and prune expired entries.
2. Test that a revoked token is rejected.

Verification: Log out then reuse the token; expect 401.

Reference: CWE-613: Insufficient Session Expiration

P4 — Backlog

No P4 mitigations.

---

11. Out of Scope

Not Covered by This Method

An automated, AI-assisted threat model built from the repository's source code and configuration - static analysis only, no dynamic or penetration testing. It models the system as built rather than as designed. It does not replace a threat-modeling session with the team: business context, design intent, runtime behaviour and production configuration are not covered, and each finding still needs confirmation in the deployed system.

- Design intent and the reasoning behind it - no design documents, ADRs or workshop context are read.
- Business processes and user journeys that leave the code.
- Runtime behaviour, deployment topology and production-only configuration.
- External and organizational controls.

Treat this report as review input, not sign-off.

Excluded from This Assessment

The following items are **explicitly excluded** from this threat model. Findings against these areas should be tracked separately.

- Third-party hosted dependencies and SaaS endpoints
- Browser runtime vulnerabilities and end-user device security
- Operating system kernel and container runtime
- Underlying network infrastructure (DNS, BGP, ISP)
- Physical security of hosting facilities
- Opt-in threat actors, not assessed: malicious insiders with repository or pipeline access; malicious insiders with production access; attackers holding a user's device. Enable them with `enable:` in `.appsec/actors.yaml`.

Appendix: Run Statistics

Field	Value
Invocation	`/appsec-advisor:appsec-analyzer --assessment-depth standard --hard-budget 30`
Generated	2026-10-17 19:17 UTC
Mode	full
Assessment depth	standard
Plugin version	0.6.0-beta.5 (analysis v5)
Reasoning models	

Field	Value
	sonnet-economy - STRIDE claude-sonnet-5-5, triage claude-sonnet-5-5, merger claude-sonnet-5-5
Repository	`/home/mrohr/juice-shop`
Output directory	`/tmp/137`
Wall clock (active)	34m 11s
Agent compute (Σ parallel dispatches)	58m 10s

Per-Stage Breakdown

Stage	Description	Agent	Model	Duration	Tool calls	Tokens
2	Reconnaissance: recon scanner (1 job, 3 model calls)	`recon_scanner`	-	0m 23s	0	0
3	Architecture: actor discoverer (1 job, 1 model calls)	`actor_discoverer`	-	0m 06s	0	0
3	Architecture: architecture analyst (1 job, 2 model calls)	`architecture_analyst`	-	0m 55s	0	0
7	Trust Boundaries: trust boundary analyst (16 jobs, 23 model calls)	`trust_boundary_analyst`	-	3m 44s	0	0
8	Controls: control analyst (9 jobs, 13 model calls)	`control_analyst`	-	2m 56s	0	0
9	STRIDE: evidence verifier (21 jobs, 31 model calls)	`evidence_verifier`	-	15m 10s	0	0
9	STRIDE: stride analyzer (30 jobs, 33 model calls)	`stride_analyzer`	-	30m 23s	0	0
10	Scan Synthesis: post stride synthesizer (1 job, 1 model calls)	`post_stride_synthesizer`	-	0m 08s	0	0
10	Scan Synthesis: threat merger (2 jobs, 2 model calls)	`threat_merger`	-	0m 28s	0	0
11		`abuse_case_verifier`	-	1m 31s	0	0

Stage	Description	Agent	Model	Duration	Tool calls	Tokens
	Finalization: abuse case verifier (10 jobs, 20 model calls)					
11	Finalization: ms renderer (1 job, 1 model calls)	`ms_renderer`	-	0m 34s	0	0
11	Finalization: secarch renderer (1 job, 1 model calls)	`secarch_renderer`	-	1m 45s	0	0
Total	-	-	-	**58m 10s**	**0**	**0**

Appendix A — Attack Vector Taxonomy

This appendix defines the attacker-starting-position labels used in the Top Threats table and throughout §8 Findings Register. Each label answers the question *what does the attacker need before the exploit begins?*

Internet Anon

Attacker position: Unauthenticated attacker from the public internet · **Breach distance:** 1

Preconditions:

- Endpoint is reachable from the internet (no IP allowlist, no VPN)
- No authentication middleware blocks the request

Typical CWEs: CWE-89 · CWE-79 · CWE-306 · CWE-327 · CWE-611 · CWE-918

Typical OWASP Top 10: A01:2025, A05:2025, A07:2025

Internet User

Attacker position: Any authenticated low-privilege user (valid JWT / session) · **Breach distance:** 2

Preconditions:

- Attacker has signed up or otherwise obtained a valid user session
- Endpoint is behind auth but not behind role/admin checks

Typical CWEs: CWE-434 · CWE-611 · CWE-918 · CWE-352 · CWE-287

Typical OWASP Top 10: A01:2025, A06:2025, A02:2025

Internet Priv User

Attacker position: Authenticated admin-level user (JWT with admin role or equivalent) · **Breach distance:** 2

Preconditions:

- Attacker holds admin credentials or has elevated privileges
- Endpoint gated on admin role but still exploitable once reached

Typical CWEs: CWE-862 · CWE-79 · CWE-94

Typical OWASP Top 10: A01:2025

Victim-Required

Attacker position: Attacker needs victim interaction - social engineering, crafted link, or live session · **Breach distance:** 2

Preconditions:

- Victim must click a link, load a page, or have an active session
- Applies to XSS, CSRF, click-jacking, open redirect

Typical CWEs: CWE-79 · CWE-352 · CWE-601 · CWE-1021

Typical OWASP Top 10: A01:2025, A05:2025

Build-Time

Attacker position: Attacker controls a build input - CI runner, dependency, base image, or external data fetched during build · **Breach distance:** 3

Preconditions:

- Compromise of a dependency, registry, or base image
- OR compromise of a CI runner with write access to artifacts

Typical CWEs: CWE-506 · CWE-829 · CWE-1039 · CWE-1104

Typical OWASP Top 10: A08:2025

Repo-Read

Attacker position: Attacker gains read access to source repository (leaked clone, forked fork, insider, compromised developer workstation) · **Breach distance:** 3

Preconditions:

- Read access to the source tree at or after commit time
- No runtime exploit needed - the vulnerability is the content of the repo

Typical CWEs: CWE-798 · CWE-312 · CWE-540

Typical OWASP Top 10: A04:2025, A07:2025

Architectural (no runtime entry point)

Attacker position: Architectural / meta-finding - no runtime entry point, the finding describes a design defect aggregating multiple code-level findings

Preconditions:

- Finding is AF-NNN (architectural) rather than F-NNN (code-level)